



Université de Tunis El Manar – Faculté des Sciences de Tunis

Département Informatique

En partenariat avec **Tunisie Telecom**

Rapport de Projet de Fin d'Études

Pour l'obtention du diplôme de
Licence Nationale en Informatique

Sécurisation des flux de communication pour un écosystème IoT simulé

Architecture E2E – PKI – TLS/mTLS – SIEM – ML

Réalisé par :

Amine GHANNOUCHI

Encadrante universitaire : Mme. **ELLOUZE NOURHENE** – FST

Encadrant entreprise : M. Moez **KHLIFI** – Tunisie Telecom

Année universitaire **2025 – 2026**

Dédicace

*À mes parents, pour leur soutien indéfectible
et leur confiance tout au long de ce parcours.*

*À mes enseignants de la Faculté des Sciences de Tunis
qui m'ont transmis la passion de l'informatique.*

*À tous ceux qui œuvrent pour la sécurité
des systèmes d'information de demain.*

Remerciements

Ce travail n'aurait pu aboutir sans le concours de nombreuses personnes à qui je tiens à exprimer ma gratitude.

Je remercie tout d'abord **Mme. ELLOUZE NOURHENE**, encadrante universitaire à la Faculté des Sciences de Tunis, pour sa disponibilité, ses précieux conseils et son suivi rigoureux tout au long de ce projet.

Mes remerciements vont également à **M. Moez KHLIFI** de *Tunisie Telecom* pour m'avoir accueilli au sein de son équipe, orienté vers des problématiques réelles de l'industrie télécom et soutenu dans mes expérimentations techniques.

Je remercie l'ensemble du corps enseignant du **Département Informatique de la FST** pour la qualité de la formation dispensée.

Un grand merci aux équipes des projets open-source **Wazuh**, **HashiCorp Vault**, **Mosquitto** et **Suricata** dont les documentations exhaustives ont grandement facilité l'implémentation.

Enfin, je remercie ma famille et mes amis pour leur encouragement constant durant cette année académique.

Table des matières

Dédicace	i
Remerciements	iii
Liste des acronymes	xii
Introduction générale	1
1 Contexte général du projet	2
1.1 Introduction	2
1.2 Cadre général du projet	2
1.3 L'IoT dans les réseaux de télécommunications	2
1.3.1 Croissance et enjeux du marché IoT	2
1.3.2 Rôle des opérateurs télécom dans l'écosystème IoT	3
1.3.3 Approches de sécurisation disponibles pour les opérateurs	4
1.4 Présentation de l'organisme d'accueil	4
1.4.1 Service d'accueil	5
1.4.2 Positionnement de Tunisie Telecom face aux enjeux IoT	6
1.5 Problématique	6
1.6 Objectifs spécifiques	7
1.7 Périmètre et contraintes du projet	8
1.7.1 Périmètre fonctionnel	8
1.7.2 Contraintes du projet	8
1.7.3 Hypothèses de travail	9
1.8 Démarche méthodologique	9
1.9 Conclusion	10
2 État de l'art	11
2.1 Introduction	11
2.2 Menaces et vulnérabilités spécifiques à l'IoT	11
2.2.1 Surface d'attaque étendue	11
2.2.2 OWASP IoT Top 10 et menaces réseau	11
2.2.3 Études de cas de référence	12
2.3 Protocoles applicatifs IoT	12
2.4 Sécurisation du transport : TLS et mTLS	13
2.4.1 TLS 1.3	13
2.4.2 Authentification mutuelle	13

2.5	Gestion des identités : PKI et automatisation	14
2.6	Détection d'intrusion et supervision	14
2.6.1	IDS réseau orienté MQTT	14
2.6.2	SIEM Wazuh	15
2.7	Apprentissage automatique pour la détection d'anomalies	16
2.8	Analyse critique de l'existant	17
2.9	Synthèse du chapitre	17
2.10	Conclusion	17
3	Conception	18
3.1	Introduction	18
3.2	Diagnostic de l'existant	18
3.2.1	État des lieux de la sécurité IoT dans les déploiements opérateur	18
3.2.2	Synthèse du diagnostic	19
3.3	Spécification des besoins	20
3.3.1	Besoins fonctionnels (BF)	20
3.3.2	Besoins non fonctionnels (BNF)	20
3.3.3	Diagramme de cas d'utilisation	21
3.4	Vue d'ensemble de la solution proposée	23
3.5	Planification des sprints	25
3.6	Backlog produit	26
3.7	Modélisation UML	27
3.7.1	Diagramme de classes du simulateur	27
3.7.2	Diagramme de séquence : émission d'un certificat client	29
3.7.3	Diagramme d'activité : cycle de publication mTLS	31
3.8	Planification temporelle	36
3.9	Conclusion	36
4	Réalisation	37
4.1	Introduction	37
4.2	Environnement de développement	37
4.3	Architecture globale et topologie réseau	38
4.3.1	Architecture de déploiement	38
4.3.2	Topologie réseau	40
4.4	Infrastructure à clés publiques (Vault)	47
4.4.1	Hierarchie	47
4.4.2	Rôles Vault et émission	47
4.5	Broker MQTT en TLS 1.3 + mTLS	49
4.6	Scénarios d'attaque et contre-mesures	51
4.7	Pipeline d'apprentissage automatique	55

4.7.1	Données et features	60
4.8	Captures d'écran des interfaces	61
4.9	Conclusion	68
5	Tests, résultats et validation	69
5.1	Introduction	69
5.2	Plan de tests	69
5.3	Résultats de sécurité	69
5.3.1	Validation par l'exécution : preuves visuelles	70
5.4	Résultats du module ML	77
5.4.1	Isolation Forest	77
5.4.2	Random Forest	79
5.5	Résultats de performance	81
5.5.1	Handshake TLS et latence	81
5.5.2	Débit et RTT	82
5.6	Limites identifiées	83
5.7	Perspectives	83
5.8	Conclusion	84
	Conclusion générale	85
	Références bibliographiques	87

Table des figures

1.1	Logo de Tunisie Telecom	4
1.2	Organigramme de Tunisie Telecom	5
3.1	Cas d'utilisation — Partie 1 : Dispositif IoT (authentification, publication, renouvellement)	21
3.2	Cas d'utilisation — Partie 2 : Administrateur sécurité (PKI, ACL, IDS, SIEM)	22
3.3	Cas d'utilisation — Partie 3 : Opérateur TT/SOC (dashboard, alertes, ML, performances)	22
3.4	Vue d'ensemble de l'architecture de sécurisation proposée. <i>Abréviations : GW = passerelle applicative, FW = pare-feu.</i>	24
3.5	Enchaînement des six sprints et leurs dépendances	25
3.6	Diagramme de classes — Partie 1 : orchestration (<code>FleetOrchestrator</code> , <code>DeviceSimulator</code> , <code>DeviceStats</code>)	28
3.7	Diagramme de classes — Partie 2 : configuration, PKI et dépendances externes (<code>BrokerConfig</code> , <code>CertificateManager</code> , <code>CertBundle</code>)	29
3.8	Séquence PKI — Partie 1 : initialisation (CA racine, CA intermédiaire, rôle)	30
3.9	Séquence PKI — Partie 2 : émission d'un certificat client (~87 ms)	31
3.10	Activité — Phase 1 : initialisation des 50 threads de la flotte	32
3.11	Activité — Phase 2 : provisionnement du certificat et handshake mTLS	33
3.12	Activité — Phase 3 : boucle de publication MQTT et agrégation des résultats	35
3.13	Diagramme de Gantt du projet	36
4.1	Architecture de déploiement — Partie 1 : infrastructure réseau (pfSense, MikroTik), stack IoT simulée et stack Broker/IDS (Mosquitto, Suricata)	39
4.2	Architecture de déploiement — Partie 2 : stack PKI (Vault), stack SIEM (Wazuh) et stack ML/Analyse + interfaces de données	40
4.3	Topologie réseau — Partie 1 : infrastructure réseau, pfSense, MikroTik et segmentation en 4 VLANs	41
4.4	Topologie réseau — Partie 2 : VLAN 10 Zone IoT (fleet-sim, 50 devices) et VLAN 20 Zone DMZ (Mosquitto, Suricata)	42
4.5	Topologie réseau — Partie 3 : VLAN 30 Zone PKI (Vault) et VLAN 40 Zone SIEM (Wazuh, ML Engine) + flux de données	43

4.6	Topologie réseau — Partie 4 : politique de filtrage inter-VLAN (ACEPT/DROP) sur pfSense	44
4.7	Flux E2E — Phase 1 : provisionnement de l'identité	44
4.8	Flux E2E — Phase 2 : établissement de la connexion TLS 1.3 mTLS	45
4.9	Flux E2E — Phase 3 : publication des données IoT	45
4.10	Flux E2E — Phase 4 : détection et supervision	46
4.11	Flux E2E — Phase 5 : rotation automatique des certificats	46
4.12	Hiérarchie de la PKI à deux niveaux (CA racine <i>offline</i> + CA intermédiaire Vault)	48
4.13	<i>Handshake</i> TLS 1.3 avec authentification mutuelle (mTLS)	50
4.14	Chaîne de détection Suricata → Wazuh — Partie 1 : capture AF_PACKET et décodage MQTT	52
4.15	Chaîne de détection Suricata → Wazuh — Partie 2 : application des règles SID et génération d'alertes	53
4.16	Chaîne de détection Suricata → Wazuh — Partie 3 : EVE JSON → Filebeat → Wazuh → OpenSearch → Dashboard	54
4.17	Pipeline ML — Partie 1 : chargement, EDA et feature engineering	56
4.18	Pipeline ML — Partie 2 : entraînement parallèle de trois modèles (IsolationForest, RF binaire, RF multi-classes)	57
4.19	Pipeline ML — Partie 3 : comparaison des modèles, export et intégration Wazuh	59
4.20	Analyse exploratoire — distribution des classes	60
4.21	Analyse exploratoire — matrice de corrélation des features numériques	61
4.22	Vault UI — Partie 1 : secrets engines PKI (pki_root offline, pki_int active) et méthodes d'authentification	62
4.23	Vault UI — Partie 2 : détail de pki_int — rôles role-server / role-device, CRL et séparation des usages	63
4.24	Wazuh Dashboard : alertes corrélées issues des scénarios d'attaque	64
4.25	GNS3 — Partie 1 : infrastructure hôte (VMware, VM GNS3), pfSense, MikroTik CHR et Docker Engine	65
4.26	GNS3 — Partie 2 : conteneurs Docker répartis sur les quatre zones VLAN (IoT, DMZ, PKI, SIEM)	66
4.27	GNS3 — Partie 3 : flux de données inter-nœuds (MQTT/TLS, Vault API, Filebeat, port miroir SPAN, révocation CRL)	67
5.1	pfSense – règles de filtrage inter-VLAN : flux autorisés (vert) et flux bloqués (rouge)	70
5.2	pfSense – règles NAT Outbound : les VLANs internes masqués derrière l'adresse WAN	71

5.3	Simulateur de flotte IoT : 100 dispositifs actifs, 35 085 messages MQTT chargés	72
5.4	Logs Mosquitto : refus PUBLISH par ACL pour topics non autorisés . .	73
5.5	Conteneur d’attaque atk-siem : lancement du scénario A1-flood depuis le VLAN IoT	74
5.6	Résultat A1-flood : 24/24 connexions bloquées (Errno 113 Host is unreachable)	75
5.7	Résumé JSON du scénario A1-flood : success_connections: 0, failed_connections: 24	76
5.8	Isolation Forest – matrice de confusion	78
5.9	Isolation Forest – distribution des scores d’anomalie	78
5.10	Random Forest : matrice de confusion et courbe ROC	79
5.11	Random Forest – confusion multi-classes (8 classes)	79
5.12	Random Forest – importance des features	80
5.13	Comparaison synthétique des modèles	80
5.14	Distribution du handshake mTLS	81
5.15	Latence de connexion	82
5.16	Débit applicatif	82
5.17	RTT par taille de message	82
5.18	Comparaison MQTT clair vs MQTT/TLS vs MQTT/mTLS	83

Liste des tableaux

1.1	Comparaison des approches de sécurisation IoT pour les opérateurs . .	4
1.2	Objectifs spécifiques du projet	7
1.3	Périmètre du projet – inclus et exclus	8
1.4	Comparaison Scrum, Kanban et XP	10
2.1	Comparaison des principaux protocoles applicatifs IoT	12
2.2	Comparaison des solutions IDS open source	15
2.3	Comparaison des solutions SIEM	16
3.1	Diagnostic des lacunes et briques de réponse correspondantes	19
3.2	Besoins fonctionnels	20
3.3	Besoins non fonctionnels	20
3.4	Correspondance lacunes – briques de sécurité – objectifs spécifiques . .	24
3.5	Planification des sprints	25
3.6	Backlog produit (extrait priorisé)	26
4.1	Composants de l’environnement de développement	37
5.1	Plan de tests	69
5.2	Taux de détection des scénarios d’attaque (10 rejeux par scénario) . . .	70
5.3	Performances ML (validation 5-fold)	80

Liste des acronymes

IoT	Internet of Things – Internet des Objets
TLS	Transport Layer Security
mTLS	Mutual TLS – TLS mutuel (authentification bidirectionnelle)
DTLS	Datagram TLS
PKI	Public Key Infrastructure – Infrastructure à Clés Publiques
CA	Certificate Authority – Autorité de Certification
CSR	Certificate Signing Request
CRL	Certificate Revocation List
OCSP	Online Certificate Status Protocol
MQTT	Message Queuing Telemetry Transport
CoAP	Constrained Application Protocol
HTTP	HyperText Transfer Protocol
AMQP	Advanced Message Queuing Protocol
SIEM	Security Information and Event Management
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
DPI	Deep Packet Inspection
DoS	Denial of Service
DDoS	Distributed Denial of Service
MiTM	Man-in-the-Middle
ML	Machine Learning – Apprentissage Automatique
RF	Random Forest
IF	Isolation Forest

ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
RTT	Round-Trip Time
QoS	Quality of Service
DMZ	DeMilitarized Zone
VLAN	Virtual Local Area Network
GNS3	Graphical Network Simulator 3
VM	Virtual Machine
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
ACL	Access Control List
CN	Common Name
SAN	Subject Alternative Name
E2E	End-to-End
RSA	Rivest–Shamir–Adleman
ECDHE	Elliptic Curve Diffie-Hellman Ephemeral
OVA	Open Virtual Appliance
TT	Tunisie Telecom
FST	Faculté des Sciences de Tunis
OWASP	Open Worldwide Application Security Project
ENISA	European Union Agency for Cybersecurity
NIST	National Institute of Standards and Technology
RFC	Request For Comments
ADSL	Asymmetric Digital Subscriber Line

LPWAN Low-Power Wide-Area Network

SOC Security Operations Center

ETL Extract Transform Load

BF-1 Besoin Fonctionnel n°1

BF-2 Besoin Fonctionnel n°2

BF-3 Besoin Fonctionnel n°3

BF-4 Besoin Fonctionnel n°4

BF-5 Besoin Fonctionnel n°5

BF-6 Besoin Fonctionnel n°6

BNF-1 Besoin Non Fonctionnel n°1

BNF-2 Besoin Non Fonctionnel n°2

BNF-3 Besoin Non Fonctionnel n°3

BNF-4 Besoin Non Fonctionnel n°4

BNF-5 Besoin Non Fonctionnel n°5

BNF-6 Besoin Non Fonctionnel n°6

O1 Objectif n°1

O2 Objectif n°2

O3 Objectif n°3

O4 Objectif n°4

O5 Objectif n°5

O6 Objectif n°6

O7 Objectif n°7

Introduction générale

Les objets connectés occupent aujourd’hui une place importante dans la vie de tous les jours. On les retrouve dans les maisons, les hôpitaux, les usines, les transports ou encore dans les services publics. Leur rôle est simple : observer une situation, envoyer une information et aider à réagir plus vite. Ils permettent ainsi de gagner du temps, d’améliorer le suivi de certaines activités et de rendre plusieurs services plus pratiques.

Cependant, plus les appareils échangent d’informations, plus les risques augmentent. Si ces échanges sont mal protégés, une personne malveillante peut lire des messages, se faire passer pour un appareil autorisé, perturber un service ou récupérer des données qui ne devraient pas être partagées. La sécurité devient alors une condition essentielle pour préserver la confiance, assurer le bon fonctionnement du service et protéger les informations échangées.

Ce projet de fin d’études a pour but de proposer une manière claire et complète de mieux protéger ces échanges. L’idée est de ne pas compter sur une seule protection, mais d’en réunir plusieurs : vérifier qui communique, protéger les messages pendant leur envoi, limiter ce que chacun a le droit de faire et surveiller le système pour repérer rapidement ce qui semble anormal. Le travail a été réalisé dans un cadre de test proche d’une situation réelle, afin de vérifier à la fois l’efficacité de la protection proposée et sa facilité d’utilisation.

Ce rapport est organisé en cinq chapitres. Le premier présente le contexte du projet, le problème posé et les objectifs retenus. Le deuxième expose les notions utiles à la compréhension du sujet et les principales menaces. Le troisième décrit la démarche suivie et les choix de conception. Le quatrième présente la réalisation du travail et les scénarios étudiés. Enfin, le cinquième regroupe les essais, les résultats, les limites et les pistes d’amélioration, avant la conclusion générale.

Chapitre 1

Contexte général du projet

1.1 Introduction

Ce chapitre expose le cadre général dans lequel s’inscrit le projet de fin d’études. Il situe d’abord le travail dans le contexte du marché IoT et dans celui, plus spécifique, des opérateurs télécom, avant de présenter l’organisme d’accueil et son positionnement face aux enjeux de sécurité. Le chapitre formalise ensuite la problématique, précise les objectifs spécifiques, délimite le périmètre de l’étude et explicite la démarche méthodologique retenue.

1.2 Cadre général du projet

Dans le cadre de ce travail, l’objectif est de construire un prototype expérimental capable de sécuriser les flux de communication d’un écosystème IoT simulé. Le projet s’appuie sur une architecture représentative d’un environnement opérateur : segmentation réseau, services de confiance, broker MQTT, supervision de sécurité et analyse comportementale. Cette approche permet de valider les mécanismes proposés avant toute projection vers un déploiement réel à plus grande échelle.

1.3 L’IoT dans les réseaux de télécommunications

1.3.1 Croissance et enjeux du marché IoT

L’Internet des Objets constitue l’une des évolutions technologiques les plus structurantes de la décennie actuelle. Selon l’ENISA [1], le nombre de dispositifs connectés actifs dans le monde poursuit une croissance soutenue, portée par la multiplication des cas d’usage dans l’industrie, les villes intelligentes, la santé connectée, les infrastructures énergétiques et les réseaux de transport. Cette expansion génère un volume massif de données dont la collecte, le transport et le traitement reposent en très grande partie sur les infrastructures des opérateurs télécom.

Cette croissance s’accompagne cependant d’une complexification notable des exigences de sécurité. Meneghello et al. [2] soulignent que les dispositifs IoT présentent structurellement des vulnérabilités liées à leur conception : ressources processeur et mémoire limitées rendant difficile l’intégration de mécanismes cryptographiques complets, cycles de mise à jour longs ou inexistants, et exposition à des réseaux partagés peu maîtrisés. Ces fragilités sont exploitées de manière croissante par des acteurs malveillants,

comme l'illustre l'attaque Mirai [3], qui a mobilisé des centaines de milliers d'objets compromis pour générer des attaques par déni de service dépassant 1 Tbit/s (analysée en détail au chapitre 2).

1.3.2 Rôle des opérateurs télécom dans l'écosystème IoT

Les opérateurs télécom occupent une position centrale et particulière dans l'écosystème IoT : ils fournissent la connectivité (2G/4G/LTE/5G, NB-IoT, LoRaWAN), hébergent les plateformes de gestion des dispositifs et proposent des services à valeur ajoutée incluant la supervision, l'analyse des données et la sécurité managée. Ce positionnement crée cependant des responsabilités spécifiques qui n'ont pas d'équivalent dans les déploiements IoT privés :

- **Multi-tenance et isolation** : un même réseau et une même plateforme IoT peuvent héberger simultanément les objets de plusieurs clients verticaux distincts (énergie, transport, santé, industrie). L'isolation rigoureuse des flux entre locataires est une exigence de sécurité critique que les architectures IoT génériques ne prennent pas toujours en charge nativement ;
- **Volumétrie et passage à l'échelle** : les messages IoT peuvent atteindre des millions de publications par heure sur un seul cluster de brokers. Les mécanismes de sécurité — chiffrement, authentification, contrôle d'accès — doivent être conçus pour fonctionner à cette échelle sans dégrader la qualité de service ni la latence applicative ;
- **Hétérogénéité du parc de terminaux** : l'opérateur doit gérer des dispositifs aux capacités très différentes — microcontrôleurs disposant de quelques kilooctets de RAM, passerelles industrielles sous Linux embarqué, modems cellulaires NB-IoT — en appliquant des politiques de sécurité cohérentes à l'ensemble du parc ;
- **Responsabilité contractuelle et réglementaire** : une fuite ou une compromission de flux IoT clients expose l'opérateur à des risques réglementaires (protection des données personnelles, responsabilité de service) et à des impacts réputationnels majeurs, le contraignant à démontrer un niveau de sécurité maîtrisé et auditable.

Ces contraintes font de la sécurisation des flux IoT un enjeu opérationnel de premier ordre pour les opérateurs télécom, bien au-delà d'une simple exigence technique ponctuelle. Elles motivent directement la conception d'une architecture dédiée, reproductible et validée expérimentalement dans le cadre de ce projet.

Tab. 1.1 : Comparaison des approches de sécurisation IoT pour les opérateurs

Approche	Avantages	Limites pour un opérateur
Plateformes cloud commerciales (AWS IoT, Azure IoT Hub)	Déploiement rapide, fonctionnalités riches, mises à jour gérées	Dépendance fournisseur, souveraineté des données, coûts récurrents variables
Distributions open source intégrées (ThingsBoard, Eclipse Kura)	Open source, personnalisable, large communauté	TLS souvent optionnel, PKI non intégrée, supervision et corrélation limitées
Architecture sur mesure (composants open source)	Contrôle total, souveraineté, adaptabilité au contexte opérateur	Complexité initiale de mise en œuvre, nécessite expertise interne

1.3.3 Approches de sécurisation disponibles pour les opérateurs

Face à ces enjeux, plusieurs approches de sécurisation sont envisageables. Le tableau 1.1 présente une comparaison structurée des trois grandes familles de solutions.

Pour un opérateur télécom souhaitant conserver la maîtrise de son infrastructure, répondre aux exigences réglementaires locales et éviter tout verrouillage fournisseur, une architecture sur mesure fondée sur des composants open source éprouvés constitue l'approche la plus cohérente. C'est ce positionnement qui oriente l'ensemble des choix techniques opérés dans ce projet.

1.4 Présentation de l'organisme d'accueil

Tunisie Telecom est l'opérateur historique de télécommunications en Tunisie. Fondée en 1995, l'entreprise joue un rôle clé dans le développement des infrastructures de communication du pays et dans la fourniture de services de téléphonie, d'Internet et de transmission de données [4]. La figure 1.1 présente le logo de Tunisie Telecom.



Fig. 1.1 : Logo de Tunisie Telecom

L'opérateur propose une large gamme de services destinés aussi bien aux particuliers qu'aux entreprises, incluant :

- la téléphonie fixe et mobile .
- l'accès à Internet haut débit et très haut débit .

- les services de données et de connectivité .
- les solutions cloud et les services numériques.

Grâce à un vaste réseau d'infrastructures couvrant l'ensemble du territoire national, Tunisie Telecom contribue activement à la transformation numérique du pays. L'entreprise investit également dans des technologies émergentes telles que la 5G, l'Internet des objets et les solutions de cybersécurité afin de répondre aux besoins croissants en connectivité et en sécurité.

Dans ce contexte, l'opérateur développe des initiatives visant à renforcer la sécurité des infrastructures réseau et des services numériques, notamment face à l'augmentation des cybermenaces.

1.4.1 Service d'accueil

La structure organisationnelle de Tunisie Telecom est constituée d'une direction générale, de neuf directions centrales et de 24 directions régionales. Chacune des parties prenantes de l'organisation joue un rôle indéniablement important. Cette organisation est illustrée dans la figure 1.2.

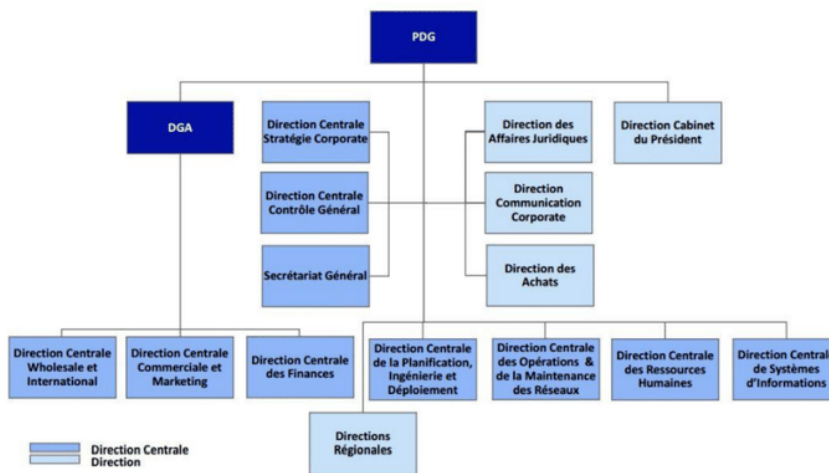


Fig. 1.2 : Organigramme de Tunisie Telecom

Le projet a été réalisé au sein du service technique chargé des infrastructures réseau et de la sécurité des systèmes de communication. Ce service joue un rôle essentiel dans la gestion, la supervision et la protection des infrastructures utilisées pour les services télécom et les plateformes numériques.

Les principales activités du service incluent :

- la gestion et l'administration des infrastructures réseau .
- la supervision du trafic et des performances des systèmes .

- la mise en œuvre de solutions de cybersécurité .
- l’analyse des incidents et des menaces de sécurité .
- l’évaluation et l’intégration de nouvelles technologies.

Dans ce cadre, le service s’intéresse particulièrement aux problématiques de sécurité liées aux architectures modernes telles que les environnements cloud, les réseaux virtualisés et les écosystèmes IoT.

Le stage réalisé dans ce service offre ainsi l’opportunité de travailler sur des problématiques concrètes liées à la sécurisation des communications dans les systèmes IoT.

1.4.2 Positionnement de Tunisie Telecom face aux enjeux IoT

Dans le cadre de sa stratégie de transformation numérique, *Tunisie Telecom* développe progressivement des offres dans le domaine de l’IoT et des services connectés. L’opérateur est confronté aux mêmes impératifs de sécurisation que ses homologues internationaux, dans un contexte supplémentaire de contraintes réglementaires nationales et de ressources techniques à mobiliser de manière optimale.

Le service d’accueil est particulièrement sensibilisé aux risques liés à l’interconnexion de dispositifs IoT sur les infrastructures de l’opérateur : hétérogénéité des équipements connectés, absence de standards unifiés d’authentification à l’échelle du parc, difficultés de supervision des flux applicatifs de type MQTT et manque d’outils permettant la détection de comportements anormaux au niveau applicatif. Ces constats constituent le point de départ direct du projet et justifient la construction d’un démonstrateur expérimental permettant de valider une approche intégrée avant tout passage à l’échelle opérationnel.

1.5 Problématique

L’écosystème d’un opérateur télécom présente plusieurs spécificités qui rendent la sécurisation des flux IoT particulièrement délicate :

- **Hétérogénéité** des dispositifs (microcontrôleurs ESP32, passerelles industrielles, modems cellulaires) et des protocoles applicatifs (MQTT, CoAP, AMQP, LwM2M) .
- **Volumétrie** massive et asynchrone des messages (publications périodiques, événements rares mais critiques) .
- **Contraintes embarquées** : ressources CPU/mémoire limitées, batteries, latence radio variable .

- **Multi-tenance** et segmentation réseau : un même opérateur héberge plusieurs clients verticaux qui ne doivent jamais accéder aux flux des autres .
- **Gestion à grande échelle** d'identités cryptographiques (certificats X.509) pour des dizaines de milliers d'objets, avec des cycles de vie variables .

La problématique centrale de ce projet de fin d'études peut être formulée ainsi :

Problématique

Comment concevoir, déployer et valider, dans un environnement simulé représentatif d'un opérateur télécom, une architecture de sécurisation **de bout en bout** des flux de communication d'un écosystème IoT, qui combine : (i) un chiffrement et une authentification mutuelle systématiques entre objets et infrastructures . (ii) une gestion automatisée du cycle de vie des certificats . (iii) une supervision active des menaces par corrélation d'alertes réseau et apprentissage automatique . (iv) tout en restant compatible avec les contraintes de performance et de scalabilité du domaine ?

1.6 Objectifs spécifiques

À partir de cette problématique, sept objectifs spécifiques (O1–O7) structurent l'ensemble des travaux, qui sont présentés dans le tableau 1.2 :

Tab. 1.2 : Objectifs spécifiques du projet

ID	Objectif
O1	Concevoir une architecture IoT segmentée et reproductible, déployable dans un environnement simulé.
O2	Déployer une infrastructure à clés publiques (PKI) automatisée fondée sur HashiCorp Vault.
O3	Imposer un chiffrement TLS 1.3 avec authentification mutuelle (mTLS) sur le broker MQTT.
O4	Simuler une flotte hétérogène de 50 dispositifs IoT et générer un trafic réaliste mêlant comportements légitimes et attaques.
O5	Mettre en place une analyse réseau native MQTT au moyen d'un IDS Suricata.
O6	Centraliser et corréler les événements de sécurité dans un SIEM Wazuh.
O7	Développer un module de détection d'anomalies par apprentissage automatique (IsolationForest et Random Forest) et évaluer son apport.

Ces objectifs traduisent opérationnellement la problématique et structurent les différentes phases du projet.

1.7 Périmètre et contraintes du projet

1.7.1 Périmètre fonctionnel

Ce projet se concentre sur la sécurisation des communications IoT au niveau du transport et de l'application, depuis la gestion des identités cryptographiques des dispositifs jusqu'à la détection comportementale des anomalies. Le tableau 1.3 précise explicitement ce qui est inclus et exclu du périmètre, afin d'éviter toute ambiguïté dans l'interprétation des résultats présentés.

Tab. 1.3 : Périmètre du projet – inclus et exclus

Inclus dans le périmètre	Exclu du périmètre
Segmentation réseau par VLANs sous GNS3	Déploiement en environnement de production réel
PKI à deux niveaux avec HashiCorp Vault	Gestion des mises à jour firmware des objets (OTA)
Broker MQTT en TLS 1.3 + mTLS avec ACL par topic	Sécurité de la couche matérielle des dispositifs
Simulation d'une flotte de 50 dispositifs hétérogènes	Flotte supérieure à 50 dispositifs (contrainte ressources hôte)
Détection IDS Suricata + corrélation SIEM Wazuh	Intégration aux outils SOC existants de Tunisie Telecom
Pipeline ML hors ligne (IsolationForest, Random Forest)	Scoring ML en temps réel sur flux de données en streaming
Évaluation des performances (latence, débit mTLS)	Tests de charge en environnement radio réel (4G/5G/NB-IoT)

1.7.2 Contraintes du projet

La réalisation de ce projet s'est inscrite dans un ensemble de contraintes techniques, temporelles et opérationnelles qui ont directement influencé les choix architecturaux :

- **Contrainte matérielle** : l'ensemble de l'infrastructure est simulé sur une station Windows 10/11 dotée de 16 Go de RAM minimum, sans accès à un environnement cloud dédié ni à des équipements physiques IoT réels. Cette contrainte délimite la taille de la flotte simulable et le niveau de réalisme des mesures de performance ;
- **Contrainte temporelle** : le stage d'une durée de six mois impose une priorisation stricte des livrables. La méthodologie Scrum, adoptée pour ce projet et décrite dans la section suivante, garantit que les fonctionnalités critiques (PKI, mTLS, détection) sont livrées et testées en priorité ;

- **Contrainte de reproductibilité** : l'ensemble de l'environnement doit être reconstituable depuis zéro à partir des scripts et configurations versionnés, sans intervention manuelle non documentée ;
- **Contrainte de souveraineté technologique** : tous les composants retenus sont open source, afin de garantir la transparence, la maîtrise du code et l'absence de dépendance envers un fournisseur commercial.

1.7.3 Hypothèses de travail

Trois hypothèses structurent l'expérimentation conduite dans ce projet :

1. Le jeu de données utilisé pour l'entraînement des modèles de ML (76 000 messages répartis en huit classes : une classe normale et sept catégories d'attaques) est représentatif des profils de trafic rencontrés dans un écosystème IoT réel [5] ;
2. Les performances mesurées dans l'environnement simulé (latence du handshake mTLS, débit du broker) constituent des ordres de grandeur valides pour une évaluation comparative, sans prétention à reproduire exactement un réseau radio réel avec ses contraintes de propagation et de mobilité ;
3. La détection par règles statiques (Suricata + Wazuh) et la détection par apprentissage automatique sont traitées comme des couches complémentaires et non substituables, chacune couvrant un sous-ensemble différent du spectre des menaces.

1.8 Démarche méthodologique

Le pilotage d'un projet technique de cette envergure dans un délai académique contraint requiert une méthodologie explicitement choisie. Trois cadres agiles ont été évalués : **Scrum** [6], **Kanban** [7] et **Extreme Programming (XP)** [8]. Le tableau 1.4 en présente la comparaison sur sept critères déterminants.

Scrum a été retenu pour trois raisons décisives :

- **Alignement avec les jalons académiques** : les revues de sprint coïncident naturellement avec les points d'avancement imposés (rapport intermédiaire, soutenance), offrant une visibilité régulière sur l'état du projet ;
- **Flexibilité face aux imprévus techniques** : le réajustement du backlog entre deux sprints permet de réagir à un blocage (configuration réseau, problème de certificats) sans remettre en cause l'ensemble du planning ;
- **Maîtrise du périmètre** : la priorisation du backlog garantit que les fonctionnalités essentielles sont livrées en premier, limitant le risque de ne pas finaliser les parties critiques avant la soutenance.

Tab. 1.4 : Comparaison Scrum, Kanban et XP

Critère	Scrum	Kanban	XP
Cadre de travail	Itératif (sprints)	Continu (flux)	Itératif court
Taille équipe	3–9 personnes	Variable	2–12
Gestion des changements	Encadrée (entre sprints)	Très flexible	Flexible
Cycle de livraison	1–4 semaines	Continu	1–3 semaines
Documentation	Modérée (US, backlog)	Faible	Faible
Pratiques d’ingénierie	Indépendantes	Indépendantes	Strictes (TDD, pair-prog)
Courbe d’apprentissage	Modérée	Faible	Élevée
Adaptation au PFE	Très adaptée	Peu adaptée	Peu adaptée

Kanban, dont l’absence de cadence fixe est inadaptée à un projet soumis à des jalons académiques, et XP, dont les pratiques d’ingénierie (TDD, *pair-programming*) sont disproportionnées pour un développement solo, ont été écartés. Six sprints de deux semaines structurent la phase de réalisation ; leur planification détaillée figure au chapitre 3.

1.9 Conclusion

Ce chapitre a permis d’établir le cadre de référence du projet en articulant successivement le contexte sectoriel, l’ancrage institutionnel, la problématique, les objectifs, le périmètre d’étude et la démarche méthodologique. Cet ensemble fournit la base nécessaire à la compréhension des choix effectués dans la suite du mémoire. Le chapitre suivant prolonge ce cadrage par un état de l’art consacré aux menaces, aux protocoles, aux mécanismes de protection et aux outils qui fondent scientifiquement les choix architecturaux retenus.

Chapitre 2

État de l’art

2.1 Introduction

Avant de concevoir une architecture de sécurisation des flux IoT, il est indispensable de cadrer le sujet sur trois plans complémentaires : (i) caractériser les **menaces** qui pèsent spécifiquement sur les écosystèmes IoT à grande échelle ; (ii) recenser et évaluer de manière critique les **technologies et protocoles** candidats pour le transport, l’authentification et la supervision ; (iii) analyser les **travaux et solutions existants** afin d’en extraire les bonnes pratiques, d’identifier leurs limites et de positionner notre contribution. Ce chapitre est structuré autour de ces trois axes, suivi d’une analyse critique de l’existant et d’une synthèse positionnant le projet.

2.2 Menaces et vulnérabilités spécifiques à l’IoT

2.2.1 Surface d’attaque étendue

Les écosystèmes IoT présentent une surface d’attaque significativement plus large que les systèmes informatiques classiques. Meneghello *et al.* [2] en distinguent quatre couches : (i) la couche *matérielle* (composants embarqués, JTAG/UART exposés, attaques par canal auxiliaire) . (ii) la couche *firmware* (mises à jour non signées, exécution de code arbitraire) . (iii) la couche *réseau* (transports en clair, protocoles propriétaires faibles) . (iv) la couche *services et écosystème* (API non authentifiées, applications mobiles vulnérables, multi-tenance mal cloisonnée).

2.2.2 OWASP IoT Top 10 et menaces réseau

L’OWASP synthétise dans son *IoT Top 10* [9] les dix risques applicatifs majeurs : mots de passe faibles ou par défaut, services réseau non sécurisés, interfaces écosystème non protégées, mécanisme de mise à jour absent, composants obsolètes, protection insuffisante de la vie privée, transferts et stockage non chiffrés, absence de gestion des dispositifs, paramètres par défaut non sûrs, absence de hardening physique. Dans ce projet, nous nous concentrons sur les menaces **réseau** et **applicatives** : interception (*sniffing*), *man-in-the-middle*, force brute, exfiltration applicative, déni de service, vol/abus de certificats.

2.2.3 Études de cas de référence

L'analyse de l'attaque **Mirai** [3], [10] illustre toutes ces faiblesses simultanément : le botnet exploitait des couples identifiant/mot de passe par défaut sur des objets exposés en SSH/Telnet, recrutait des centaines de milliers de dispositifs en quelques jours, et générait des attaques DDoS dépassant 1 Tbit/s. Le rapport *ENISA Threat Landscape for IoT* [1] confirme que ces vecteurs restent dominants et identifie comme contre-mesures prioritaires : l'authentification forte, le chiffrement systématique, la segmentation réseau et la supervision active. Ces orientations rejoignent les recommandations du NIST [11] pour la cybersécurité des dispositifs IoT.

Cette cartographie des menaces permet d'identifier les vecteurs prioritaires à contrer dans la solution : interception des flux, usurpation d'identité, déni de service et abus de certificats. Le premier levier sur lequel il est possible d'agir est le **protocole de communication applicatif** lui-même : c'est à ce niveau que la surface d'attaque réseau est déterminée et que les protections de base peuvent être imposées.

2.3 Protocoles applicatifs IoT

Le tableau 2.1 compare quatre protocoles applicatifs représentatifs.

Tab. 2.1 : Comparaison des principaux protocoles applicatifs IoT

Critère	MQTT 5	CoAP	AMQP 1.0	HTTP/REST
Modèle	Pub/Sub	Req/Rep	Pub/Sub + queues	Req/Rep
Transport	TCP/TLS	UDP/DTLS	TCP/TLS	TCP/TLS
Empreinte	Très faible	Très faible	Moyenne	Moyenne/élevée
QoS	0/1/2	Confirmable	0/1	Aucun natif
Sécurité native	TLS+ACL	DTLS+OSCORE	SASL+TLS	TLS+JWT
Maturité IoT	Très large	Large	Industrielle	Universelle

Le choix de **MQTT 5** [12] pour ce projet repose sur trois critères : empreinte mémoire et réseau adaptée aux microcontrôleurs . large adoption dans les écosystèmes industriels et les opérateurs IoT . existence de mécanismes natifs d'authentification (TLS+ACL) facilement combinables avec mTLS.

Le protocole applicatif étant fixé, la question du **chiffrement du canal** devient prépondérante : MQTT, en lui-même, ne garantit ni la confidentialité ni l'authenticité des messages échangés. TLS 1.3 avec authentification mutuelle répond précisément à ces deux exigences, comme le détaille la section suivante.

2.4 Sécurisation du transport : TLS et mTLS

2.4.1 TLS 1.3

Le protocole TLS 1.3 [13] apporte par rapport à TLS 1.2 une simplification significative : suppression des suites cryptographiques obsolètes, négociation en **1-RTT** (et 0-RTT optionnel), *forward secrecy* obligatoire, signatures et échanges de clés modernisés (ECDHE, EdDSA, AES-GCM, ChaCha20-Poly1305). Les recommandations opérationnelles publiées dans la RFC 9325 [14] guident le choix des paramètres et la durée de vie des certificats.

2.4.2 Authentification mutuelle

Dans un déploiement classique, seul le serveur s’authentifie auprès du client. Le **mTLS** (*mutual TLS*) impose en outre au client de présenter un certificat X.509 valide signé par une autorité de confiance. Cette double authentification permet : (i) d’éviter qu’un objet illégitime se connecte au broker . (ii) de chiffrer chaque session par bout . (iii) de tracer chaque message via l’identité cryptographique de son émetteur. Cette propriété est essentielle pour la mise en place ultérieure d’ACL par certificat dans Mosquitto [15].

L’authentification mutuelle par certificats suppose cependant l’existence d’une infrastructure capable d’**émettre, renouveler et révoquer** ces certificats à l’échelle d’un parc de dispositifs : c’est précisément le rôle d’une PKI automatisée, dont les principes sont analysés dans la section suivante.

Resultat

Positionnement critique : TLS 1.3 + mTLS Le choix de TLS 1.3 en remplacement de TLS 1.2 répond à trois arguments techniques précis : (i) la suppression des suites vulnérables (RC4, DES, RSA pur sans *forward secrecy*) réduit la surface cryptographique exploitable ; (ii) la négociation en 1-RTT diminue la latence du *handshake*, critique pour les dispositifs contraints ; (iii) la *forward secrecy* obligatoire garantit que la compromission ultérieure d’une clé privée ne met pas en péril les sessions passées. L’ajout du mTLS va au-delà du chiffrement : il substitue à une authentification par mot de passe (facilement rejouable) une preuve cryptographique d’identité non duplicable sans accès à la clé privée du dispositif. Cette double garantie – confidentialité *et* authenticité mutuelle – est la seule combinaison capable de fermer simultanément les vecteurs d’interception et d’usurpation identifiés dans la cartographie des menaces.

2.5 Gestion des identités : PKI et automatisation

Une PKI à deux niveaux (CA racine *offline* + CA intermédiaire *online*) constitue la pratique recommandée [11] : la CA racine ne signe que des CA intermédiaires, et reste hors ligne . les CA intermédiaires signent les certificats opérationnels (services, dispositifs). En cas de compromission d’une CA intermédiaire, seul le sous-arbre concerné doit être révoqué.

L’automatisation de l’émission est indispensable lorsque le parc dépasse quelques dizaines d’objets. **HashiCorp Vault** [16] fournit un *secrets engine pki* qui expose une API HTTP de demande de certificats avec contrôle fin (rôles, ACL, TTL). C’est la solution retenue dans ce projet.

Resultat

Positionnement critique : Vault vs alternatives PKI Trois alternatives à HashiCorp Vault ont été évaluées : (i) **OpenSSL CLI** : complet techniquement mais entièrement manuel ; toute émission de certificat requiert une intervention humaine, ce qui l’exclut dès que le parc dépasse une dizaine de dispositifs ; (ii) **EJBCA (Keyfactor)** : solution PKI d’entreprise mature, mais sa complexité de déploiement et sa courbe d’apprentissage sont disproportionnées pour un projet académique de six mois ; (iii) **Let’s Encrypt / ACME** : idéal pour les certificats TLS publics, mais inadapté à la PKI interne d’un réseau IoT isolé (absence de validation domaine interne, pas de certificats client mTLS natifs). Vault a été retenu car il est le seul à réunir : API REST d’émission automatisée, gestion fine des rôles et TTL, intégration aux pipelines DevOps, et déploiement entièrement on-premise – critère non négociable dans un contexte opérateur télécom soucieux de souveraineté des données.

Même avec une PKI robuste et un chiffrement bout en bout, il n’est pas possible d’exclure tout incident : un certificat peut être volé, un dispositif légitime peut adopter un comportement déviant, ou une configuration insuffisante peut ouvrir un vecteur non anticipé. La **supervision active** – combinant IDS réseau et SIEM – constitue le filet de sécurité complémentaire à ces mécanismes préventifs.

2.6 Détection d’intrusion et supervision

2.6.1 IDS réseau orienté MQTT

Suricata est un IDS/IPS open source qui dispose, depuis la version 6, d’un *parser* natif MQTT capable d’extraire les champs **CONNECT**, **PUBLISH**, **SUBSCRIBE** [17]. Cette capacité permet de définir des règles métier (par exemple : alerter sur un **CONNECT** sans **ClientID** reconnu, ou sur une rafale de **CONNECT** indiquant un *brute force*) qui ne seraient pas accessibles à un IDS générique.

Le tableau 2.2 compare trois solutions IDS open source représentatives sur les critères déterminants pour ce projet.

Tab. 2.2 : Comparaison des solutions IDS open source

Critère	Suricata 6	Snort 3	Zeek 6
Parser MQTT natif	Oui (v6+)	Non	Partiel (scripts)
IDS/IPS actif	Oui	Oui	Non (analyse seule)
Multi-threading	Oui (natif)	Oui (v3)	Oui
Intégration Wazuh	Native (eve.json)	Adaptateur requis	Scripts Lua
Règles Sigma/Suricata	Native	Conversion requise	Non
Maturité IoT	Élevée	Moyenne	Élevée (analytique)
Verdict	Retenu	Non retenu	Complémentaire

Suricata est retenu pour trois raisons décisives : (i) son parser MQTT natif élimine tout besoin de post-traitement applicatif ; (ii) son format de sortie `eve.json` est nativement consommé par Wazuh, réduisant la complexité d’intégration ; (iii) son moteur multi-thread garantit une analyse sans dégradation de débit sur les segments IoT à forte densité de messages.

2.6.2 SIEM Wazuh

Wazuh [18] est une plateforme de *Security Information and Event Management* qui agrège les journaux d’agents endpoint, de sources réseau (Suricata) et applicatifs (Vault, Mosquitto). Elle s’intègre nativement à OpenSearch pour le stockage et au tableau de bord Wazuh Dashboard pour la visualisation et la corrélation. Sa modularité (règles XML, décodeurs personnalisables) en fait un candidat naturel pour la consolidation des alertes IoT.

Le tableau 2.3 compare Wazuh aux deux autres plates-formes SIEM courantes sur les critères déterminants pour un projet IoT académique à contraintes de coût et de souveraineté.

Wazuh est retenu pour sa triple intégration native : avec Suricata (format `eve.json` sans transformation), avec les agents endpoint sur les conteneurs Docker du laboratoire, et avec OpenSearch pour un stockage illimité. La licence open source garantit en outre la souveraineté des données, critère non négociable dans le contexte opérateur télécom.

Les règles IDS, aussi précises soient-elles, ne peuvent couvrir que les attaques **déjà connues et formalisées**. Pour détecter des comportements nouveaux ou subtils – comme l’utilisation d’un certificat valide depuis un contexte comportemental anormal – une couche d’apprentissage automatique est nécessaire. La section suivante en présente les approches retenues.

Tab. 2.3 : Comparaison des solutions SIEM

Critère	Wazuh 4.7	ELK Stack	Splunk Free
Licence	Open source (OSS)	Open source	Propriétaire (limité)
Intégration Suricata	Native (eve.json)	Via Filebeat	Via add-on HEC
Corrélation native	Oui (règles XML)	Non (Elastic SIEM en payant)	Oui (SPL)
Agents endpoint	Oui (Wazuh agent)	Non natif	Oui (UF)
Volume journaux (gratuit)	Illimité	Illimité	500 MB/j
Courbe d’apprentissage	Modérée	Élevée (tuning)	Modérée
Souveraineté données	Totale (on-premise)	Totale	Partielle (cloud)
Verdict	Retenu	Non retenu	Non retenu

2.7 Apprentissage automatique pour la détection d’anomalies

Les approches par règles statiques sont efficaces pour les attaques connues mais peinent à détecter les comportements nouveaux. Hasan *et al.* [5] ont montré qu’un classifieur supervisé entraîné sur des features réseau et applicatives obtient des taux de détection supérieurs à 99 % sur sept catégories d’attaques IoT. Les algorithmes les plus utilisés sont :

- **Isolation Forest** [19] : méthode non supervisée d’isolement, particulièrement efficace en haute dimension et adaptée aux scénarios où les anomalies sont rares .
- **Random Forest** [20] : ensemble d’arbres de décision, robuste, interprétable (*feature importance*) et performant en classification multi-classes.

La mobilisation conjointe de ces deux algorithmes n’est pas arbitraire : elle répond à deux exigences analytiquement distinctes. Isolation Forest est **non supervisé** : il modélise la normalité sans nécessiter de données étiquetées, ce qui le rend indispensable pour détecter les anomalies *inconnues* – typiquement, l’utilisation d’un certificat valide depuis un contexte comportemental inhabituel (scénario S6). Random Forest est **supervisé** : entraîné sur des exemples d’attaques étiquetées, il fournit une classification précise en sept catégories et une mesure d’importance par feature permettant d’identifier les attributs les plus discriminants. Ces deux algorithmes sont donc complémentaires : l’un couvre l’inconnu, l’autre maximise la précision sur le connu. La bibliothèque *scikit-learn* [21] fournit des implémentations matures de ces deux algorithmes . elle constitue le socle de notre pipeline ML.

2.8 Analyse critique de l’existant

Plusieurs approches ont été proposées pour sécuriser les flux IoT : gateways IoT propriétaires (AWS IoT Core, Azure IoT Hub), distributions intégrées (ThingsBoard, Eclipse Kura), ou architectures sur mesure. Notre analyse fait ressortir quatre limitations récurrentes :

1. **Verrouillage fournisseur** pour les solutions cloud commerciales, peu compatible avec une stratégie souveraine d’opérateur télécom .
2. **Couverture partielle** de la sécurité chez les distributions intégrées (souvent TLS oui, mais pas mTLS systématique ni PKI automatisée) .
3. **Faible intégration** entre la couche réseau (IDS) et la couche applicative (broker, ML) .
4. **Reproductibilité limitée** des architectures publiées dans la littérature, rarement accompagnées d’un environnement de simulation complet.

2.9 Synthèse du chapitre

Au regard de cet état de l’art, le présent projet se positionne comme une **architecture de référence open source** pour la sécurisation des flux IoT chez un opérateur télécom, avec quatre contributions distinctives : (1) déploiement de bout en bout reproductible sous GNS3+Docker . (2) PKI Vault entièrement automatisée avec mTLS systématique . (3) chaîne de détection unifiée Suricata + Wazuh avec règles MQTT spécifiques . (4) couplage de cette détection signature avec un module ML (Isolation-Forest + RandomForest) pour couvrir les anomalies non connues.

2.10 Conclusion

Ce chapitre a établi le socle conceptuel du projet en suivant un fil directeur : des menaces spécifiques à l’IoT découlent les exigences de protocole . du choix de MQTT découle le besoin de TLS 1.3 . de TLS avec mTLS découle la nécessité d’une PKI automatisée . de la PKI découle le besoin de supervision IDS/SIEM . des limites des règles statiques découle l’apport de l’apprentissage automatique. Cette logique de causalité guidera la conception présentée au chapitre suivant, où chaque brique technique trouvera sa justification dans l’analyse conduite ici.

Chapitre 3

Conception

3.1 Introduction

Ce chapitre a pour objectif de traduire les éléments de cadrage présentés au chapitre précédent en une conception structurée, argumentée et directement exploitable pour la phase de réalisation. Il s'ouvre par un diagnostic de l'existant, destiné à mettre en évidence les insuffisances observées dans les déploiements IoT de type opérateur et à justifier les choix de conception retenus. Le chapitre présente ensuite la spécification des besoins fonctionnels et non fonctionnels, puis une vue d'ensemble de la solution proposée. Il expose enfin les principaux artefacts de pilotage et de modélisation du projet : la planification des sprints, le backlog produit, les diagrammes UML et la planification temporelle, qui constituent la référence de travail pour la mise en œuvre.

3.2 Diagnostic de l'existant

3.2.1 État des lieux de la sécurité IoT dans les déploiements opérateur

L'analyse conduite en amont de ce projet, combinant l'étude des pratiques observées dans des déploiements IoT typiques et les enseignements publiés par l'ENISA [1] et le NIST [11], met en évidence plusieurs lacunes de sécurité récurrentes dans ce type d'environnement :

- **Transport non systématiquement chiffré** : le protocole MQTT est fréquemment déployé sur le port 1883 (texte clair), sans TLS, par souci de simplicité ou de performance perçue. Un attaquant positionné sur le segment réseau peut alors capturer l'intégralité des messages publiés, y compris les données métier sensibles ;
- **Authentification unidirectionnelle ou absente** : le broker ne vérifie généralement pas le certificat du client. Les dispositifs s'authentifient souvent par un simple couple identifiant/mot de passe, voire sans aucun mécanisme d'authentification, rendant toute usurpation d'identité triviale ;
- **Absence de PKI dédiée** : lorsque des certificats existent, ils sont fréquemment auto-signés, non révocables automatiquement et gérés de façon ad hoc, sans cycle de vie défini ni procédure de révocation d'urgence en cas de compromission ;
- **Contrôle d'accès applicatif insuffisant** : les listes de contrôle d'accès aux topics MQTT sont rarement mises en œuvre, permettant à tout dispositif connecté d'accé-

der à l'ensemble des canaux de publication et de souscription, indépendamment de son rôle légitime ;

- **Supervision fragmentée** : les outils de monitoring réseau et applicatif ne communiquent pas entre eux, rendant difficile la corrélation d'événements de sécurité. La détection d'une attaque nécessite une analyse manuelle croisée de plusieurs sources de journaux hétérogènes ;
- **Absence de détection comportementale** : les mécanismes de détection d'intrusion existants se limitent aux signatures connues. Ils ne permettent pas de détecter des anomalies comportementales subtiles telles que l'utilisation d'un certificat valide depuis un contexte inhabituel, une dérive de fréquence de publication ou une exfiltration lente via des topics légitimes.

3.2.2 Synthèse du diagnostic

Le tableau 3.1 synthétise ce diagnostic en mettant en regard chaque lacune identifiée avec la brique de sécurité retenue dans la solution, établissant ainsi le lien direct entre le contexte opérationnel observé et les choix de conception du projet.

Tab. 3.1 : Diagnostic des lacunes et briques de réponse correspondantes

Lacune identifiée	Brique de réponse retenue
Flux MQTT en clair, sans chiffrement de bout en bout	Activation obligatoire de TLS 1.3 sur Mosquitto (port 8883)
Authentification client absente ou basée sur mot de passe faible	Certificat X.509 client obligatoire, vérifié par le broker en mTLS
Gestion manuelle et non automatisée des certificats	PKI HashiCorp Vault à deux niveaux avec émission et révocation via API
Accès applicatifs non restreints par dispositif	ACL Mosquitto fondées sur le <i>Common Name</i> du certificat client
Supervision réseau et applicative déconnectées	Intégration de Suricata au SIEM Wazuh pour la corrélation
Absence de détection comportementale	Module ML fondé sur IsolationForest et Random Forest

Ce diagnostic structuré constitue la justification directe des choix techniques opérés dans ce projet : chaque composant de la solution répond à une lacune précisément identifiée. Cette correspondance garantit la pertinence de l'architecture proposée au-delà d'une simple accumulation d'outils technologiques.

3.3 Spécification des besoins

3.3.1 Besoins fonctionnels (BF)

Tab. 3.2 : Besoins fonctionnels

ID	Description
BF-1	Émettre, renouveler et révoquer automatiquement des certificats X.509 pour services et dispositifs.
BF-2	Imposer le chiffrement TLS 1.3 et l'authentification mutuelle (mTLS) sur le broker MQTT.
BF-3	Restreindre l'accès aux topics MQTT par dispositif via des listes de contrôle d'accès.
BF-4	Simuler une flotte hétérogène de 50 dispositifs IoT générant un trafic réaliste.
BF-5	Détecter les attaques réseau et applicatives par règles IDS et corréler les alertes dans un SIEM.
BF-6	Détecter les anomalies non couvertes par les règles via un modèle d'apprentissage automatique.

3.3.2 Besoins non fonctionnels (BNF)

Tab. 3.3 : Besoins non fonctionnels

ID	Description
BNF-1	Sécurité : aucun flux IoT en clair . rotation des secrets . séparation CA racine / intermédiaire.
BNF-2	Performance : latence handshake mTLS < 50 ms . débit broker > 500 msg/s par client.
BNF-3	Reproductibilité : l'environnement complet doit être réinstallable à partir des scripts fournis.
BNF-4	Scalabilité : l'architecture doit accepter sans refonte au moins 200 dispositifs.
BNF-5	Maintenabilité : configurations versionnées, modules indépendants, journalisation centralisée.
BNF-6	Observabilité : tableau de bord Wazuh accessible aux analystes en moins de 3 clics.

Ces besoins définissent le *quoi* de la solution : les fonctionnalités attendues et les contraintes opérationnelles à respecter. Le diagramme de cas d'utilisation est présenté en trois parties, une par acteur de la plateforme.

3.3.3 Diagramme de cas d'utilisation

Trois acteurs principaux interagissent avec la plateforme : le **dispositif IoT**, l'**administrateur sécurité** et l'**opérateur SOC**. Les relations d'inclusion correspondent à des sous-cas obligatoires, tandis que les relations d'extension représentent des comportements optionnels. Dans ce cadre, la connexion MQTT impose systématiquement l'authentification mTLS et le chiffrement TLS 1.3, alors que l'IDS et le module ML complètent la détection par des mécanismes additionnels de blocage ou d'analyse.

Partie 1 — Dispositif IoT. La figure 3.1 détaille les quatre cas d'utilisation du capteur : demande de certificat X.509 avec vérification de la chaîne CA, connexion au broker MQTT avec authentification mTLS et chiffrement TLS 1.3, publication de télémétrie et renouvellement automatique du certificat.

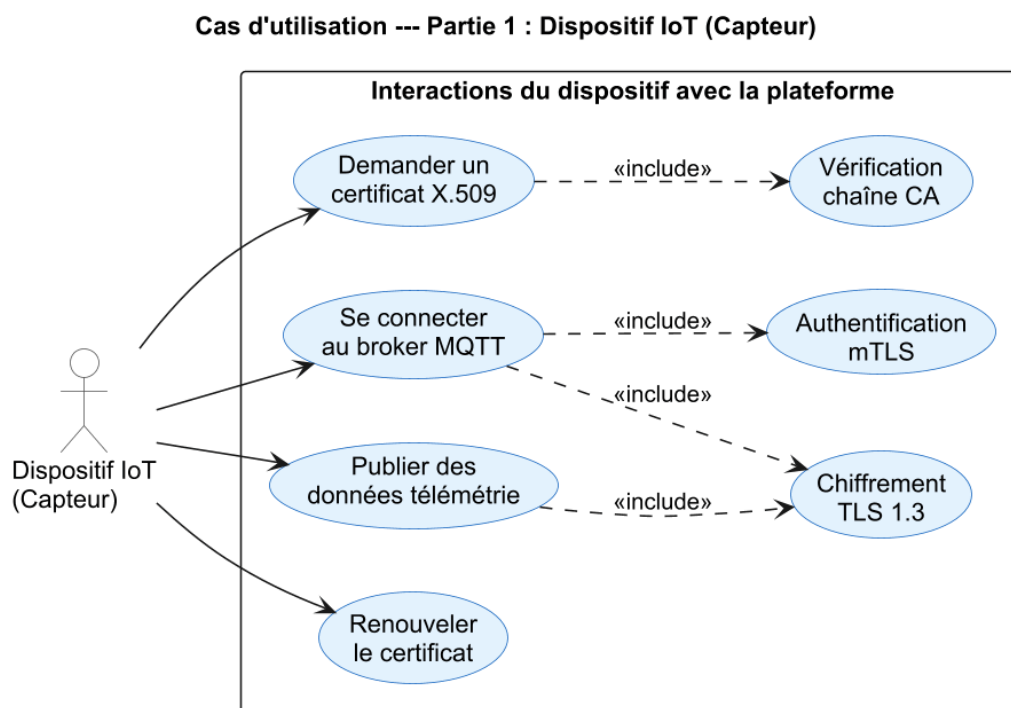


Fig. 3.1 : Cas d'utilisation — Partie 1 : Dispositif IoT (authentification, publication, renouvellement)

Partie 2 — Administrateur sécurité. La figure 3.2 montre les quatre cas d'utilisation de l'administrateur : configuration de la PKI Vault (hiérarchie CA, rôles, TTL), définition des règles ACL Mosquitto, paramétrage des règles IDS Suricata et déploiement des règles de corrélation SIEM Wazuh.

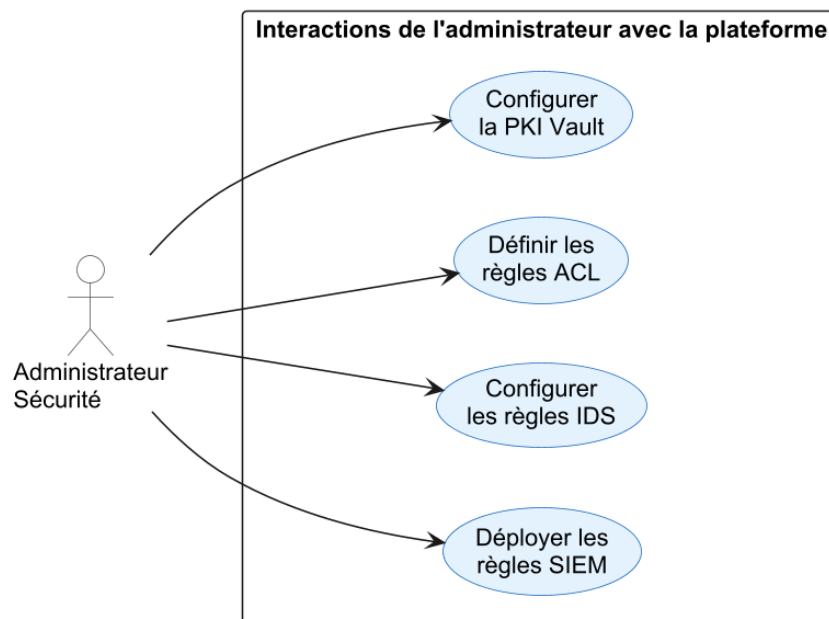
Cas d'utilisation --- Partie 2 : Administrateur Sécurité

Fig. 3.2 : Cas d'utilisation — Partie 2 : Administrateur sécurité (PKI, ACL, IDS, SIEM)

Partie 3 — Opérateur SOC. La figure 3.3 représente les quatre cas d'utilisation de l'opérateur : consultation du tableau de bord Wazuh, analyse des alertes IDS, lancement de la détection ML d'anomalies et évaluation des performances de la plateforme.

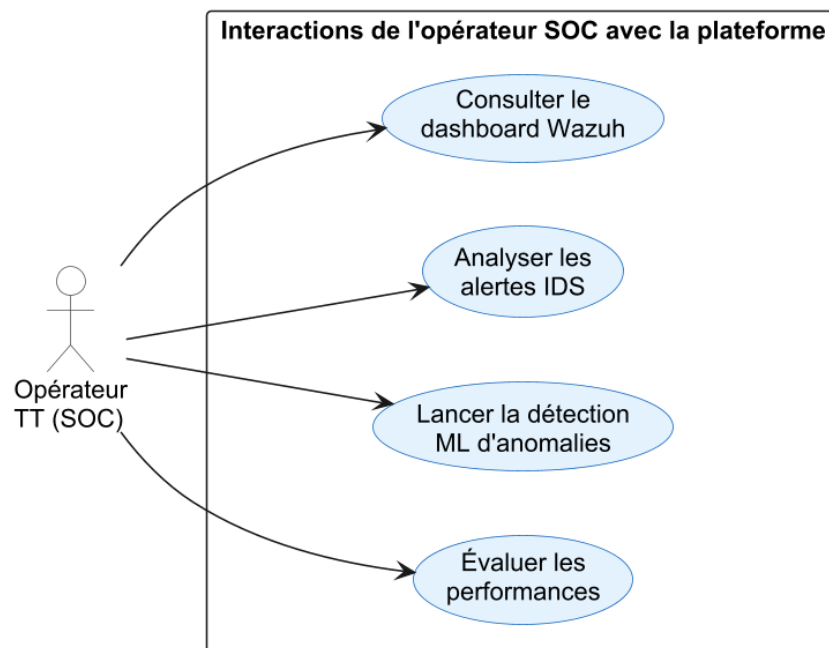
Cas d'utilisation --- Partie 3 : Opérateur TT (SOC)

Fig. 3.3 : Cas d'utilisation — Partie 3 : Opérateur TT/SOC (dashboard, alertes, ML, performances)

3.4 Vue d’ensemble de la solution proposée

Pour répondre à la problématique et atteindre les objectifs ci-dessus, nous proposons une **architecture de défense en profondeur** composée de six briques complémentaires, intégralement déployée dans un laboratoire simulé sous GNS3 [22] et Docker [23] :

1. Une **infrastructure réseau segmentée** (VLANs IoT, services, monitoring, management) construite autour d’un pare-feu pfSense et d’un routeur MikroTik .
2. Une **PKI automatisée** HashiCorp Vault à deux niveaux (CA racine hors ligne + CA intermédiaire en ligne), capable d’émettre à la demande des certificats X.509 pour les services et les objets [16] .
3. Un **broker MQTT Mosquitto** configuré en TLS 1.3 avec authentification mutuelle obligatoire et listes de contrôle d’accès par topic [15] .
4. Un **simulateur de flotte** de 50 dispositifs Python jouant 76 000 messages issus d’un jeu de données réel, dont sept catégories d’attaques .
5. Une **chaîne de détection** comprenant un IDS Suricata avec règles MQTT personnalisées [17] et un SIEM Wazuh [18] pour la corrélation et la visualisation .
6. Un **pipeline de ML** (IsolationForest [19] et Random Forest [20]) entraîné sur les caractéristiques extraites du trafic, capable de détecter les anomalies non couvertes par les règles statiques.

Le tableau 3.4 établit la correspondance entre les lacunes diagnostiquées à la section 3.2, les briques de la solution et les objectifs spécifiques auxquels elles répondent, assurant ainsi la traçabilité complète entre le diagnostic initial et l’architecture proposée.

La figure 3.4 schématise cette vision globale, qui sera détaillée et justifiée dans les chapitres suivants.

La démarche Scrum retenue au chapitre 1 structure la réalisation en six sprints ; la section suivante en détaille la planification et les livrables.

Tab. 3.4 : Correspondance lacunes – briques de sécurité – objectifs spécifiques

Brique de sécurité	Section de réalisation	Obj.
Topologie réseau segmentée (GNS3, pfSense, MikroTik)	§4.3	O1
PKI Vault à deux niveaux (CA racine + CA intermédiaire)	§4.4	O2
Broker Mosquitto TLS 1.3 + mTLS + ACL par topic	§4.5	O3
Simulateur de flotte de 50 dispositifs hétérogènes	§4.6	O4
IDS Suricata avec règles MQTT dédiées	§4.6	O5
SIEM Wazuh (corrélation d'alertes et tableau de bord)	§4.6	O6
Module ML (IsolationForest + Random Forest)	§4.7	O7

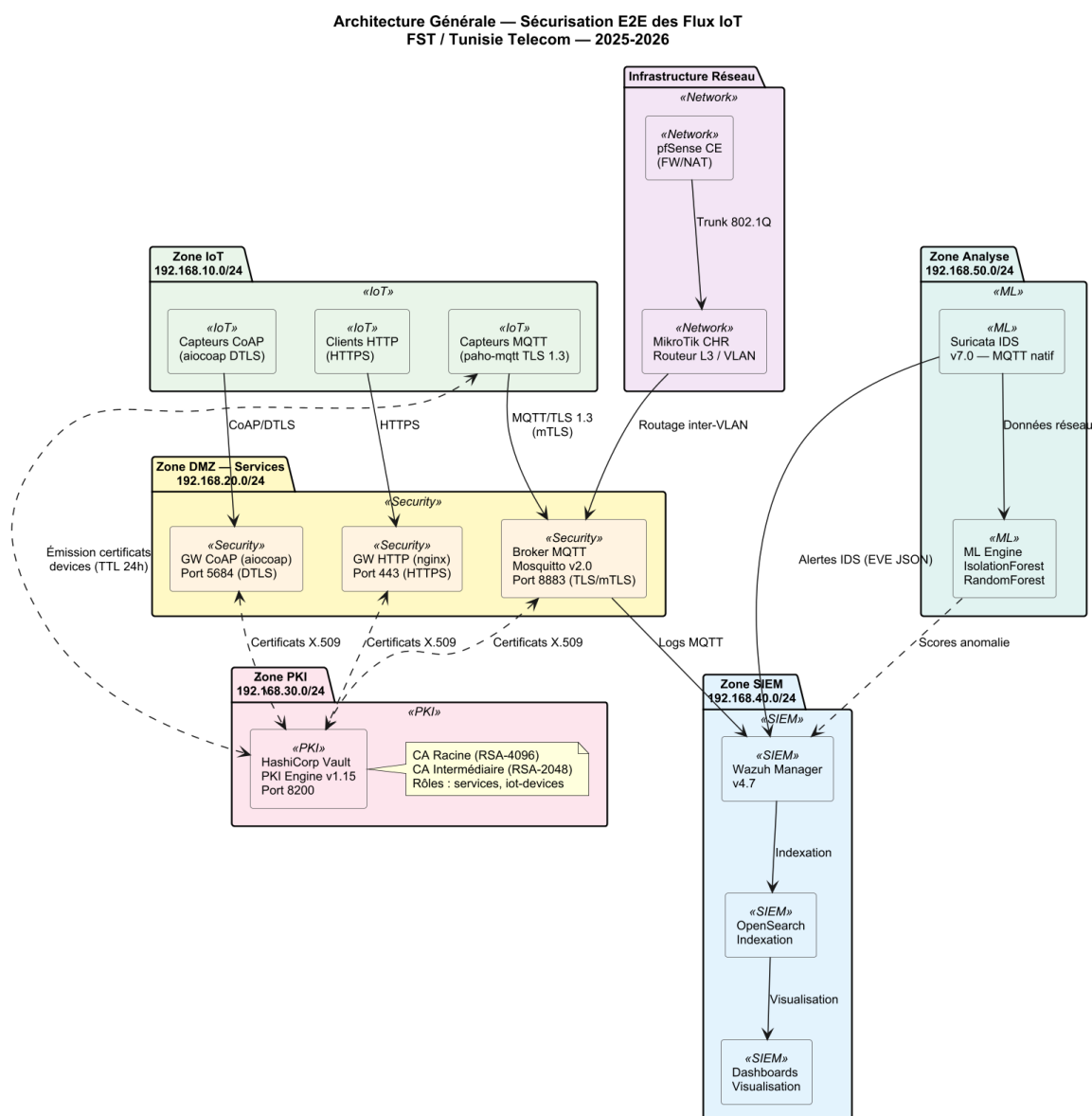


Fig. 3.4 : Vue d'ensemble de l'architecture de sécurisation proposée. *Abbreviations* : GW = passerelle applicative, FW = pare-feu.

3.5 Planification des sprints

Six sprints de deux semaines structurent la phase de réalisation. Le tableau 3.5 en présente les objectifs et les livrables. Cette planification précède intentionnellement le *backlog produit* : elle fournit la grille de lecture qui permet ensuite d'affecter les *User Stories* aux sprints concernés.

Tab. 3.5 : Planification des sprints

Sprint	Durée	Objectif principal
S1 – Cadrage & réseau	2 semaines	Topologie GNS3 segmentée, conteneurs de base, plan d'adressage.
S2 – PKI Vault	2 semaines	Vault déployé, CA racine + intermédiaire, rôles PKI, scripts d'émission.
S3 – Broker mTLS	2 semaines	Mosquitto en TLS 1.3+mTLS, ACL par certificat, tests d'authentification.
S4 – Flotte simulée	2 semaines	50 clients Python, génération de trafic, rejeu de 76 000 messages.
S5 – Détection IDS+SIEM	2 semaines	Règles Suricata MQTT, intégration Wazuh, dashboards.
S6 – ML & bilan	2 semaines	Modèles IsolationForest et RandomForest, évaluation et rédaction finale.

La figure 3.5 illustre l'enchaînement des six sprints, leurs dépendances et les jalons de validation associés. On observe que les sprints S1 à S3 forment un bloc séquentiel strict : la topologie réseau (S1) est un prérequis à l'installation de Vault (S2), elle-même prérequis à la configuration du broker mTLS (S3). À partir de S4, les flux peuvent se paralléliser partiellement : la flotte simulée et la chaîne de détection IDS+SIEM (S5) peuvent être développées simultanément dès lors que le broker est opérationnel, et le pipeline ML (S6) consomme les données capturées lors de S4–S5.

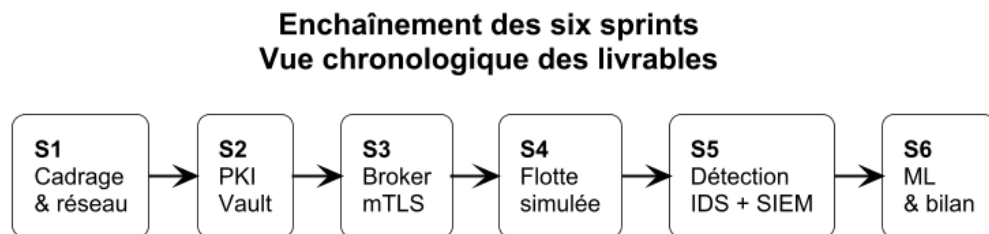


Fig. 3.5 : Enchaînement des six sprints et leurs dépendances

3.6 Backlog produit

Le *backlog produit* regroupe les *User Stories* priorisées selon la méthode **MoSCoW**. Chaque US est ensuite estimée en *story points*, sur la base d’une échelle de Fibonacci modifiée, puis rattachée à un sprint cible.

Tab. 3.6 : Backlog produit (extrait priorisé)

ID	Acteur	User Story	Prio.	SP	Sprint
US-01	Admin	Déployer une topologie réseau segmentée sous GNS3	M	8	S1
US-02	Admin	Définir un plan d’adressage par VLAN	M	3	S1
US-03	Admin	Démarrer Vault et initialiser une CA racine	M	5	S2
US-04	Admin	Provisionner une CA intermédiaire et des rôles PKI	M	8	S2
US-05	Admin	Émettre des certificats serveur/client à la demande	M	5	S2
US-06	Admin	Configurer Mosquitto en TLS 1.3 + mTLS	M	8	S3
US-07	Admin	Définir des ACL par identifiant client	M	5	S3
US-08	Dispositif	Établir une session mTLS et publier sur son topic	M	5	S3
US-09	Admin	Simuler 50 dispositifs hétérogènes	M	8	S4
US-10	Admin	Rejouer un dataset de 76 000 messages	S	5	S4
US-11	Analyste	Détecter une connexion non autorisée	M	5	S5
US-12	Analyste	Détecter un brute-force d’authentification	M	5	S5
US-13	Analyste	Visualiser les alertes corrélées dans Wazuh	M	5	S5
US-14	Analyste	Détecter une anomalie via IsolationForest	S	8	S6
US-15	Analyste	Classifier les attaques via Random Forest	S	8	S6
US-16	Analyste	Comparer ML vs règles IDS	C	5	S6

Soit un total estimé à 96 story points sur 6 sprints (~ 16 SP/sprint), conforme à la capacité d’une équipe solo avec encadrement.

Le backlog ainsi constitué traduit les objectifs en livrables concrets, chacun affecté à un sprint précis. Pour compléter cette conception, il est nécessaire de modéliser les interactions entre les composants : les diagrammes UML présentés dans la section suivante formalisent le simulateur de flotte, le cycle d’émission des certificats et le comportement d’un dispositif, fournissant une référence de conception pour la phase de réalisation.

3.7 Modélisation UML

3.7.1 Diagramme de classes du simulateur

Le simulateur de flotte (`fleet_sim.py`) est structuré en cinq classes réparties en deux groupes logiques. Le premier regroupe le cœur d’orchestration de la simulation : `FleetOrchestrator`, `DeviceSimulator` et `DeviceStats`. Le second rassemble les composants de configuration et de gestion PKI : `BrokerConfig`, `CertificateManager` et `CertBundle`. Le diagramme est présenté en deux parties afin d’en faciliter la lecture.

Partie 1 — Orchestration de la simulation. La figure 3.6 montre la hiérarchie de contrôle : `FleetOrchestrator` crée jusqu’à 50 instances `DeviceSimulator` (composition), chacune maintenant un agrégat `DeviceStats` pour la collecte des métriques (messages envoyés/échoués, débit, types d’attaques injectées). La méthode `_inject_dos_pattern()` de `DeviceSimulator` module dynamiquement le taux d’émission pour simuler les attaques par inondation du jeu de données.

Diagramme de Classes --- Partie 1 : Orchestration de la simulation

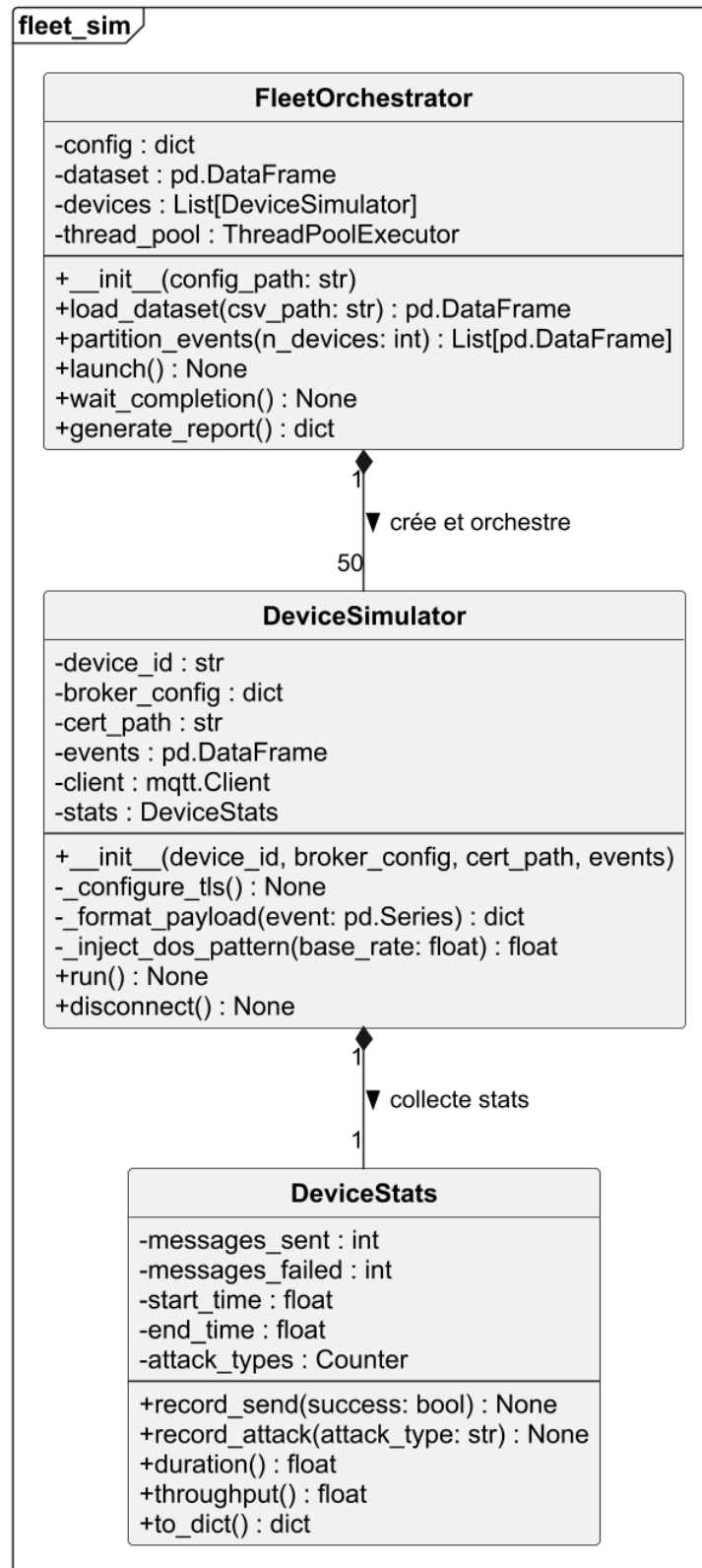


Fig. 3.6 : Diagramme de classes — Partie 1 : orchestration (FleetOrchestrator, DeviceSimulator, DeviceStats)

Partie 2 — Configuration, PKI et dépendances externes. La figure 3.7 détaille les composants de support : **BrokerConfig** encapsule les paramètres TLS du broker (hôte, port 8883, chemin du CA, QoS); **CertificateManager** interroge l'API Vault pour émettre, vérifier et renouveler les certificats X.509; **CertBundle** (data-class) transporte le triplet certificat/clé privée/chaîne CA. Les dépendances externes (**paho.mqtt.Client**, **pandas.DataFrame**) sont représentées en gris pour les distinguer du code du projet.

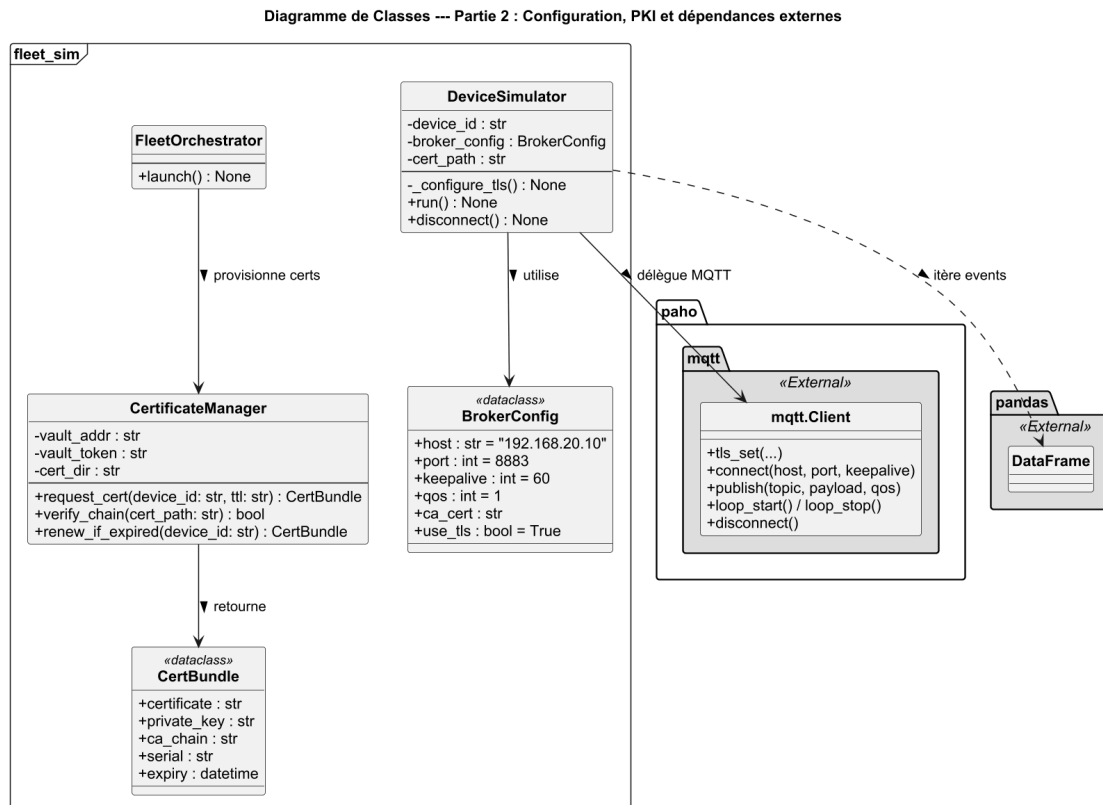


Fig. 3.7 : Diagramme de classes — Partie 2 : configuration, PKI et dépendances externes (**BrokerConfig**, **CertificateManager**, **CertBundle**)

3.7.2 Diagramme de séquence : émission d'un certificat client

La séquence est découpée en deux parties : l'initialisation de la PKI (opération réalisée une seule fois par l'administrateur) et l'émission effective d'un certificat pour chaque dispositif ou service.

Partie 1 — Initialisation de la PKI. La figure 3.8 couvre la mise en place de la hiérarchie à deux niveaux : activation du moteur PKI racine dans Vault (RSA-4096, TTL 10 ans, auto-signé et conservé *offline*), génération d'une CA intermédiaire dont le CSR est signé par la CA racine, puis import du certificat signé dans le *secrets engine* **pki_int**. Enfin, le rôle **iot-devices** est créé pour contraindre les émissions : TTL maximal de 30 jours, usage **clientAuth**, CN limité au pattern ***.iot.lab**.

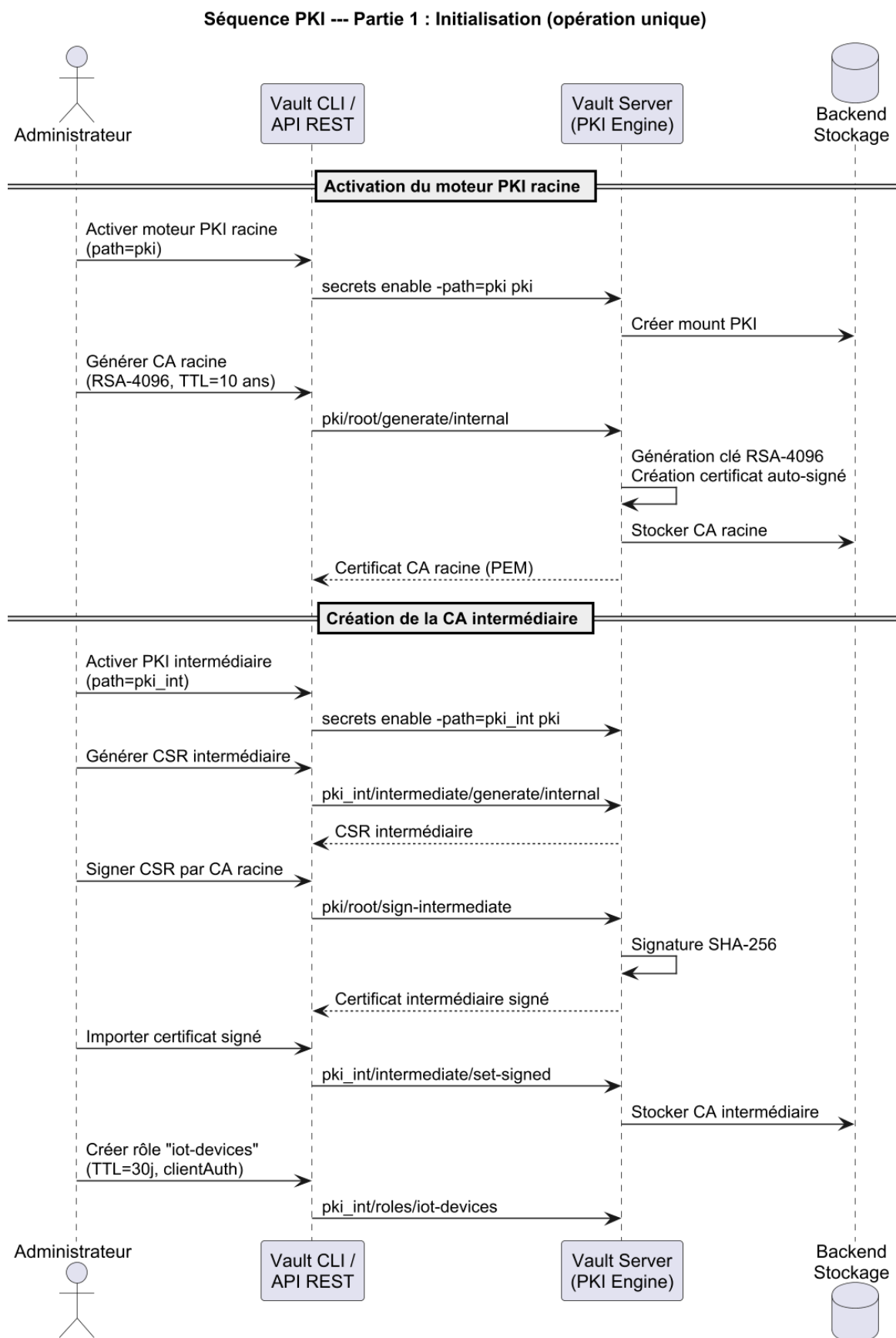


Fig. 3.8 : Séquence PKI — Partie 1 : initialisation (CA racine, CA intermédiaire, rôle)

Partie 2 — Émission par dispositif. La figure 3.9 décrit l'opération répétée pour chaque dispositif : (1) le script de provisionnement envoie une requête `POST` à l'API d'émission du rôle `iot-devices`, avec le CN et le TTL souhaité ; (2) Vault vérifie la policy `AppRole` du token présenté ; (3) Vault génère la paire de clés RSA-2048 et signe le certificat via la CA intermédiaire ; (4) le bundle (certificat + clé privée + chaîne CA) est retourné en JSON ; (5) le dispositif écrit les trois fichiers dans son système de fichiers et démarre un minuteur de renouvellement basé sur le TTL reçu (87 ms de bout en bout).

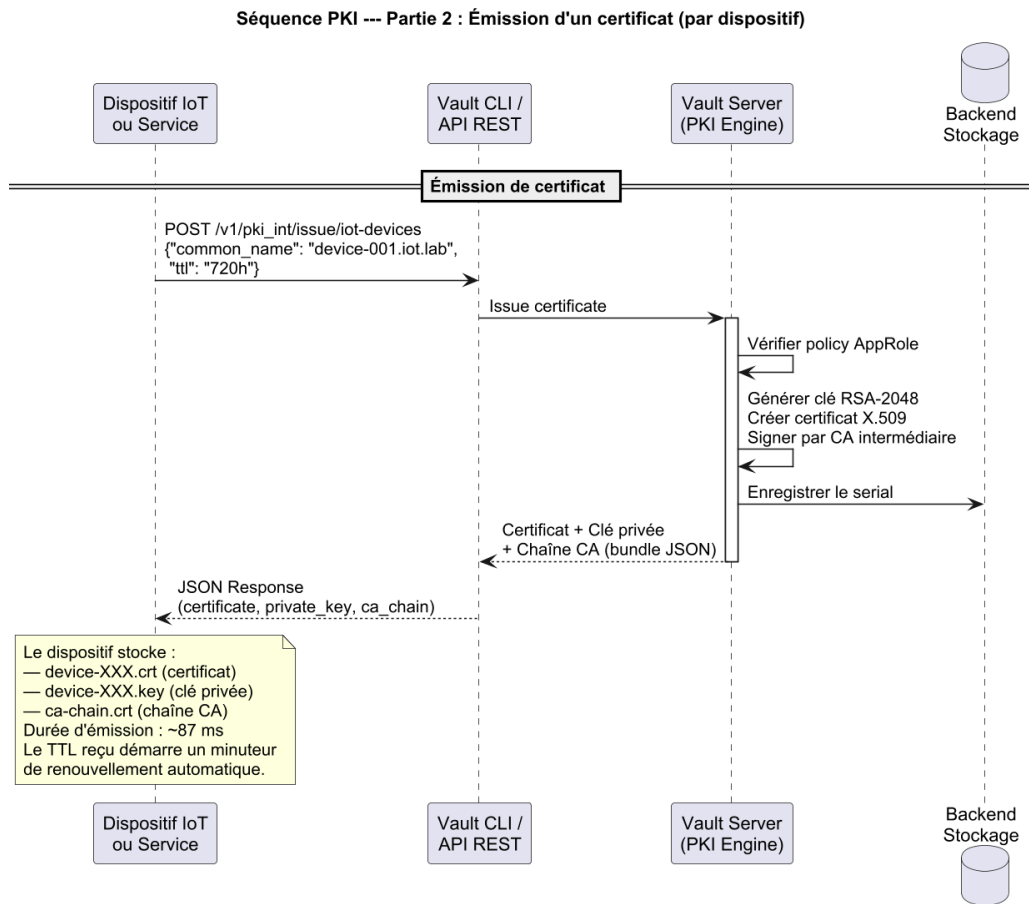


Fig. 3.9 : Séquence PKI — Partie 2 : émission d'un certificat client (~87 ms)

3.7.3 Diagramme d'activité : cycle de publication mTLS

Le cycle complet d'un dispositif simulé est décomposé en trois phases successives (figures 3.10, 3.11 et 3.12).

Phase 1 — Initialisation de la flotte. La figure 3.10 montre le démarrage du simulateur : le dataset CSV (76 000 événements) et la configuration du broker sont chargés, puis 50 threads sont instanciés en parallèle via un *fork*, un par dispositif simulé.

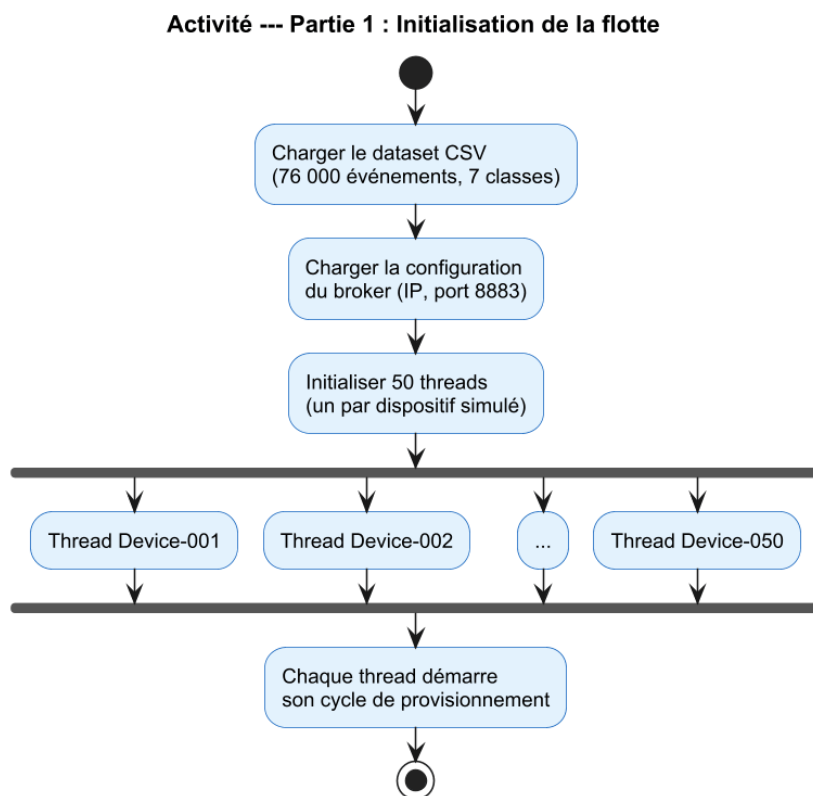


Fig. 3.10 : Activité — Phase 1 : initialisation des 50 threads de la flotte

Phase 2 — Provisionnement et connexion mTLS. La figure 3.11 détaille les étapes de provisionnement de chaque thread : chargement du certificat X.509 émis par Vault, validation des fichiers (une clé ou un certificat invalide provoque un arrêt immédiat), configuration du client MQTT en TLS 1.3, puis établissement de la connexion avec *handshake* mutuel. Un échec de connexion déclenche une attente en back-off exponentiel avant nouvelle tentative (maximum 3 essais), évitant une boucle de reconnexion aveugle.

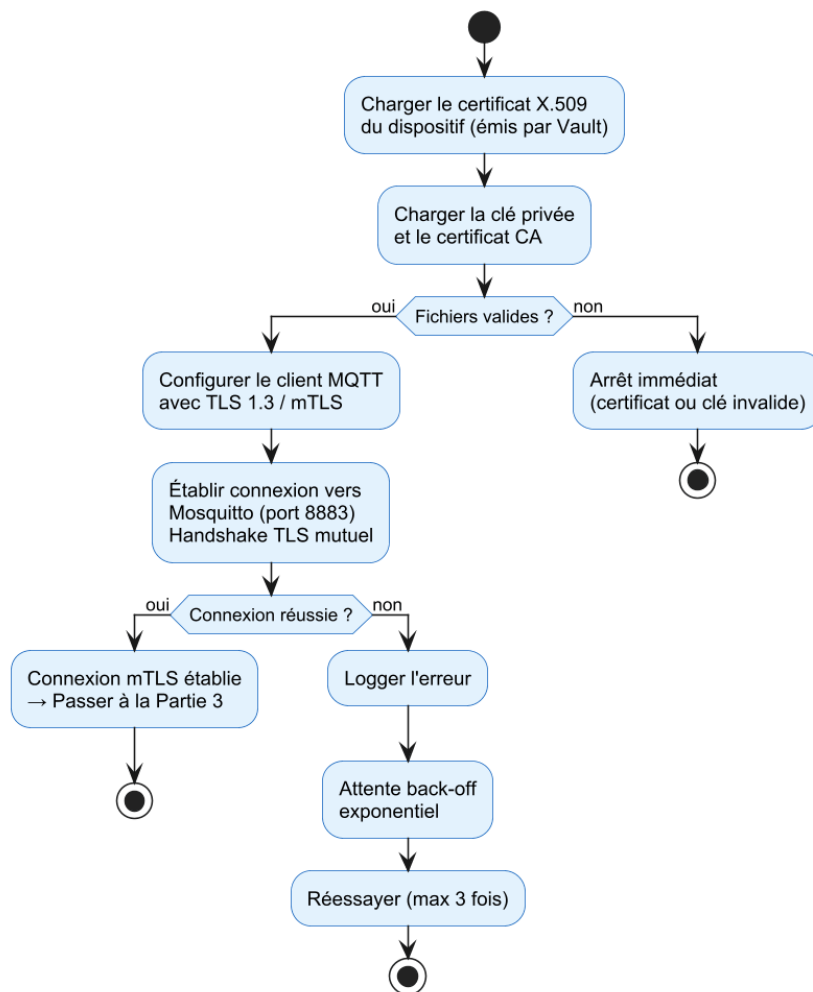
Activité --- Partie 2 : Provisionnement et connexion mTLS

Fig. 3.11 : Activité — Phase 2 : provisionnement du certificat et handshake mTLS

Phase 3 — Boucle de publication. La figure 3.12 représente la boucle principale : pour chaque événement du sous-ensemble CSV assigné, le payload JSON est formaté puis publié sur le topic `iot/sensors/device-XXX/{type}`. Les événements étiquetés comme attaques adaptent leur comportement (accélération de fréquence pour DoS, duplication pour Replay, modification de payload pour MiTM). Le nœud de bouclage se poursuit jusqu'à épuisement des événements, après quoi le client se déconnecte proprement et les statistiques sont agrégées.

Activité --- Partie 3 : Boucle de publication MQTT

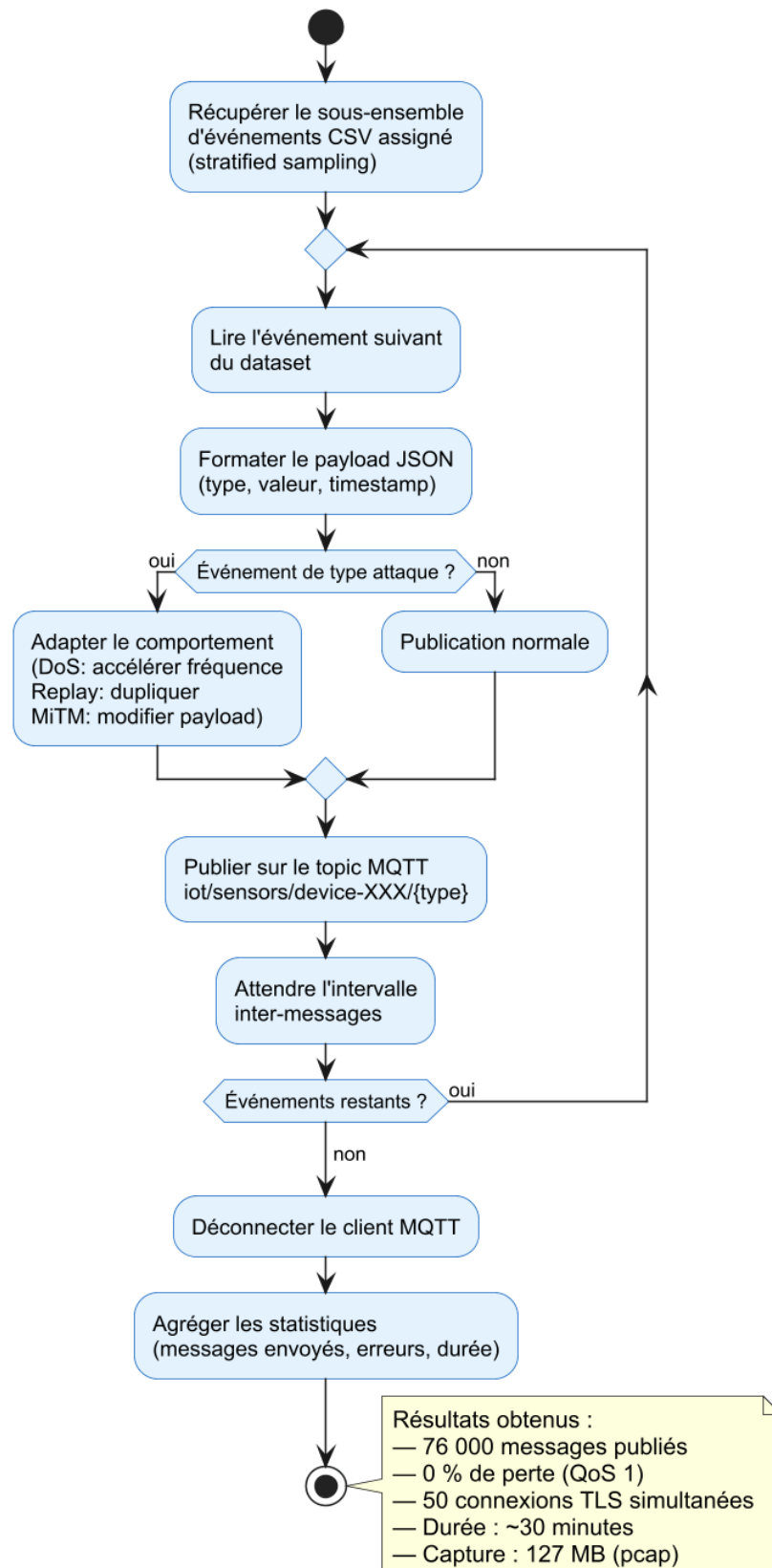


Fig. 3.12 : Activité — Phase 3 : boucle de publication MQTT et agrégation des résultats

3.8 Planification temporelle

Le diagramme de Gantt (figure 3.13) consolide l'ensemble des sprints sur la durée totale du projet. L'axe horizontal représente le temps en semaines ; les barres horizontales correspondent aux périodes de réalisation de chaque sprint. On observe que les trois premiers sprints occupent la moitié de la durée totale et constituent le socle technique : sans une PKI fonctionnelle (S2) et un broker sécurisé (S3), aucune donnée de trafic chiffré ne peut être générée ni analysée. Les flèches de dépendance illustrent les contraintes de précédence : S4 ne démarre qu'après la validation du broker (fin S3), et S6 ne peut débuter sans les captures de trafic produites par S4–S5.

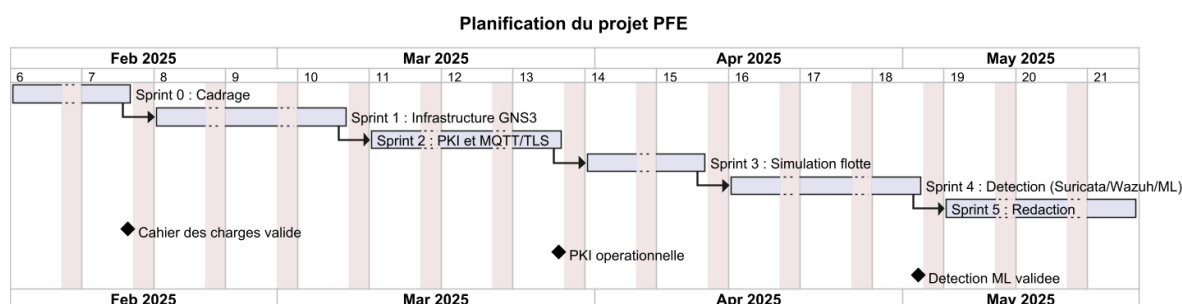


Fig. 3.13 : Diagramme de Gantt du projet

3.9 Conclusion

Ce chapitre a formalisé la conception du projet en articulant successivement le diagnostic de l'existant, la spécification des besoins, la vue d'ensemble de la solution retenue, les artefacts de pilotage et la modélisation du système. Cet ensemble fournit une base cohérente pour la phase de réalisation, en assurant la traçabilité entre les lacunes identifiées, les objectifs visés et les choix de conception adoptés. Le chapitre suivant décrit précisément **comment** chaque élément a été mis en œuvre : de l'environnement GNS3 à la PKI Vault, du broker mTLS au pipeline ML, en passant par les scénarios d'attaque.

Chapitre 4

Réalisation

4.1 Introduction

Ce chapitre présente la mise en œuvre concrète de l'architecture conçue précédemment. Nous décrivons successivement : l'environnement de développement, l'architecture globale et la topologie réseau, la PKI Vault, le broker MQTT en TLS/mTLS, les scénarios d'attaque retenus avec leurs contre-mesures, le pipeline d'apprentissage automatique (présenté *après* la couche de défense statique pour respecter une logique de défense en profondeur), puis les captures d'écran des interfaces clés (Vault UI, Wazuh, GNS3).

4.2 Environnement de développement

Le laboratoire est entièrement reproductible sur une station Windows 10/11 (16 Go de RAM minimum). Les principaux composants logiciels sont résumés dans le tableau 4.1.

Tab. 4.1 : Composants de l'environnement de développement

Composant	Version	Rôle
GNS3	2.2.x	Émulation de la topologie réseau
Docker Engine	24.x	Conteneurs services et clients IoT
HashiCorp Vault	1.15	PKI à deux niveaux et secrets engine
Eclipse Mosquitto	2.0	Broker MQTT avec TLS 1.3 + mTLS
Suricata	7.0	IDS réseau, parser MQTT
Wazuh	4.7	SIEM, agrégation et corrélation
pfSense	2.7	Pare-feu inter-VLAN
MikroTik RouterOS	7.x	Routage / NAT
Python	3.11	Simulateur de flotte, ML
scikit-learn	1.4	Algorithmes ML

Ces composants ne fonctionnent pas de manière indépendante : ils s'articulent en une **architecture de défense en profondeur** où chaque couche renforce les autres. La section suivante décrit cette organisation globale et le flux de données de bout en bout, avant d'entrer dans le détail de chaque brique.

4.3 Architecture globale et topologie réseau

4.3.1 Architecture de déploiement

L'architecture de déploiement est la traduction concrète, sur la machine hôte, des briques conceptuelles décrites au chapitre 1 (figure 3.4). Elle est présentée en deux parties pour en faciliter la lecture : la couche réseau et les stacks producteurs de données (Partie 1), puis les stacks consommateurs et d'analyse (Partie 2).

Note

Différence entre la figure 3.4 (Ch. 1) et les figures 4.1–4.2 (Ch. 4) La figure 3.4 est une vue **conceptuelle** : elle présente les zones réseau logiques (IoT, DMZ, PKI, SIEM, Analyse), les flux de données et les technologies retenues. Elle répond à la question « *quoi ?* ».

Les figures 4.1 et 4.2 constituent une vue **d'implémentation** : elles matérialisent la réalisation concrète sur la machine hôte (Intel i3-1215U / 16 Go RAM, Windows 11) en détaillant :

- les stacks Docker dans la VM GNS3 (*IoT Simulée, Broker/Gateway, PKI, SIEM, ML/Analyse*) ;
- les ports d'écoute réels (8883 MQTT/TLS, 8200 Vault, 9200 OpenSearch, 5601 Dashboard, 1514/1515 Wazuh) ;
- les relations inter-services : port miroir MikroTik→Suricata, EVE JSON→Wazuh, API REST→Vault.

Elles répondent à la question « *comment ?* » et constituent la référence de reproduction de l'environnement.

Partie 1 — Infrastructure réseau, couche IoT et couche Broker/IDS. La figure 4.1 montre les deux niveaux d'exécution du laboratoire : (i) la **VM GNS3** (VMware Workstation, 4 Go RAM) héberge pfSense (firewall/NAT, quatre VLANs) et MikroTik CHR (switch L3, port miroir vers Suricata) ; (ii) le **Docker Engine** exécute le simulateur de flotte (50 threads, 76 000 événements CSV, VLAN 10) et Mosquitto (TLS 1.3 + mTLS + ACL deny-by-default, port 8883, VLAN 20 DMZ).

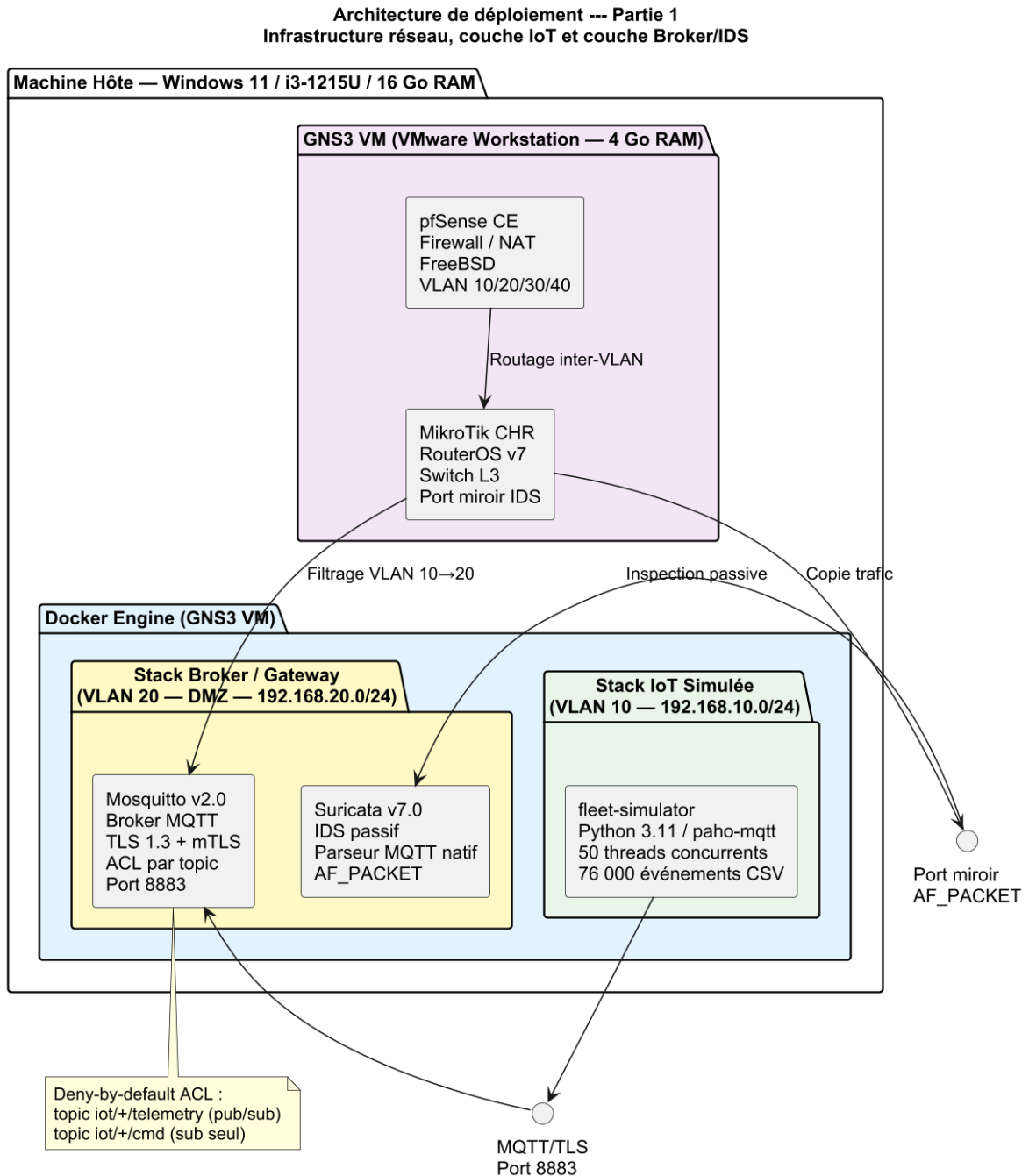


Fig. 4.1 : Architecture de déploiement — Partie 1 : infrastructure réseau (pfSense, MikroTik), stack IoT simulée et stack Broker/IDS (Mosquitto, Suricata)

Partie 2 — Stack PKI, SIEM et ML/Analyse + flux de données. La figure 4.2 détaille les stacks de traitement backend : **Vault 1.15** (PKI Engine, CA racine RSA-4096 offline + CA intermédiaire active, port 8200) ; **Wazuh 4.7** (Manager + Indexer OpenSearch + Dashboard, ports 1514/9200/5601) qui reçoit les alertes EVE JSON de Suricata via Filebeat et les logs Mosquitto via Syslog ; le **moteur ML** (IsolationForest + RandomForest, 12 features réseau) alimenté par les sorties Suricata et Wazuh.

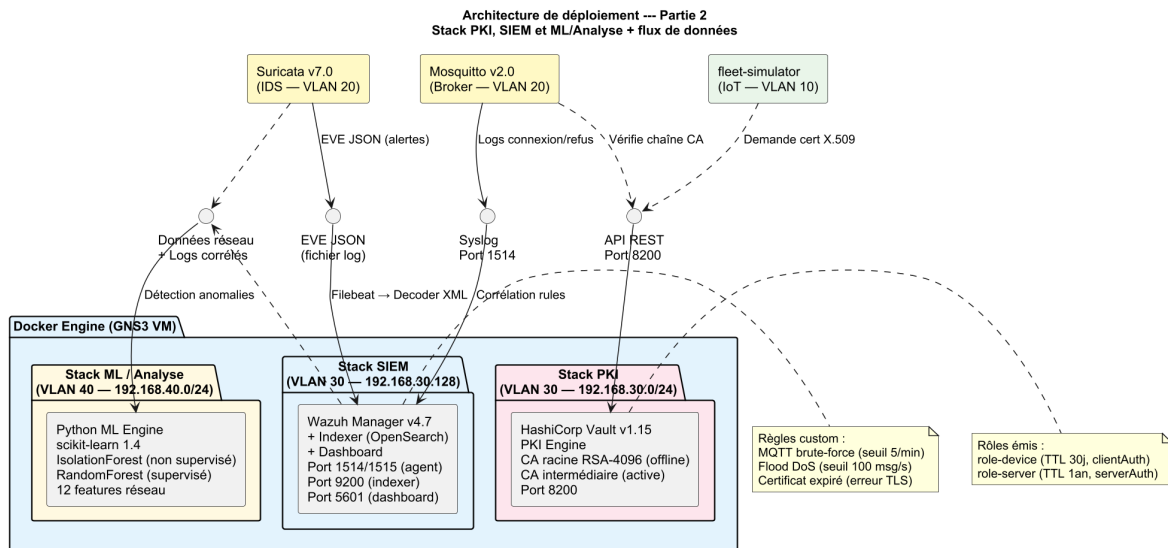


Fig. 4.2 : Architecture de déploiement — Partie 2 : stack PKI (Vault), stack SIEM (Wazuh) et stack ML/Analyse + interfaces de données

4.3.2 Topologie réseau

La topologie réseau déployée dans GNS3 est présentée en quatre parties pour plus de clarté : la vue globale de l'infrastructure (P1), les zones productrices de données MQTT (P2), les zones de traitement PKI et SIEM (P3), et la politique de filtrage inter-VLAN (P4).

Partie 1 — Infrastructure réseau et segmentation VLAN. La figure 4.3 montre la chaîne physique et logique complète : Internet, pfSense CE pour le filtrage et la traduction d'adresses, puis MikroTik CHR v7 comme routeur L3 sur trunk 802.1Q. Le MikroTik dessert quatre VLANs. Le sous-réseau IoT est 192.168.10.0/24. Ceux de la DMZ, de la PKI et du SIEM sont respectivement 192.168.20.0/24, 192.168.30.0/24 et 192.168.40.0/24. Le port miroir sur l'interface DMZ copie passivement le trafic vers Suricata.

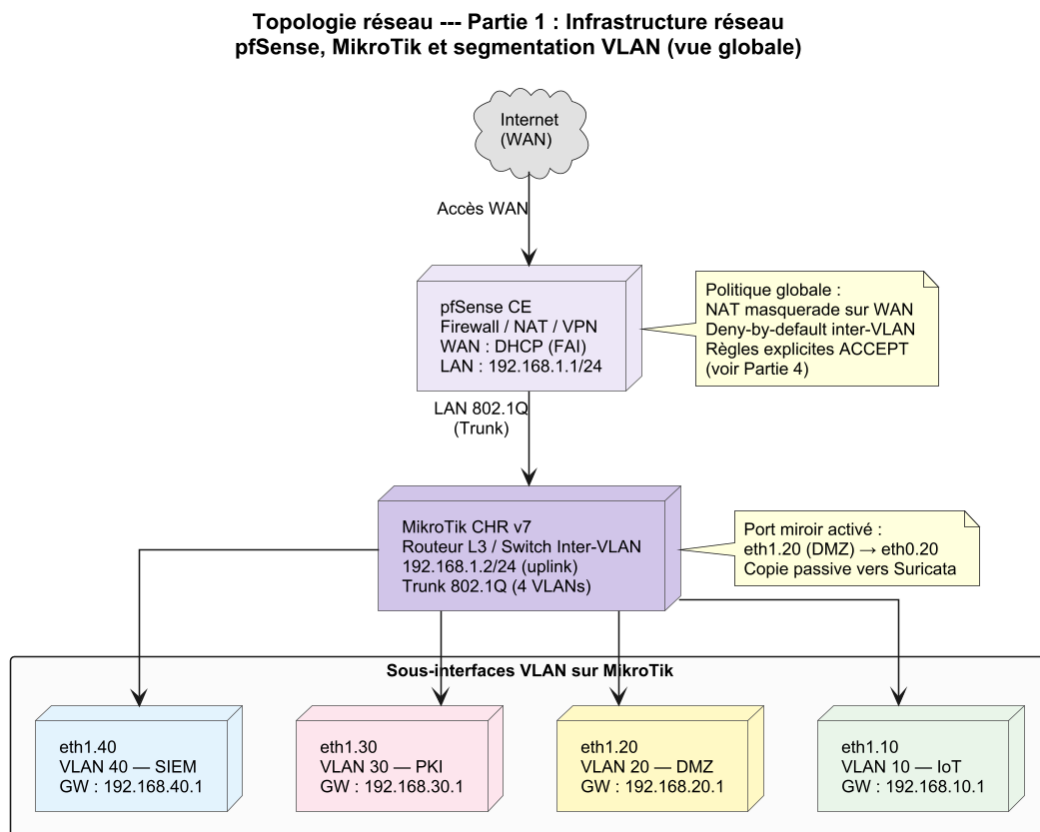


Fig. 4.3 : Topologie réseau — Partie 1 : infrastructure réseau, pfSense, MikroTik et segmentation en 4 VLANs

Partie 2 — VLAN 10 (Zone IoT) et VLAN 20 (Zone DMZ). La figure 4.4 détaille les deux zones productrices de trafic. Le VLAN 10 (192.168.10.0/24) concentre le simulateur de flotte (192.168.10.100) et les 50 dispositifs virtuels (.11–.60), chacun porteur d'un certificat X.509 individuel. Le VLAN 20 DMZ (192.168.20.0/24) héberge Mosquitto v2.0 (192.168.20.10, port 8883, TLS 1.3 + mTLS + ACL deny-by-default) et Suricata v7.0 en mode AF_PACKET passif.

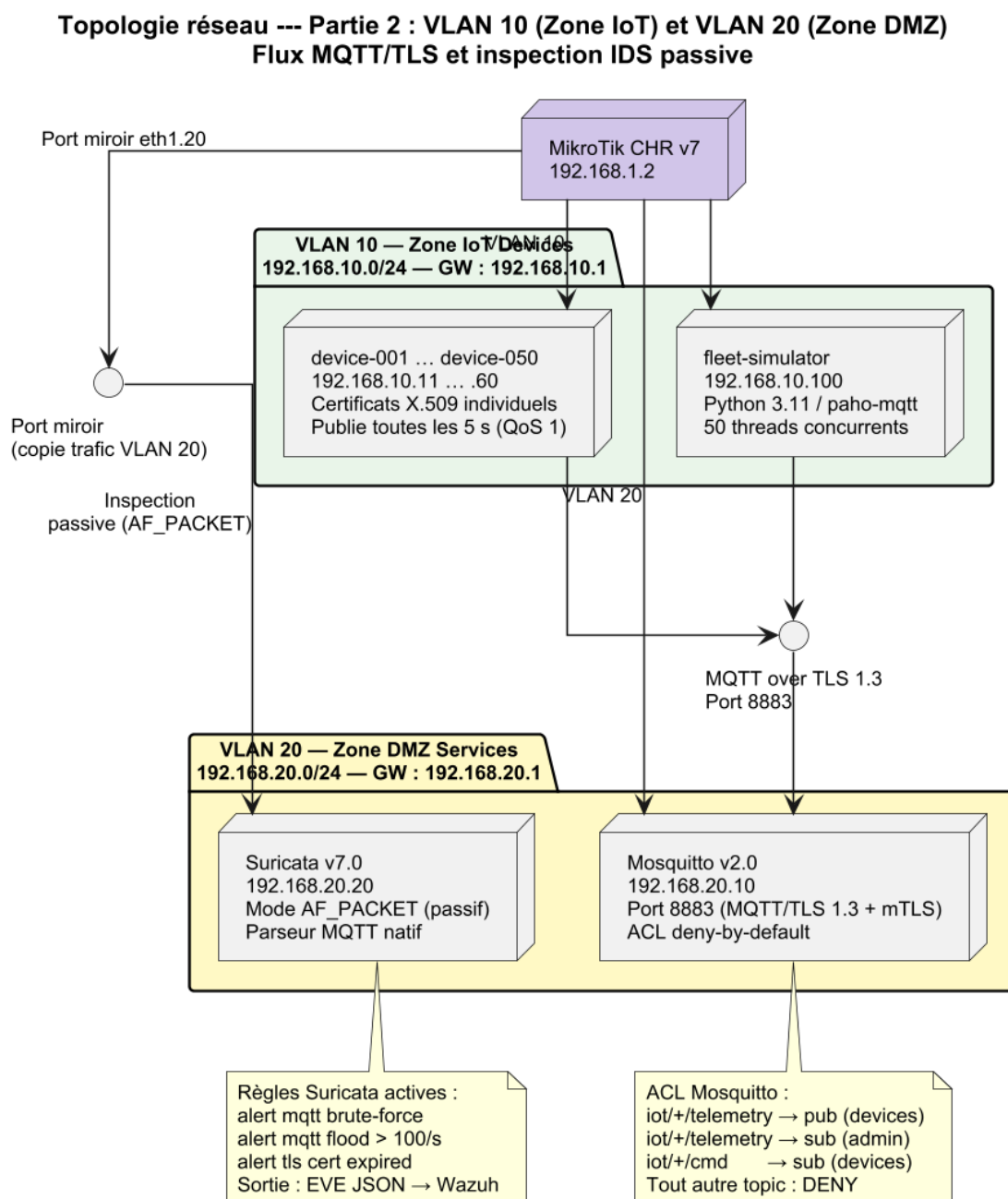


Fig. 4.4 : Topologie réseau — Partie 2 : VLAN 10 Zone IoT (fleet-sim, 50 devices) et VLAN 20 Zone DMZ (Mosquitto, Suricata)

Partie 3 — VLAN 30 (Zone PKI) et VLAN 40 (Zone SIEM). La figure 4.5 présente les zones de traitement backend. Le VLAN 30 isole Vault 1.15, accessible sur 192.168.30.10:8200, et héberge la hiérarchie de certification à deux niveaux. Le sous-réseau associé est 192.168.30.0/24. Le VLAN 40 regroupe Wazuh 4.7 sur 192.168.40.10, ses services 1514, 9200 et 5601, ainsi que le moteur ML sur 192.168.40.20. Son sous-réseau est 192.168.40.0/24. Les flux entrants proviennent d'EVE JSON via Filebeat, des logs Mosquitto transmis par Syslog/1514 et des requêtes API REST adressées à Vault depuis les VLANs 10 et 20.

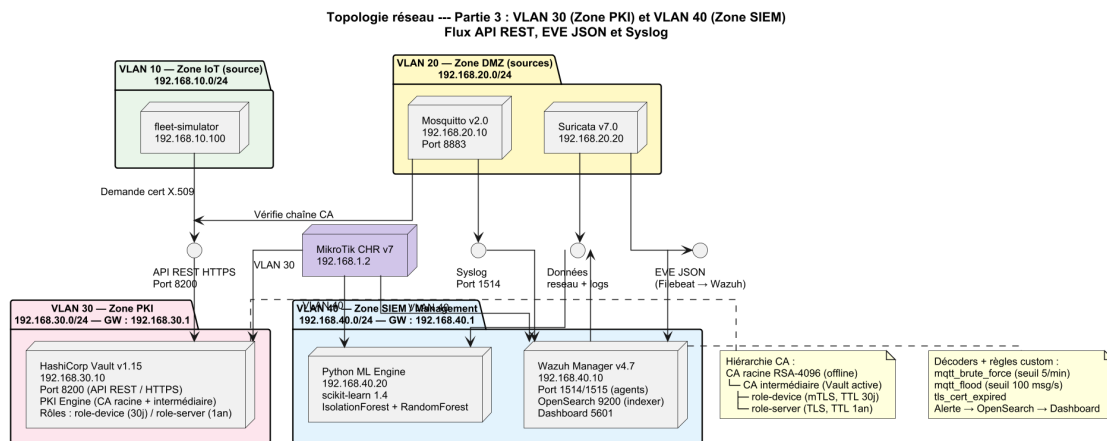


Fig. 4.5 : Topologie réseau — Partie 3 : VLAN 30 Zone PKI (Vault) et VLAN 40 Zone SIEM (Wazuh, ML Engine) + flux de données

Partie 4 — Politique de filtrage inter-VLAN (deny-by-default). La figure 4.6 synthétise les règles pfSense : seuls quatre flux sont explicitement autorisés, à savoir IoT vers DMZ sur le port 8883, IoT vers PKI sur 8200, DMZ vers SIEM sur 1514 et PKI vers SIEM sur 1514. Tous les autres flux inter-VLAN sont bloqués sans notification, en particulier l'accès direct de l'IoT vers le SIEM et celui de la DMZ vers la PKI.

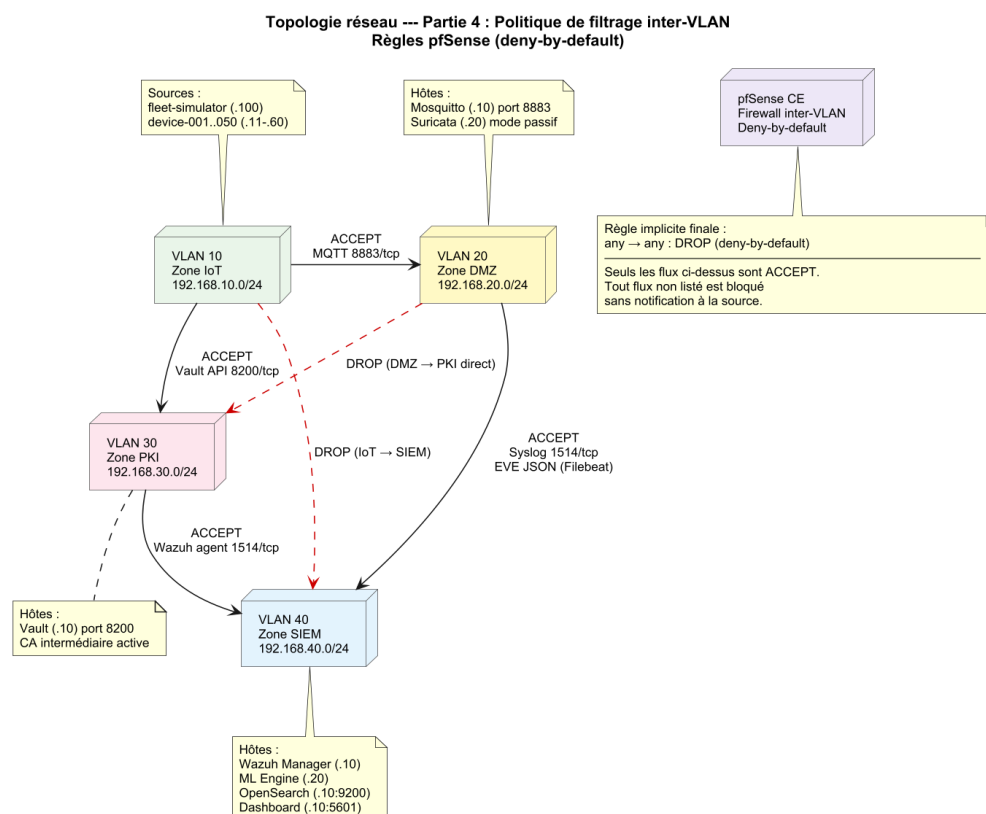


Fig. 4.6 : Topologie réseau — Partie 4 : politique de filtrage inter-VLAN (ACCEPT/DROP) sur pfSense

Le flux complet est décomposé en cinq phases présentées séparément (figures 4.7–4.11) afin d'en améliorer la lisibilité.

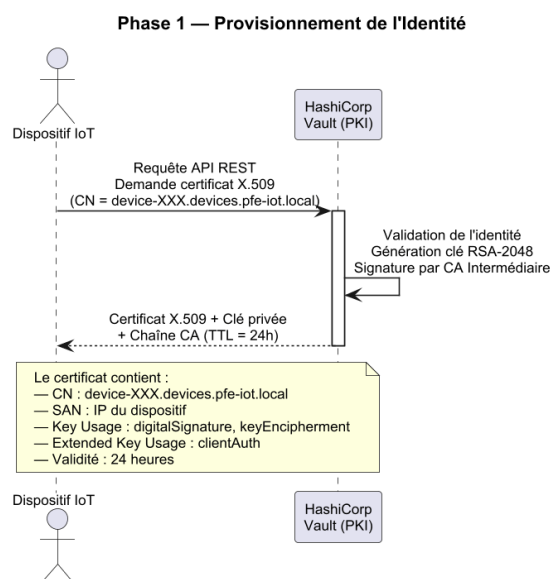


Fig. 4.7 : Flux E2E — Phase 1 : provisionnement de l'identité

Cette phase initiale décrit la demande d'un certificat client par le dispositif auprès

de Vault, depuis l'authentification AppRole jusqu'à la signature par la CA intermédiaire et la livraison sécurisée du couple clé/certificat.

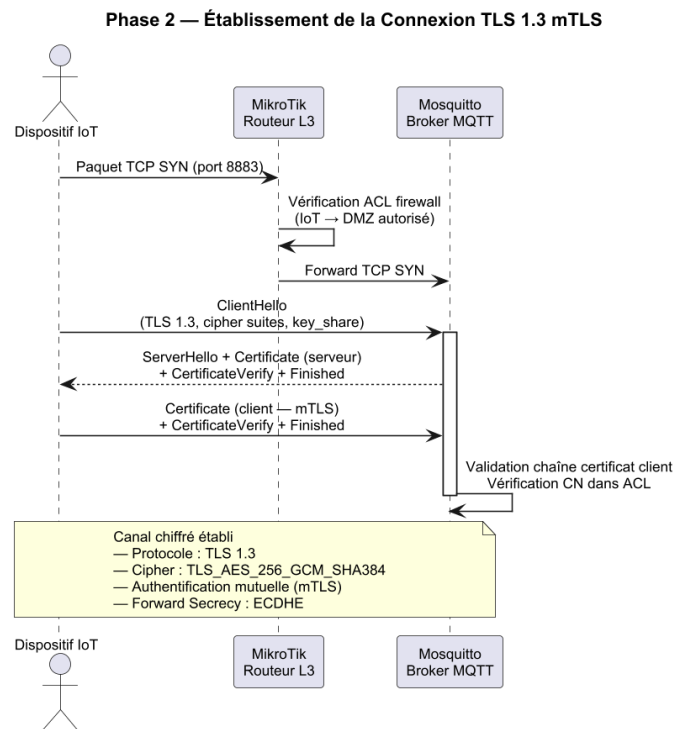


Fig. 4.8 : Flux E2E — Phase 2 : établissement de la connexion TLS 1.3 mTLS

Le schéma détaille les échanges du *handshake* TLS 1.3 mutuel entre le dispositif et le broker, en mettant en évidence la vérification croisée des certificats et la dérivation des clés de session.

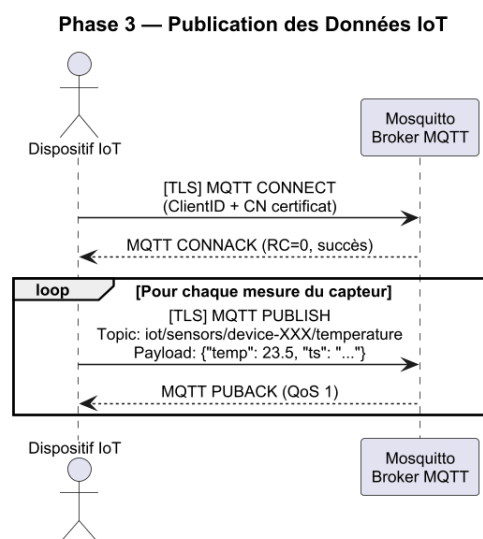


Fig. 4.9 : Flux E2E — Phase 3 : publication des données IoT

Une fois la session sécurisée établie, le dispositif publie ses messages MQTT sur ses

topics autorisés ; le broker applique alors les ACL liées au *Common Name* avant toute redistribution aux abonnés.

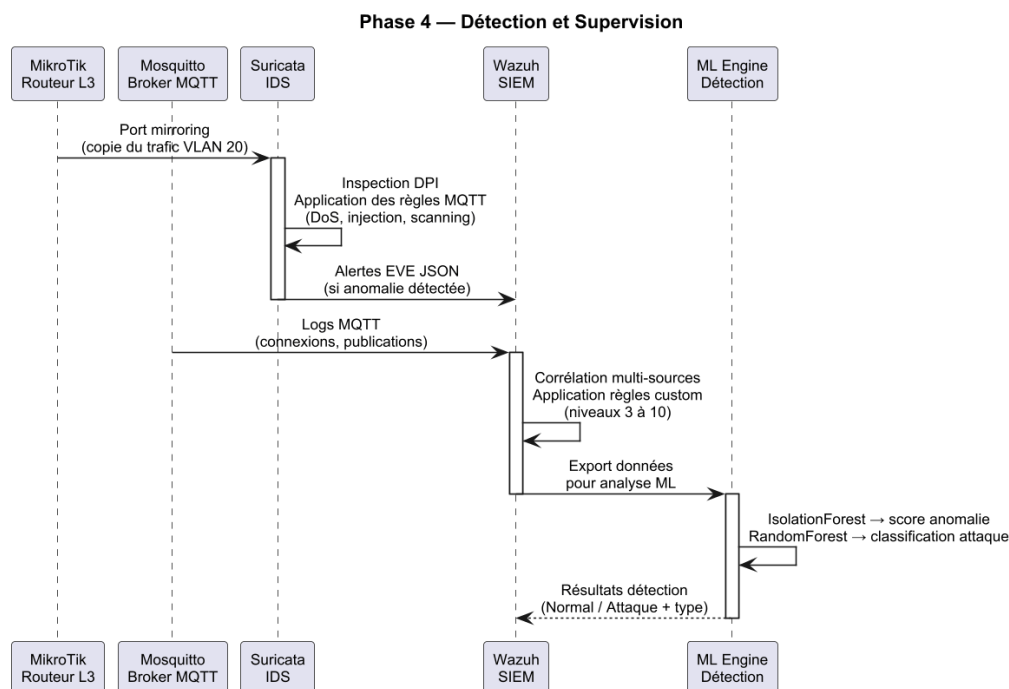


Fig. 4.10 : Flux E2E — Phase 4 : détection et supervision

Cette phase illustre la chaîne d’observation : les flux sont analysés par Suricata, les événements transmis à Wazuh pour corrélation, et les alertes restituées à l’analyste via le tableau de bord.

Phase 5 — Rotation Automatique des Certificats

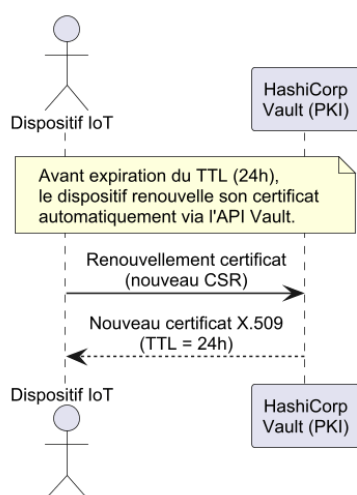


Fig. 4.11 : Flux E2E — Phase 5 : rotation automatique des certificats

La dernière phase montre comment Vault déclenche, à l’approche de l’expiration ou suite à une révocation, la régénération d’un certificat et son redéploiement transparent

sur le dispositif sans interruption de service. La rotation est automatisée par un agent de renouvellement côté dispositif (minuteur basé sur le TTL reçu lors de l'émission), qui relance la séquence de la phase 1 avant expiration, garantissant une continuité de service sans intervention manuelle.

4.4 Infrastructure à clés publiques (Vault)

4.4.1 Hiérarchie

La PKI suit le modèle à deux niveaux préconisé par le NIST [11] : une CA racine *offline* (générée hors ligne, conservée chiffrée) signe une unique CA intermédiaire en ligne, gérée par Vault [16]. La figure 4.12 illustre cette hiérarchie.

Le diagramme distingue les trois niveaux de confiance de la hiérarchie : (i) la **CA racine** (RSA-4096, self-signed, TTL 10 ans) est conservée hors ligne et ne signe que les CA intermédiaires ; sa clé privée n'est jamais exposée dans la VM GNS3 ; (ii) la **CA intermédiaire** (RSA-2048, TTL 5 ans) est embarquée dans le *secrets engine PKI* de Vault et signe les certificats opérationnels à la demande via API REST ; (iii) les **certificats finaux** sont émis selon deux rôles distincts : **role-server** (TTL 90 jours, usage `serverAuth`, pour Mosquitto et Vault), et **role-device** (TTL 30 jours, usage `clientAuth`, CN contrôlé au pattern `dev-XXX.iot.lab`). La séparation des rôles empêche qu'un certificat de service soit utilisé pour s'authentifier comme dispositif, et inversement.

4.4.2 Rôles Vault et émission

Deux rôles PKI ont été définis : **role-server** (TTL 90 jours, usage `server_auth`) et **role-device** (TTL 30 jours, usage `client_auth`, *Common Name* contraint au pattern `dev-XXX.iot.lab`). L'émission d'un certificat client se réduit à un appel HTTP authentifié par jeton AppRole.

```
1 vault write -format=json pki_int/issue/role-device \
2   common_name="dev-042.iot.lab" \
3   ttl="720h"
```

Listing 4.1: Émission d'un certificat client via Vault

Le choix des TTL résulte d'une analyse du compromis **sécurité / coût opérationnel** : un TTL court réduit la fenêtre d'exposition en cas de vol de clé privée, mais impose une rotation fréquente, source de charge et d'erreur. Pour les services (broker, Vault), un TTL de 90 jours équilibre la stabilité des configurations et la rotation régulière ; pour les dispositifs, un TTL de 30 jours est préférable car ces objets ont un cycle de vie moins maîtrisé et la rotation est automatisée par Vault, rendant son surcoût négligeable. Ce choix est cohérent avec les recommandations du NIST [11] qui

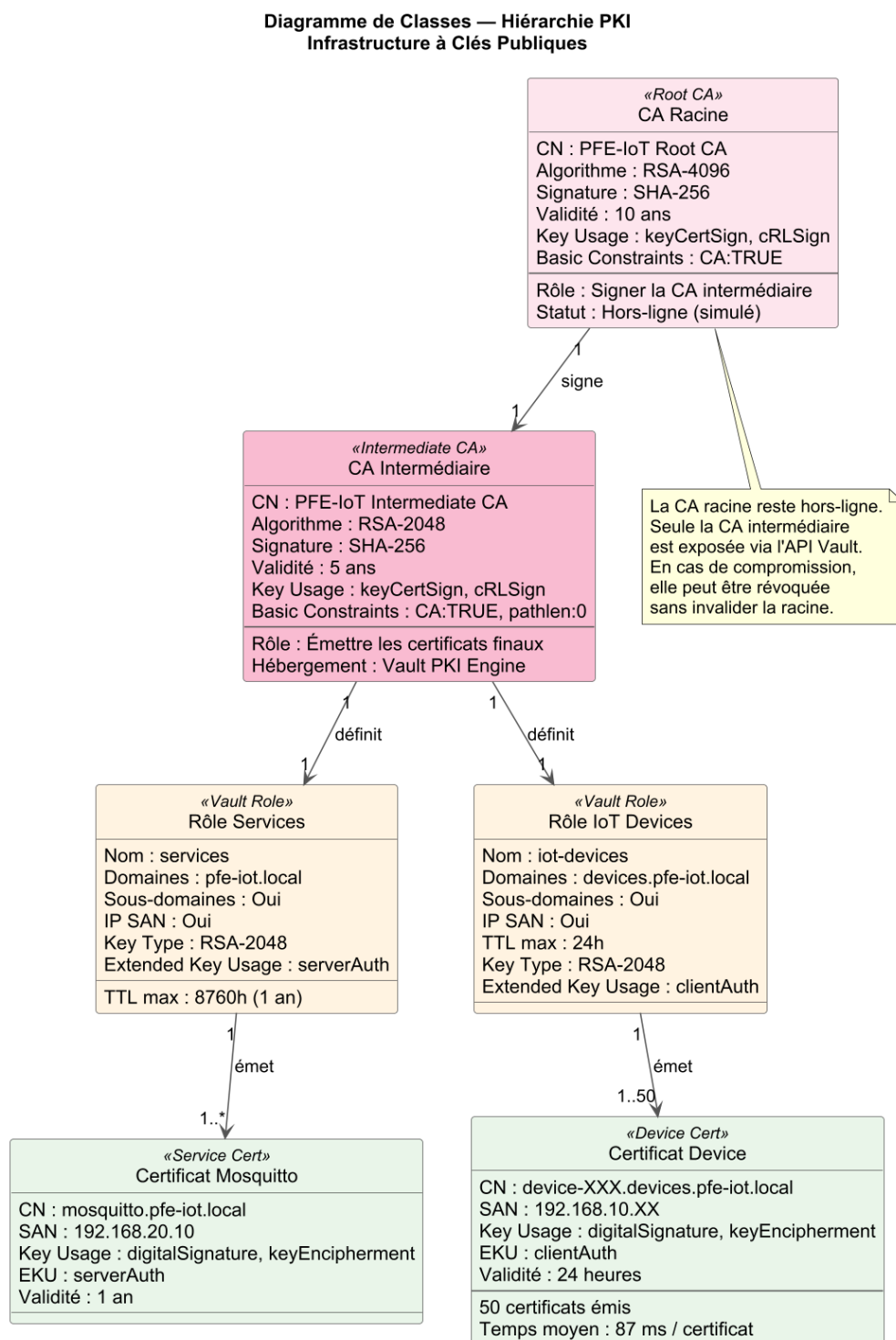


Fig. 4.12 : Hiérarchie de la PKI à deux niveaux (CA racine *offline* + CA intermédiaire Vault)

préconise des TTL inférieurs à 90 jours pour les certificats d'entité terminale dans les environnements IoT.

La PKI étant opérationnelle, chaque composant – dispositif comme service – dispose d'une identité cryptographique vérifiable par tous les pairs. C'est sur cette fondation que repose la couche transport : le broker MQTT peut désormais exiger la présentation d'un certificat signé par la CA intermédiaire avant d'accepter toute connexion, comme décrit dans la section suivante.

4.5 Broker MQTT en TLS 1.3 + mTLS

Mosquitto est configuré [15] pour exiger un certificat client, associer l'identité certifiée au nom d'utilisateur et appliquer un fichier d'ACL liant chaque *Common Name* à ses topics autorisés. La figure 4.13 retrace le *handshake* TLS 1.3 mutuel observé entre un dispositif et le broker.

Le diagramme de séquence se lit selon quatre phases distinctes :

1. **ClientHello** (dispositif → broker) : le dispositif annonce les suites cryptographiques qu'il supporte et propose les paramètres de partage de clé (ECDHE) ;
2. **ServerHello + CertificateVerify** (broker → dispositif) : le broker sélectionne la suite, transmet son certificat X.509 signé par la CA Vault et prouve la possession de sa clé privée ;
3. **Certificate (client)** (dispositif → broker) : le dispositif présente son propre certificat (rôle `role-device`, CN `dev-XXX.iot.lab`) ; le broker vérifie sa validité, son appartenance à la chaîne PKI et l'absence de révocation dans la CRL Vault ;
4. **Finished + Application Data** : les deux parties dérivent les clés de session symétriques (HKDF sur secret ECDHE) et la session chiffrée est établie en **1-RTT**.

L'authentification mutuelle garantit qu'aucun dispositif sans certificat valide ne peut accéder au broker, et qu'aucun broker frauduleux ne peut se faire passer pour Mosquitto.

L'architecture de confiance est ainsi opérationnelle : les identités sont gérées par Vault, le canal de communication est chiffré et les accès applicatifs sont restreints par ACL par certificat. Pour valider la robustesse de cet ensemble, six scénarios d'attaque ont été conçus en s'appuyant directement sur les vecteurs identifiés dans l'état de l'art : interception, usurpation d'identité, déni de service et exfiltration applicative.

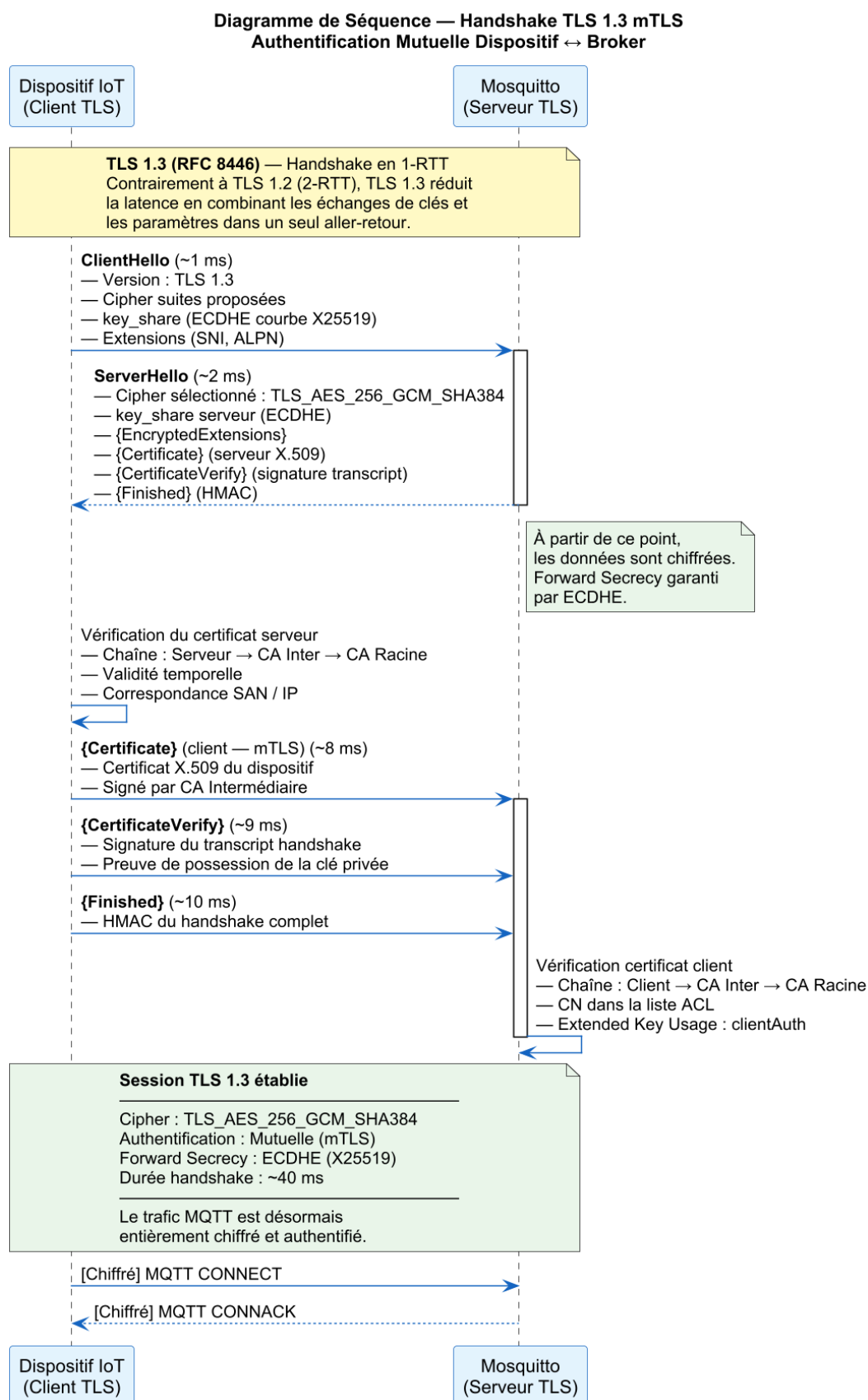


Fig. 4.13 : Handshake TLS 1.3 avec authentification mutuelle (mTLS)

4.6 Scénarios d’attaque et contre-mesures

Six scénarios ont été conçus pour couvrir les principales catégories d’attaques applicables à un broker MQTT. Chaque scénario est défini par un *vecteur*, une *méthode de détection* et une *contre-mesure*. Les scénarios sont rejoués depuis la flotte simulée et corroborés par les alertes Suricata/Wazuh.

Scénario 1 – Man-in-the-Middle (TLS désactivé)

Vecteur : un dispositif tente de se connecter sur le port 1883 (MQTT en clair) au lieu de 8883 (TLS). Un attaquant en position MITM peut alors capturer puis modifier les messages.

Détection : règle Suricata sur tout flux TCP/1883.

Contre-mesure : pare-feu pfSense bloquant 1883 . broker écoutant uniquement sur 8883 avec `require_certificate`.

Scénario 2 – Sniffing MQTT en clair

Vecteur : interception passive du trafic MQTT non chiffré sur le segment IoT.

Détection : alerte Suricata si un PUBLISH apparaît hors de la session TLS.

Contre-mesure : TLS 1.3 obligatoire. Les suites cryptographiques sont restreintes conformément à la RFC 9325 [14].

Scénario 3 – Brute-force d’authentification

Vecteur : 500 tentatives de `CONNECT` en moins de 60 s avec des certificats forgés.

Détection : règle Suricata MQTT (seuil de `CONNECT` par IP source).

Contre-mesure : certificats clients signés par CA Vault . *rate-limit* pfSense sur le port 8883.

Scénario 4 – Exfiltration via topic non autorisé

Vecteur : un dispositif compromis tente de `SUBSCRIBE` à `tt/admin/#`.

Détection : log Mosquitto `ACL denied` . règle Wazuh corrélant N occurrences/15 min.

Contre-mesure : ACL stricte par *Common Name*, granularité au topic.

Scénario 5 – Déni de service par flood `CONNECT`

Vecteur : client malveillant ouvrant plusieurs milliers de connexions TLS partielles.

Détection : alertes Suricata sur taux d’ouvertures TLS anormal.

Contre-mesure : `max_connections` Mosquitto, `state-table-limit` pfSense, black-list dynamique Wazuh.

Scénario 6 – Compromission de certificat client

Vecteur : certificat `dev-017` volé puis réutilisé depuis une IP non habituelle.

Détection : module ML (Isolation Forest) signalant l'écart comportemental . corrélation Wazuh IP $\leftrightarrow$ identité.

Contre-mesure : révocation Vault (`vault write pki_int/revoke serial=...`) et *rotation* immédiate de la CRL.

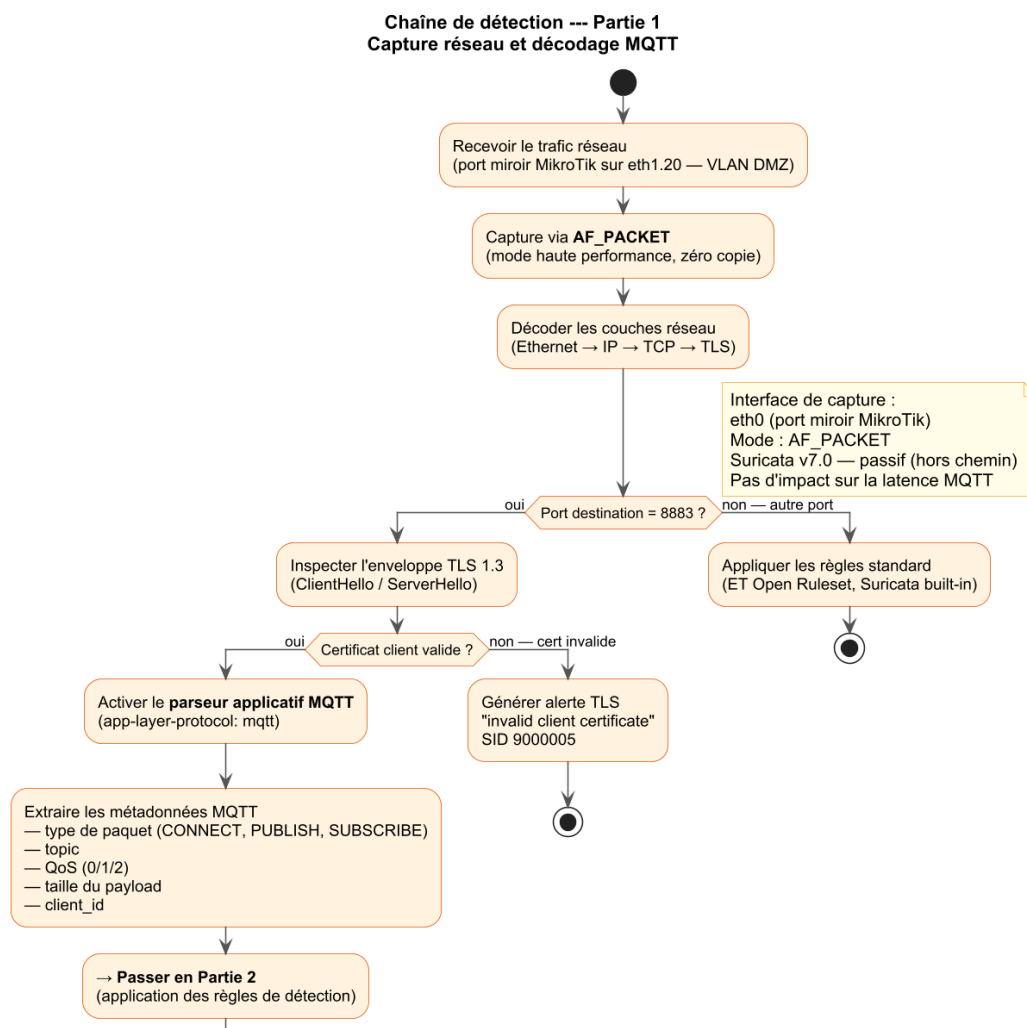


Fig. 4.14 : Chaîne de détection Suricata → Wazuh — Partie 1 : capture AF_PACKET et décodage MQTT

La première étape (figure 4.14) couvre l'inspection passive du trafic : Suricata reçoit une copie du trafic VLAN 20 via le port miroir MikroTik (AF_PACKET, zéro copie, aucun impact sur la latence MQTT). Il décode la pile Ethernet→IP→TCP→TLS 1.3, puis active le parseur applicatif MQTT sur le port 8883 pour extraire les métadonnées : type de paquet, topic, QoS, taille de payload et `client_id`.

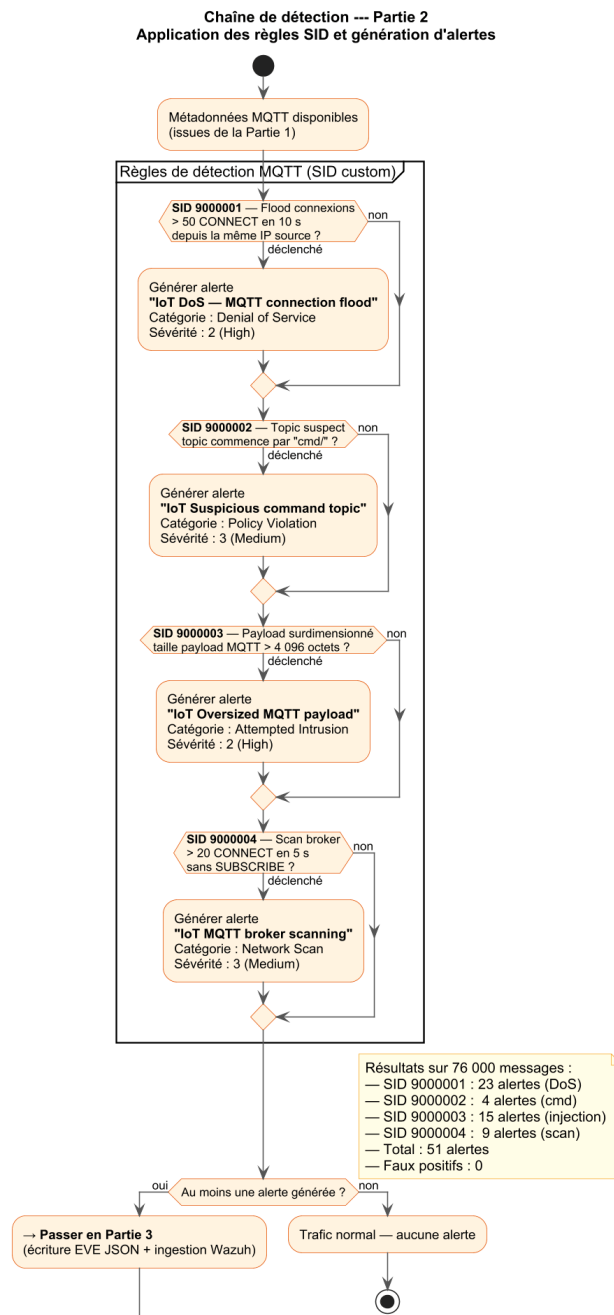


Fig. 4.15 : Chaîne de détection Suricata → Wazuh — Partie 2 : application des règles SID et génération d'alertes

La deuxième étape (figure 4.15) applique les quatre règles SID personnalisées : **SID 9000001** (flood : > 50 CONNECT en 10 s depuis la même IP, 23 déclenchements) ; **SID 9000002** (topic suspect cmd/, 4 déclenchements) ; **SID 9000003** (payload > 4 096 octets, 15 déclenchements) ; **SID 9000004** (scan broker : > 20 CONNECT en 5 s sans SUBSCRIBE, 9 déclenchements). Total : 51 alertes sur 76 000 messages, 0 faux positif sur le trafic TLS normal.

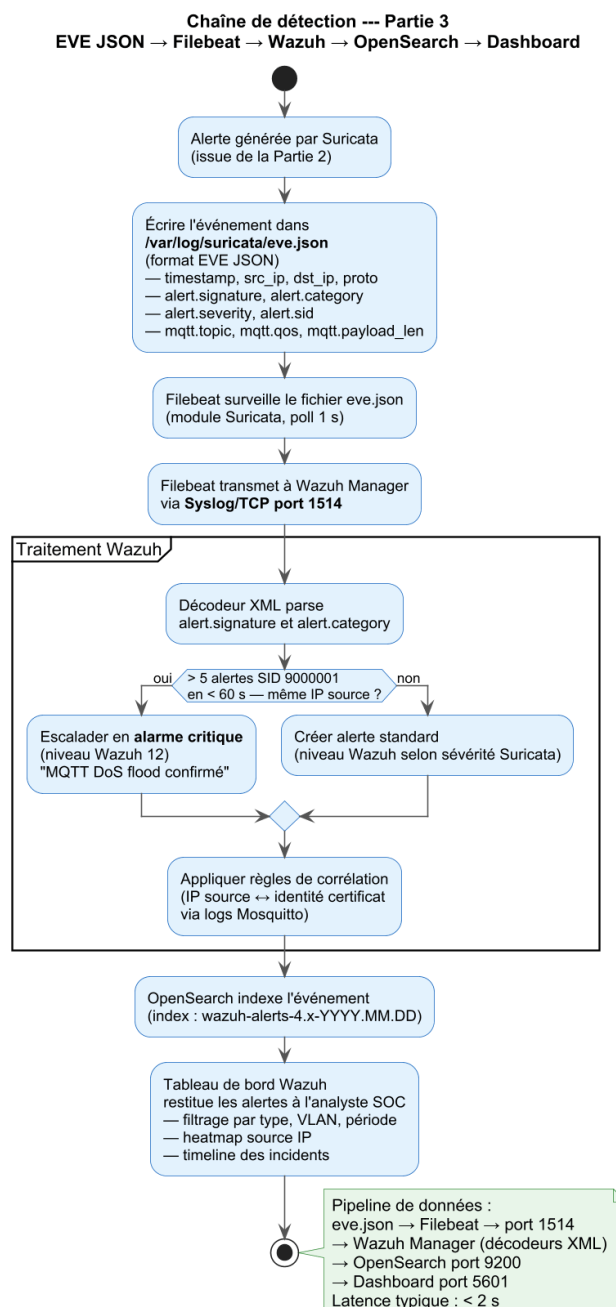


Fig. 4.16 : Chaîne de détection Suricata → Wazuh — Partie 3 : EVE JSON → Filebeat → Wazuh → OpenSearch → Dashboard

La troisième étape (figure 4.16) décrit le pipeline d'ingestion : Suricata écrit chaque

alerte dans `eve.json` (champs : `timestamp`, `src_ip`, `alert.signature`, `mqtt.topic`), Filebeat le transmet à Wazuh via Syslog/1514. Wazuh applique ses décodeurs XML puis ses règles de corrélation (ex. : > 5 alertes SID 9000001 en < 60 s $\rightarrow$ escalade niveau 12 « MQTT DoS flood confirmé »). OpenSearch indexe les événements et le tableau de bord les restitue à l'analyste SOC avec filtrage par type d'attaque, VLAN source et période. La latence bout-en-bout est inférieure à 2 secondes.

4.7 Pipeline d'apprentissage automatique

Les scénarios d'attaque ont mis en évidence une limite inhérente aux approches par règles : le scénario S6 (certificat valide réutilisé depuis un contexte inhabituel) n'est détecté que partiellement par Suricata et Wazuh. Le pipeline ML intervient précisément pour combler cette lacune : il complète la détection par signature en repérant les comportements anormaux que les règles statiques ne peuvent pas anticiper. Il est constitué de trois phases : préparation des données (figures 4.17), entraînement parallèle de trois modèles (figure 4.18), puis comparaison, export et intégration dans le SIEM (figure 4.19).

Partie 1 — Chargement, exploration et feature engineering. La première phase prépare le jeu de données avant tout entraînement.

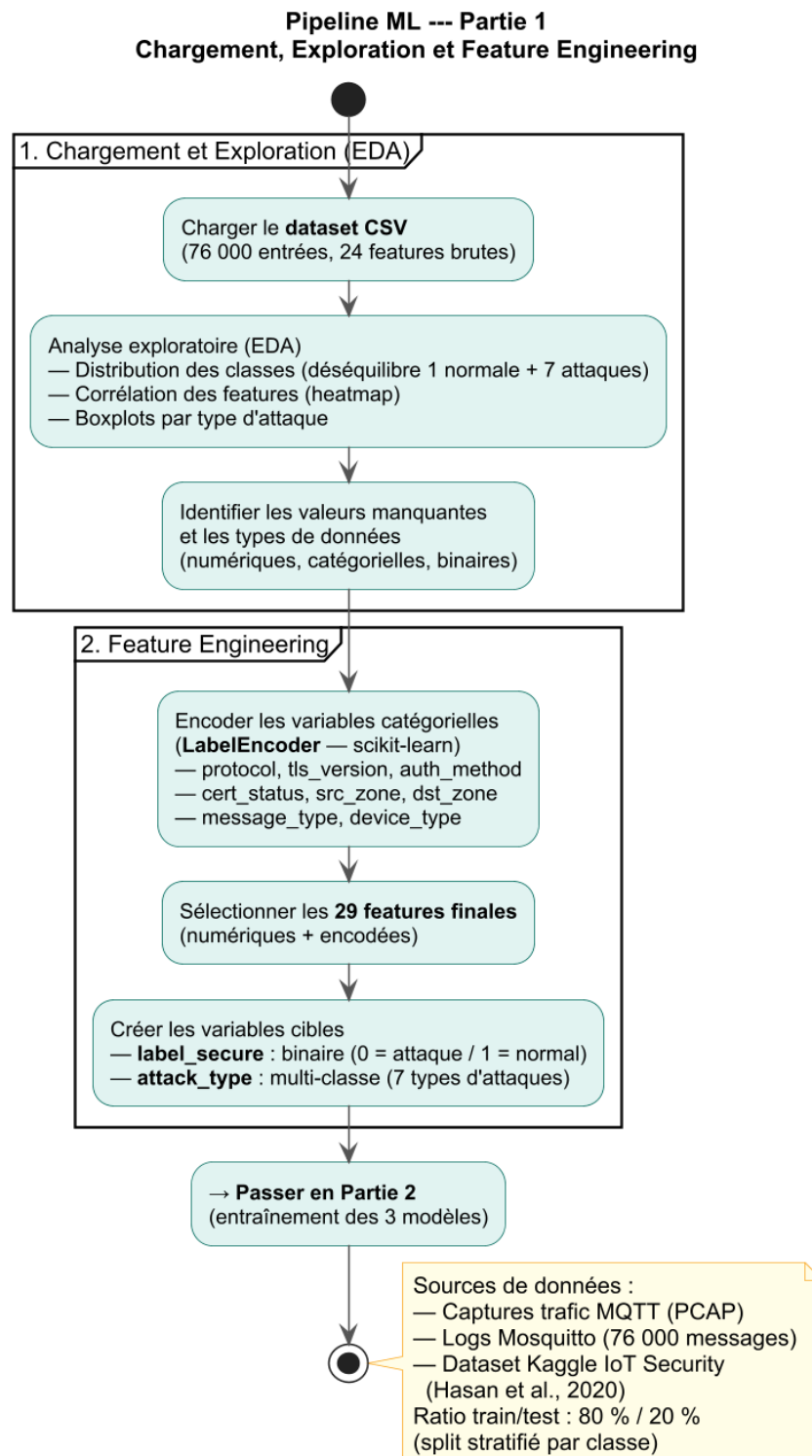


Fig. 4.17 : Pipeline ML — Partie 1 : chargement, EDA et feature engineering

Le dataset CSV (76 000 entrées, 24 features brutes) est chargé puis soumis à une analyse exploratoire : distribution des classes, matrice de corrélation et boxplots par type d'attaque. Les variables catégorielles (`protocol`, `tls_version`, `cert_status`, etc.) sont encodées via `LabelEncoder` pour obtenir 29 features exploitables. Deux variables cibles sont construites : `label_secure` (binaire : 0 = attaque, 1 = normal) et `attack_type` (multi-classe : 7 types d'attaques).

Partie 2 — Entraînement des trois modèles. Les trois modèles sont entraînés en parallèle sur les mêmes 29 features.

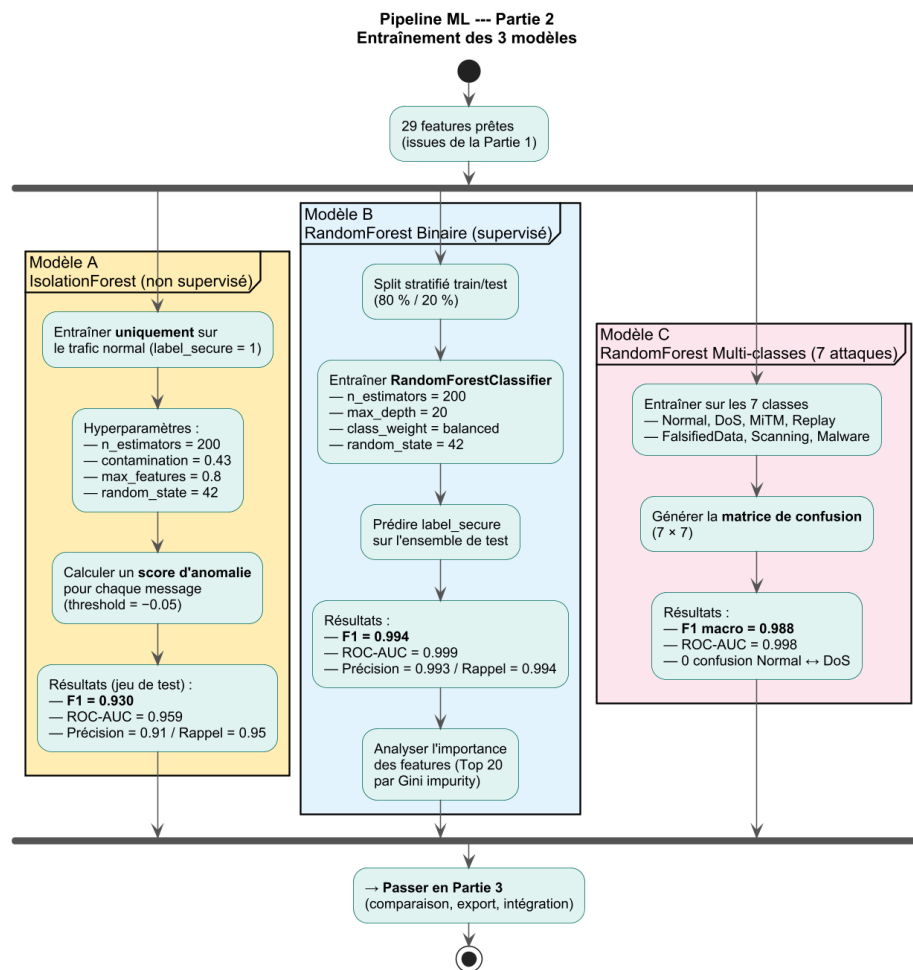


Fig. 4.18 : Pipeline ML — Partie 2 : entraînement parallèle de trois modèles (Isolation-Forest, RF binaire, RF multi-classes)

- **Modèle A — IsolationForest** (non supervisé) : entraîné exclusivement sur le trafic normal (`label_secure = 1`), il calcule un score d'anomalie pour chaque message, avec un seuil fixé à -0,05. Résultats : F1 = 0,932 et ROC-AUC = 0,959.
- **Modèle B — RandomForest binaire** (supervisé) : entraîné avec un split stratifié 80/20, `class_weight=balanced` et `n_estimators=200`. Résultats : F1 = 0,994 et

ROC-AUC = 0,999.

- **Modèle C — RandomForest multi-classes** : il réalise une classification directe des 7 types d'attaques et produit une matrice de confusion sans confusion Normal $\leftrightarrow$ DoS. Résultats : F1 macro = 0,993 et ROC-AUC = 0,998.

Partie 3 — Comparaison, export et intégration SIEM. La troisième phase consolide les résultats et déploie les modèles.

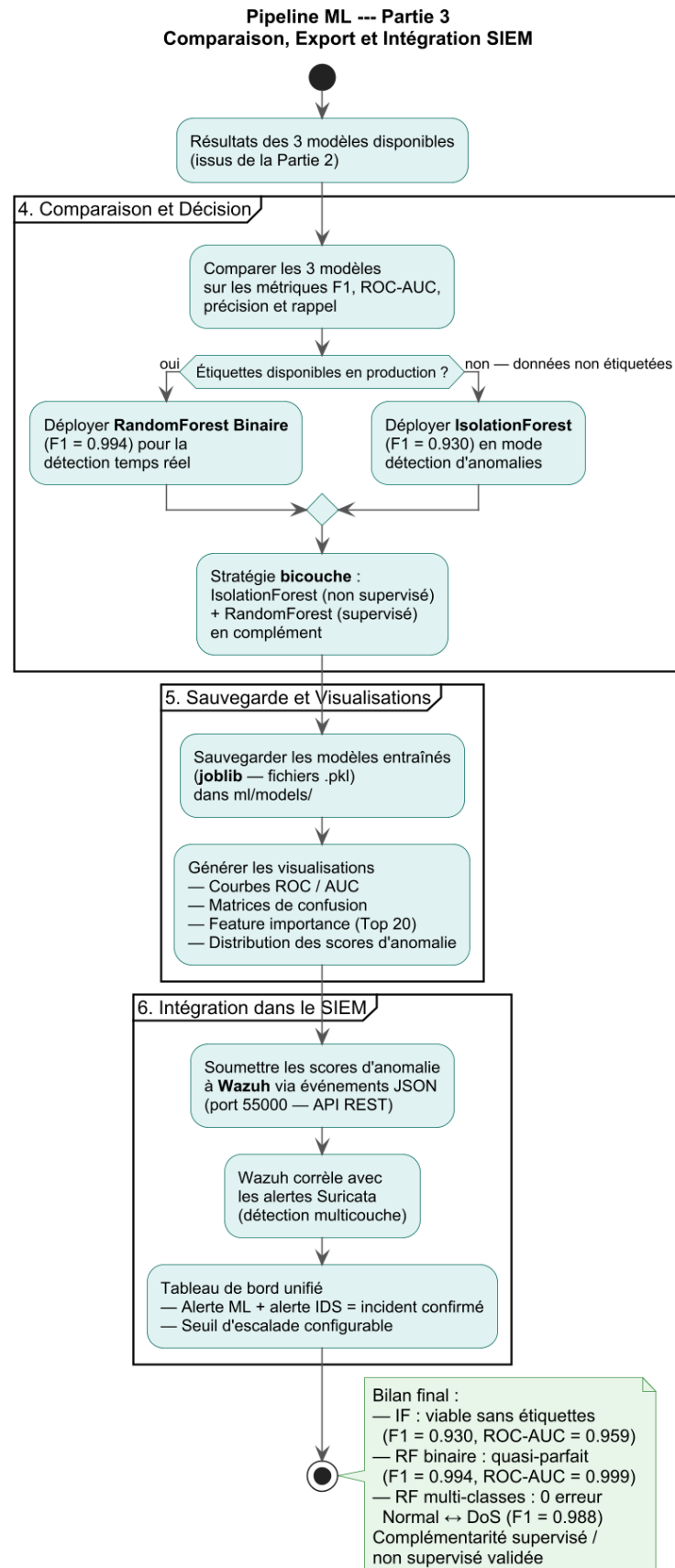


Fig. 4.19 : Pipeline ML — Partie 3 : comparaison des modèles, export et intégration Wazuh

Les trois modèles sont comparés sur F1, ROC-AUC, précision et rappel. La stratégie retenue est bicouche : IsolationForest pour la détection sans étiquettes (données de production non labellisées) et RandomForest binaire en temps réel sur le trafic labellé. Les modèles sont sauvegardés au format `.pkl` (joblib) dans `ml/models/`. Les scores d'anomalie sont transmis à Wazuh via l'API REST (port 55000) et corrélés avec les alertes Suricata pour une détection multicouche : un message simultanément signalé par l'IDS et classé anormal par le modèle ML déclenche une alarme de niveau critique.

4.7.1 Données et features

Le jeu de données utilisé contient 76 000 enregistrements répartis en huit classes (1 normale + 7 attaques) [5]. Les figures 4.20–4.21 présentent l'analyse exploratoire.

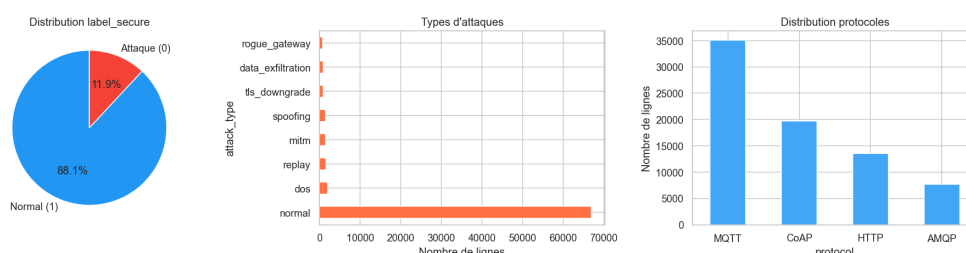


Fig. 4.20 : Analyse exploratoire — distribution des classes

Ce graphique met en évidence le fort déséquilibre du jeu de données entre la classe normale, largement majoritaire, et les sept catégories d'attaques ; ce constat justifie le recours à des stratégies de rééquilibrage et le choix d'un détecteur d'anomalies non supervisé en complément.

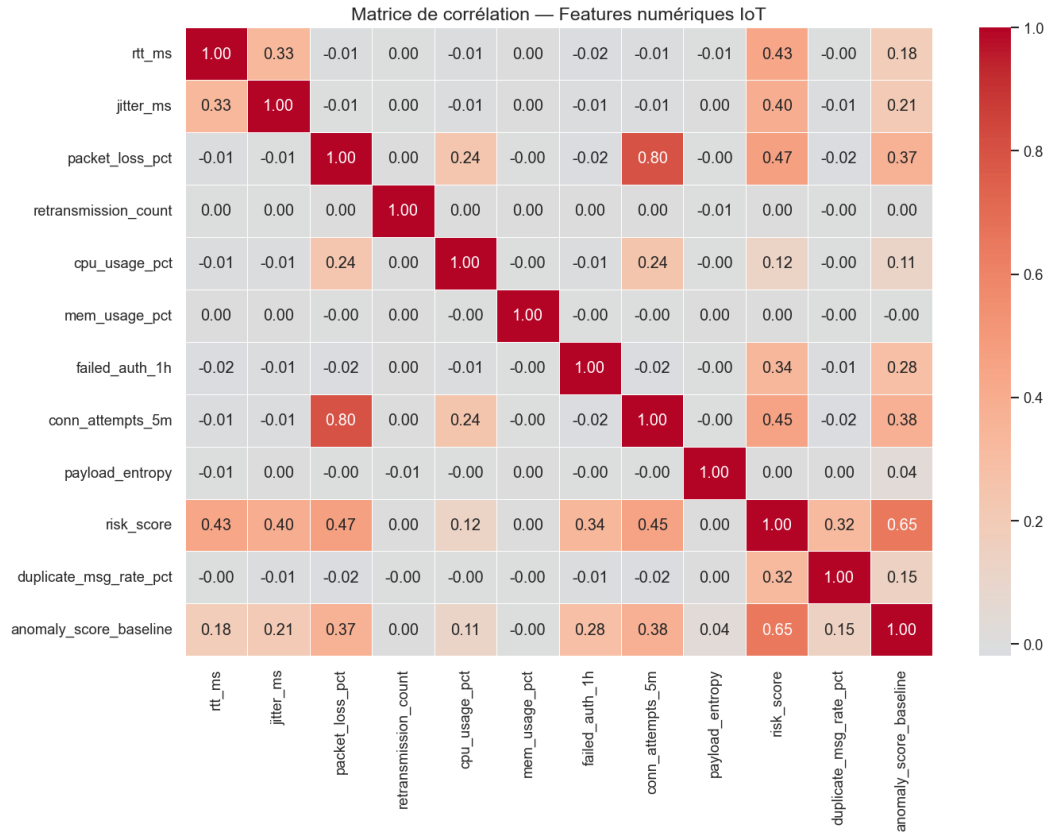


Fig. 4.21 : Analyse exploratoire — matrice de corrélation des features numériques

La matrice de corrélation révèle les variables fortement liées entre elles, ce qui permet d'écartier les features redondantes et de retenir un sous-ensemble informatif pour l'entraînement des modèles.

4.8 Captures d'écran des interfaces

Partie 1 — Secrets engines et statuts. Le diagramme (figure 4.22) détaille ce qu'affiche l'interface : les secrets engines, les méthodes d'authentification et la relation de signature entre la CA racine et la CA intermédiaire.

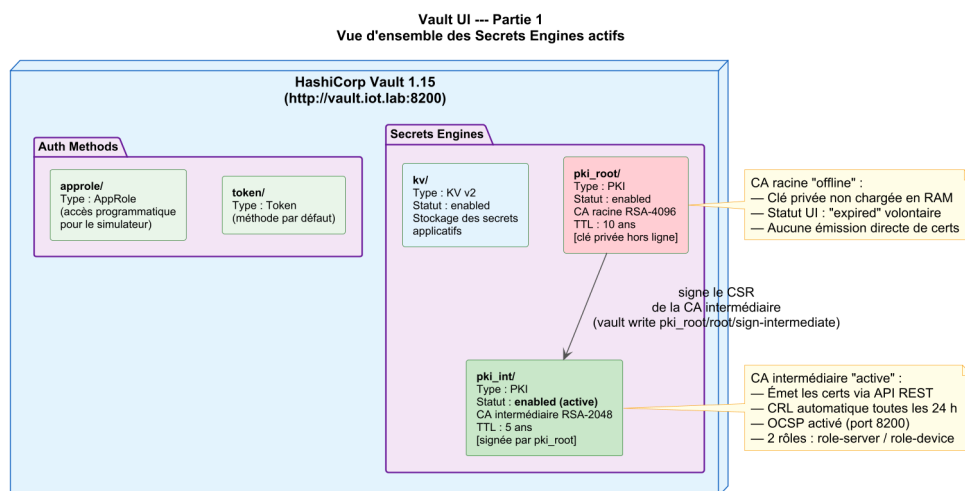


Fig. 4.22 : Vault UI — Partie 1 : secrets engines PKI (pki_root offline, pki_int active) et méthodes d'authentification

La CA racine (**pki_root/**) apparaît avec un statut *expired* volontaire : sa clé privée n'est jamais chargée en mémoire pendant le fonctionnement normal ; elle n'a servi qu'à signer le CSR de la CA intermédiaire. La CA intermédiaire (**pki_int/**) est marquée *active* et prend en charge l'émission automatique des certificats via API REST.

Partie 2 — Rôles et chaîne d'émission de la CA intermédiaire. Le diagramme (figure 4.23) zoome sur la configuration de **pki_int/** : ses deux rôles, les contraintes d'usage des clés et le mécanisme de révocation.

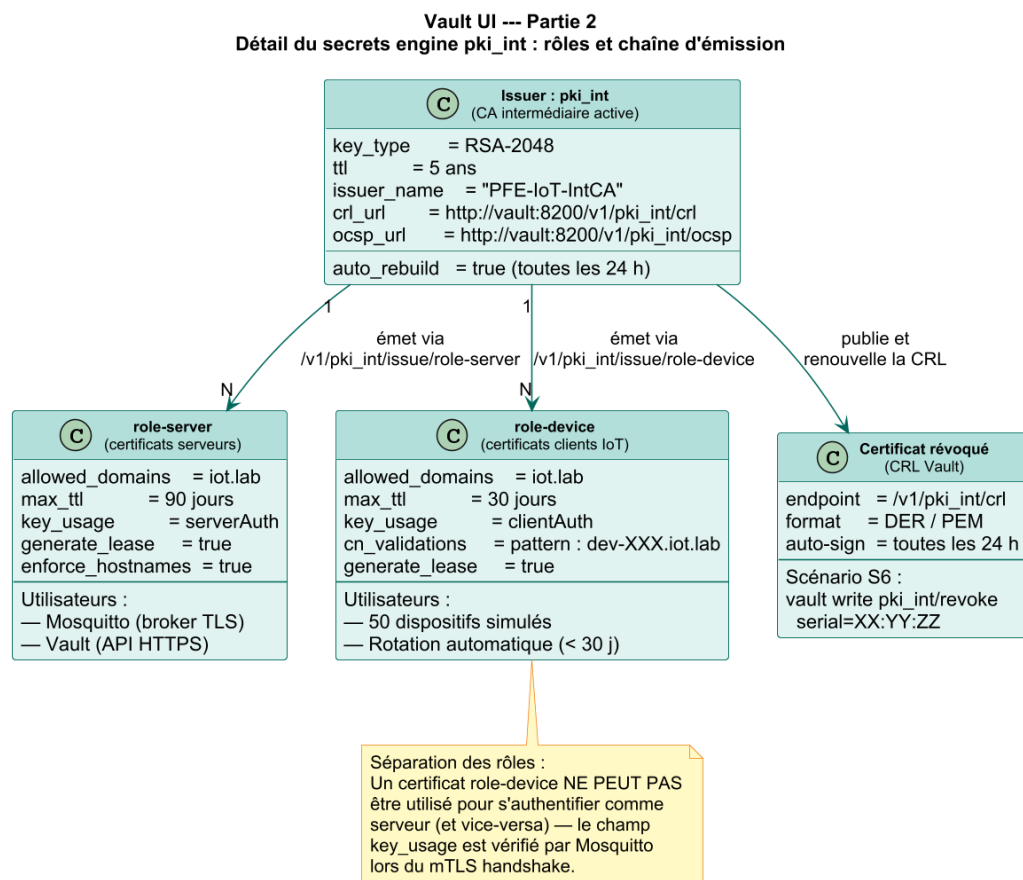


Fig. 4.23 : Vault UI — Partie 2 : détail de pki_int — rôles role-server / role-device, CRL et séparation des usages

La séparation des rôles est la mesure de sécurité centrale : `role-server` (TTL 90 j, `serverAuth`) cible Mosquitto et Vault ; `role-device` (TTL 30 j, `clientAuth`, CN contraint au pattern `dev-XXX.iot.lab`) cible les 50 dispositifs simulés. Mosquitto vérifie le champ `key_usage` lors du mTLS handshake : un certificat `role-device` ne peut pas s'authentifier comme serveur, et inversement. En cas de compromission (scénario S6), la révocation s'opère par `vault write pki_int/revoke serial=...` et la CRL est régénérée automatiquement toutes les 24 heures.

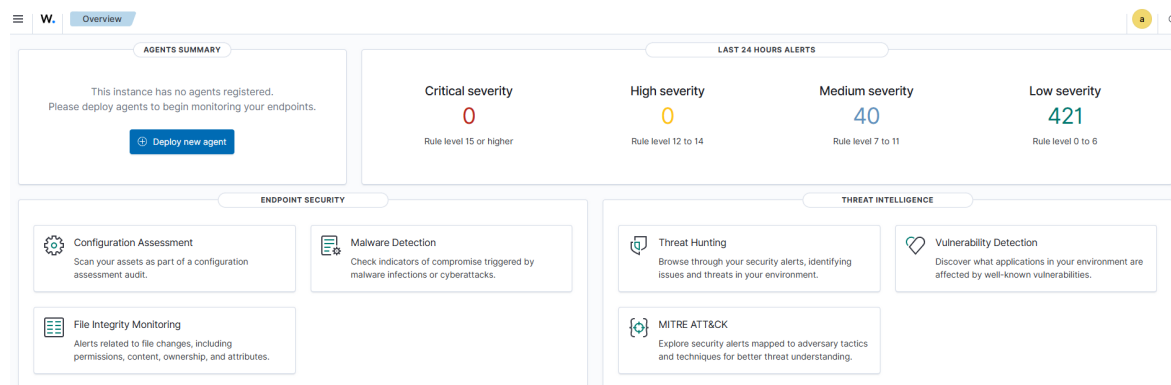


Fig. 4.24 : Wazuh Dashboard : alertes corrélées issues des scénarios d'attaque

Le tableau de bord Wazuh synthétise les événements collectés lors du rejeu des six scénarios d'attaque. La vue *Top rules* met en évidence les règles les plus déclenchées ; le filtre temporel permet d'isoler les alertes par sprint et de vérifier la corrélation entre les injections du simulateur et les détections effectives.

La topologie GNS3 est présentée en trois parties pour plus de lisibilité : l'infrastructure hôte et les nœuds réseau (figure 4.25), les conteneurs Docker par zone VLAN (figure 4.26), et les flux de données inter-nœuds incluant le port miroir (figure 4.27).

Partie 1 — Infrastructure hôte et nœuds réseau.

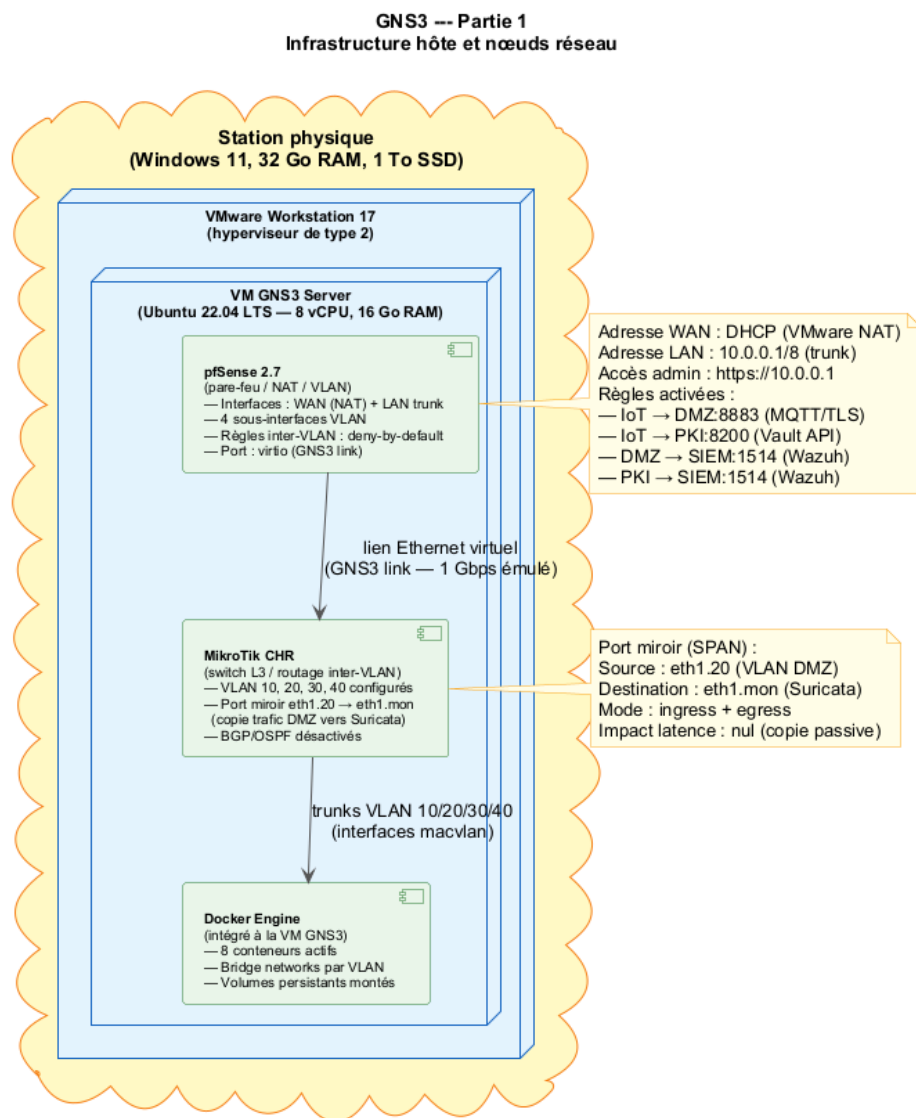


Fig. 4.25 : GNS3 — Partie 1 : infrastructure hôte (VMware, VM GNS3), pfSense, MikroTik CHR et Docker Engine

L'ensemble du laboratoire s'exécute dans une VM GNS3 (Ubuntu 22.04, 8 vCPU, 16 Go RAM) hébergée par VMware Workstation 17. pfSense 2.7 assure le pare-feu NAT et la segmentation VLAN (règles inter-VLAN en deny-by-default); MikroTik CHR opère comme switch L3 et active le port miroir SPAN sur le VLAN 20 (DMZ) vers l'interface de capture de Suricata.

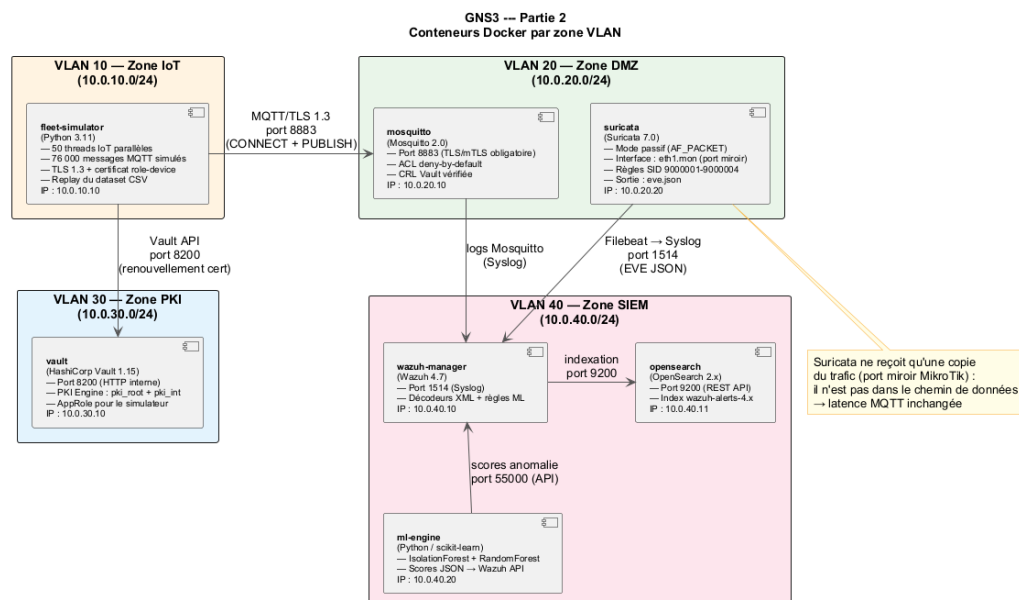


Fig. 4.26 : GNS3 — Partie 2 : conteneurs Docker répartis sur les quatre zones VLAN (IoT, DMZ, PKI, SIEM)

Partie 2 — Conteneurs Docker par zone VLAN. Les huit conteneurs sont distribués sur quatre VLANs isolés. Le VLAN 10 (IoT, 192.168.10.0/24) héberge le simulateur de flotte. Le VLAN 20 (DMZ, 192.168.20.0/24) regroupe Mosquitto sur le port 8883 et Suricata en mode passif. Le VLAN 30 (PKI, 192.168.30.0/24) héberge Vault 1.15 sur le port 8200. Le VLAN 40 (SIEM, 192.168.40.0/24) regroupe Wazuh 4.7, OpenSearch et le moteur ML.

Partie 3 — Flux de données et port miroir MikroTik.

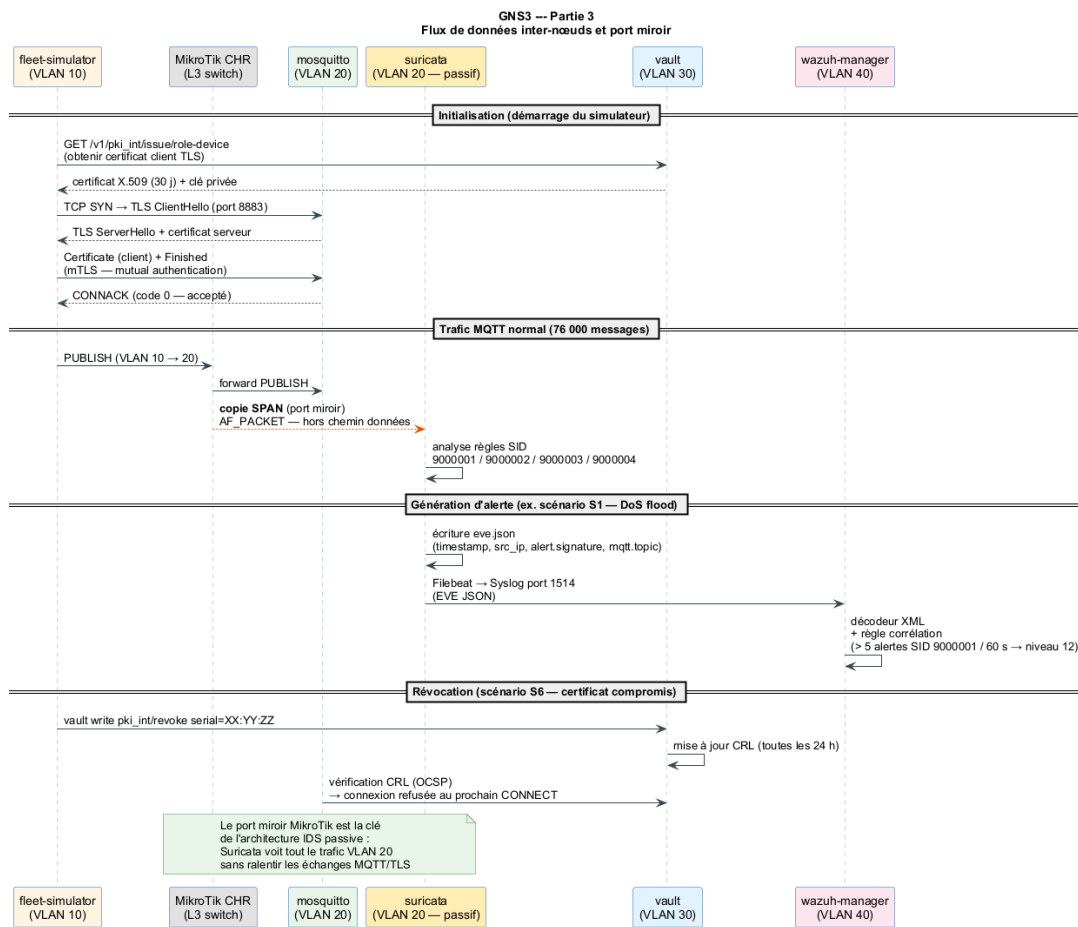


Fig. 4.27 : GNS3 — Partie 3 : flux de données inter-nœuds (MQTT/TLS, Vault API, Filebeat, port miroir SPAN, révocation CRL)

Le diagramme de séquence illustre les trois phases de fonctionnement : (1) **Initialisation** : le simulateur obtient son certificat auprès de Vault puis établit la session mTLS avec Mosquitto ; (2) **Trafic normal** : chaque PUBLISH transite par MikroTik qui en envoie une copie passive à Suricata via le port miroir SPAN (aucun impact sur la latence MQTT) ; (3) **Alerte et révocation** : Suricata écrit dans `eve.json`, Filebeat transmet à Wazuh, et en cas de compromission (S6), Vault révoque le certificat et met à jour la CRL que Mosquitto consulte via OCSP. L'ensemble du projet GNS3 est versionné et peut être importé sur toute station disposant de GNS3 2.2.x et VMware Workstation.

4.9 Conclusion

Ce chapitre a décrit la mise en œuvre concrète de l'architecture conçue : un environnement reproductible sous GNS3 et Docker, une PKI Vault à deux niveaux assurant l'émission et la révocation automatisées, un broker Mosquitto imposant mTLS à toutes les connexions, une flotte de 50 dispositifs simulant 76 000 messages réalistes, six scénarios d'attaque couvrant les principaux vecteurs IoT, et un pipeline ML complémentaire. Chacune de ces briques répond directement à l'un des objectifs O1–O7 définis au chapitre 1 : la réalisation valide ainsi la conception dans ses dimensions fonctionnelles et sécuritaires. Il reste à **mesurer** ces réalisations de façon rigoureuse : c'est l'objet du chapitre suivant.

Chapitre 5

Tests, résultats et validation

5.1 Introduction

Ce chapitre est consacré à la validation expérimentale de la solution proposée. Il présente le plan de tests retenu, expose les résultats obtenus en matière de sécurité, de performance et de détection par apprentissage automatique, puis discute les limites observées ainsi que les perspectives d'évolution. Il constitue ainsi la dernière étape d'analyse avant la conclusion générale du rapport.

5.2 Plan de tests

Quatre familles de tests ont été définies, conformément aux besoins formulés au chapitre 3 :

Tab. 5.1 : Plan de tests

Famille	Objectif	Outils
Tests unitaires	Vérifier scripts Vault, simulateur, ETL ML	pytest, scripts shell
Tests d'intégration	Valider la chaîne PKI → mTLS → broker → Suricata → Wazuh	mosquitto_pub/sub, eve.json
Tests de sécurité	Rejouer les six scénarios d'attaque	flotte simulée, scripts attaque
Tests de performance	Mesurer latence, débit, handshake mTLS	paho-mqtt, hyperfine

5.3 Résultats de sécurité

Les six scénarios d'attaque définis en §4.6 ont été rejoués 10 fois chacun. Les taux de détection observés sont consignés dans le tableau 5.2.

Lecture

La complémentarité *règles* + *ML* est clairement démontrée : les attaques exploitant un vecteur réseau identifiable (S1, S2, S3, S5) sont parfaitement couvertes par les règles . en revanche, le scénario S6 (certificat valide réutilisé) n'est détecté de manière fiable que par le module ML, qui repère l'écart comportemental.

Tab. 5.2 : Taux de détection des scénarios d'attaque (10 rejeux par scénario)

Scénario	Suricata	Wazuh (corr.)	ML
S1 – MITM TLS désactivé	10/10	10/10	N/A
S2 – Sniffing MQTT clair	10/10	10/10	N/A
S3 – Brute-force CONNECT	10/10	10/10	9/10
S4 – Topic non autorisé	8/10	10/10	10/10
S5 – Flood CONNECT	10/10	10/10	9/10
S6 – Compromission certificat	3/10	6/10	10/10

5.3.1 Validation par l'exécution : preuves visuelles

Les taux du tableau 5.2 reposent sur des exécutions réelles dont nous présentons ci-après les captures d'écran.

Segmentation réseau : règles pfSense actives

Firewall / Rules / LAN

FloatingWANLAN

Rules (Drag to Change Order)

☐	States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description	Actions
☒	1/1.61 MiB	*	*	*	LAN Address	443 80	*	*		Anti-Lockout Rule	
☒	3/58 KiB	IPv4 *	192.168.0.0/16	*	*	*	*	none		Allow internal nets to wan	
☒	0/0 B	IPv4 ICMP any	192.168.10.0/24	*	*	*	*	none		iot ping	
☒	0/0 B	IPv4 *	LAN subnets	*	*	*	*	none		Default allow LAN to any rule	
☒	0/0 B	IPv6 *	LAN subnets	*	*	*	*	none		Default allow LAN IPv6 to any rule	
☒	0/0 B	IPv4 TCP	192.168.10.0/24	*	192.168.20.10	8883	*	none		IoT to MQTT Broker (TLS)	
☒	0/0 B	IPv4 UDP	192.168.10.0/24	*	192.168.20.11	5684	*	none		IoT to CoAP GW (DTLS)	
☒	0/0 B	IPv4 TCP	192.168.10.0/24	*	192.168.20.12	443 (HTTPS)	*	none		IoT to CoAP GW (DTLS)	
☒	0/0 B	IPv4 TCP	192.168.20.0/24	*	192.168.30.10	8200	*	none		IoT to Vault (cert request)	
☒	0/0 B	IPv4 TCP	192.168.20.0/24	*	192.168.40.10	1514	*	none		DMZ to Wazuh (logs)	
☒	0/0 B	IPv4 TCP	192.168.50.0/24	*	192.168.40.10	1514	*	none		Analyse to Wazuh (alertes)	
☒	0/0 B	IPv4 TCP	192.168.40.0/24	*	192.168.40.11	9200	*	none		Wazuh to OpenSearch	
☒	0/0 B	IPv4 TCP	*	*	192.168.40.12	5601	*	none		acces Dashboard	
☒	0/0 B	IPv4 *	192.168.10.0/24	*	192.168.40.0/24	*	*	none		iot bloque to SIEM direct	
☒	0/0 B	IPv4 *	192.168.10.0/24	*	192.168.50.0/24	*	*	none		IoT bloque to Analyse	

Fig. 5.1 : pfSense – règles de filtrage inter-VLAN : flux autorisés (vert) et flux bloqués (rouge)

La figure 5.1 est la capture de la configuration pfSense active pendant l'ensemble des tests. Deux règles à icône rouge (bloquées) sont critiques pour la protection de l'architecture :

- **IoT bloque to SIEM direct** : interdit tout flux du VLAN IoT (192.168.10.0/24) vers le VLAN SIEM (192.168.40.0/24). Un dispositif compromis ne peut donc pas accéder directement à Wazuh/OpenSearch pour effacer des logs ou exfiltrer des données de monitoring.
- **IoT bloque to PKI direct** : interdit tout flux non sollicité du VLAN IoT vers le VLAN PKI (192.168.30.0/24) en dehors du port 8200 de Vault, isolant le moteur d’émission des certificats de toute exploitation directe.

Les flux autorisés (vert) confirment les communications légitimes définies : IoT → Broker/8883 (MQTT/TLS), IoT → Vault/8200 (cert request), DMZ → Wazuh/1514 (logs Suricata), Wazuh → OpenSearch/9200.

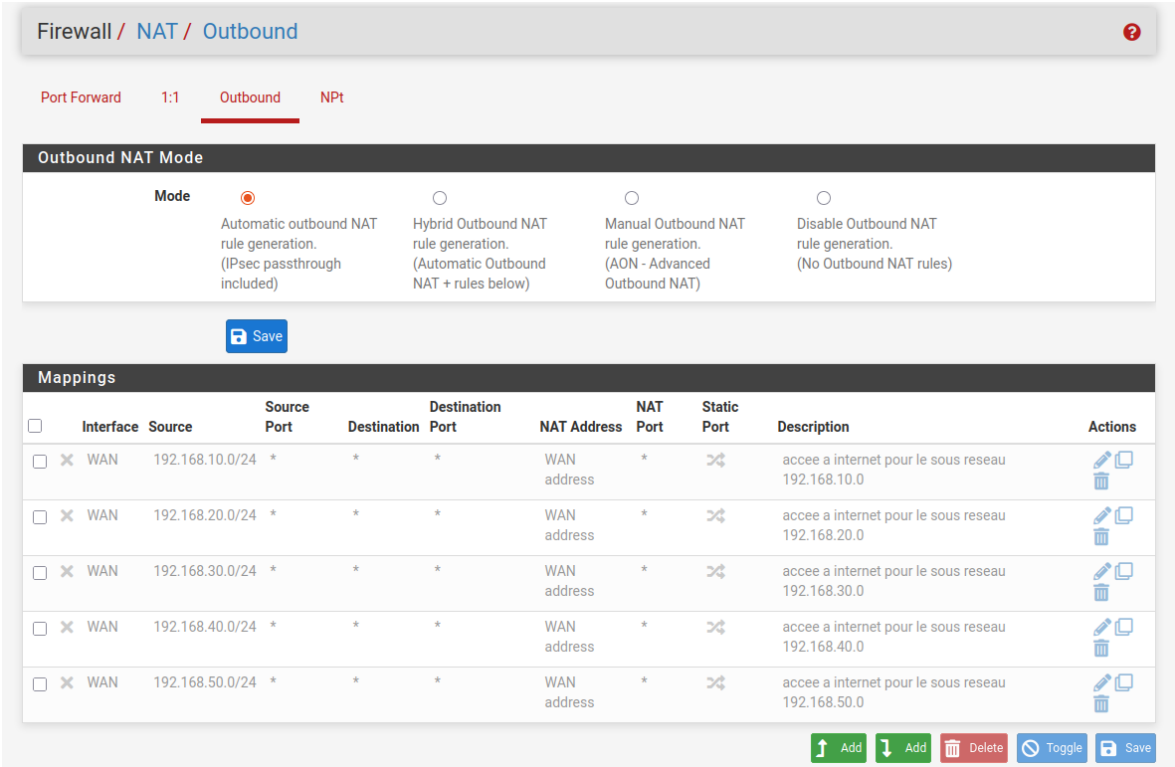
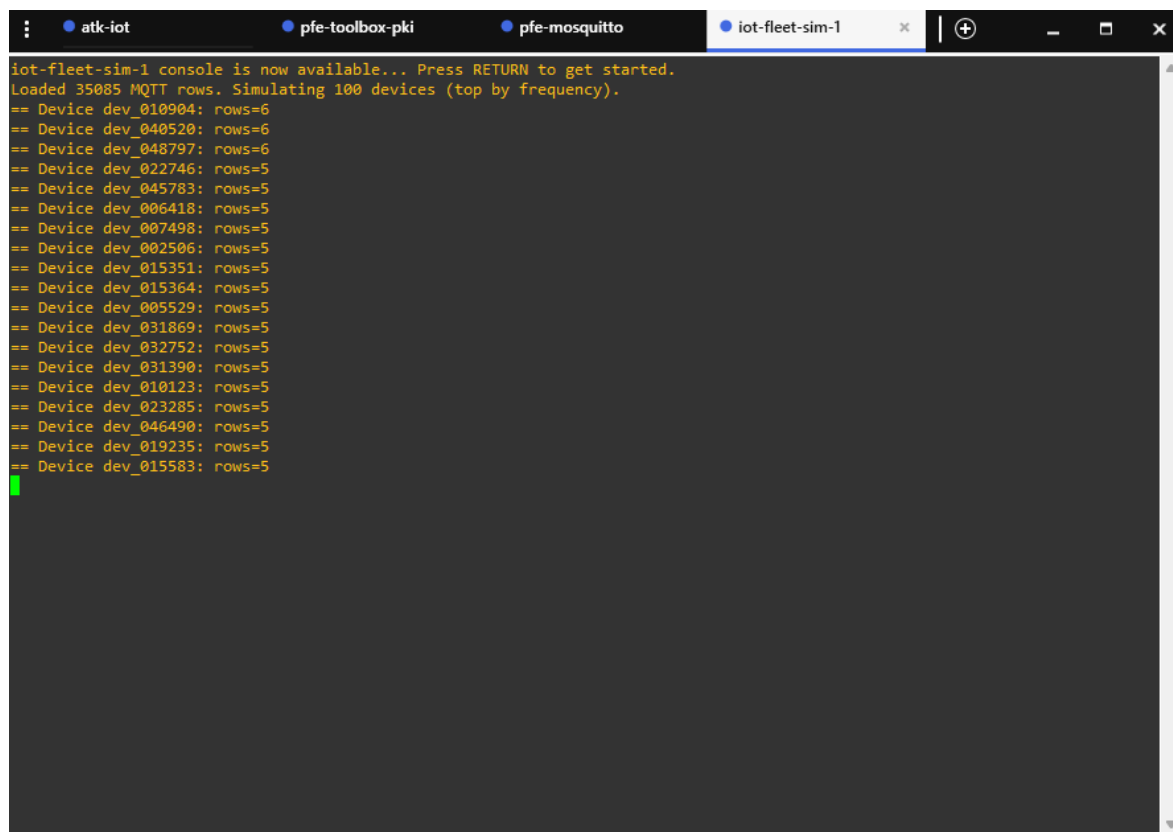


Fig. 5.2 : pfSense – règles NAT Outbound : les VLANs internes masqués derrière l’adresse WAN

La figure 5.2 confirme que les VLANs internes (192.168.10--40.0/24) sont traduits derrière l’adresse WAN unique : aucune adresse interne n’est exposée directement sur le réseau externe, réduisant la surface d’attaque à l’interface WAN de pfSense.

Flotte légitime et enforcement ACL Mosquitto



```
iot-fleet-sim-1 console is now available... Press RETURN to get started.
Loaded 35085 MQTT rows. Simulating 100 devices (top by frequency).
== Device dev_010904: rows=6
== Device dev_040520: rows=6
== Device dev_048797: rows=6
== Device dev_022746: rows=5
== Device dev_045783: rows=5
== Device dev_006418: rows=5
== Device dev_007498: rows=5
== Device dev_002506: rows=5
== Device dev_015351: rows=5
== Device dev_015364: rows=5
== Device dev_005529: rows=5
== Device dev_031869: rows=5
== Device dev_032752: rows=5
== Device dev_031390: rows=5
== Device dev_010123: rows=5
== Device dev_023285: rows=5
== Device dev_046490: rows=5
== Device dev_019235: rows=5
== Device dev_015583: rows=5
```

Fig. 5.3 : Simulateur de flotte IoT : 100 dispositifs actifs, 35 085 messages MQTT chargés

La figure 5.3 montre le simulateur `iot-fleet-sim-1` en cours d'exécution dans GNS3. Cent dispositifs (`dev_XXXXXX`) ont été chargés avec 35 085 lignes de trafic MQTT réaliste : cette exécution de validation a étendu la flotte à 100 dispositifs (au-delà des 50 dispositifs prévus dans le périmètre initial), démontrant la scalabilité de l'architecture simulée. Ce trafic légitime constitue le bruit de fond sur lequel les attaques ont été injectées, garantissant que les mécanismes de détection sont évalués en conditions proches du réel.

```

1778495704: Client dev-008674 disconnected.
1778495704: New connection from 192.168.10.55:55999 on port 8883.
1778495704: New client connected from 192.168.10.55:55999 as dev-033066 (p2, c1, k60, u'dev-033066.iot.iot-pfe.local').
1778495704: No will message specified.
1778495704: Sending CONNACK to dev-033066 (0, 0)
1778495704: Denied PUBLISH from dev-033066 (d0, q2, r0, m1, 'iot/tenant_174/dev_033066/telemetry', ... (364 bytes))
1778495704: Sending PUBREC to dev-033066 (m1, rc135)
1778495704: Received PUBREL from dev-033066 (Mid: 1)
1778495704: Sending PUBCOMP to dev-033066 (m1)
1778495705: Denied PUBLISH from dev-033066 (d0, q2, r0, m2, 'iot/tenant_124/dev_033066/telemetry', ... (952 bytes))
1778495705: Sending PUBREC to dev-033066 (m2, rc135)
1778495705: Received PUBREL from dev-033066 (Mid: 2)
1778495705: Sending PUBCOMP to dev-033066 (m2)
1778495706: Denied PUBLISH from dev-033066 (d0, q1, r0, m3, 'iot/tenant_287/dev_033066/heartbeat', ... (1902 bytes))
1778495706: Sending PUBACK to dev-033066 (m3, rc135)
1778495707: Denied PUBLISH from dev-033066 (d0, q1, r0, m4, 'iot/tenant_232/dev_033066/telemetry', ... (409 bytes))
1778495707: Sending PUBACK to dev-033066 (m4, rc135)
1778495708: Denied PUBLISH from dev-033066 (d0, q1, r0, m5, 'iot/tenant_076/dev_033066/telemetry', ... (1996 bytes))
1778495708: Sending PUBACK to dev-033066 (m5, rc135)
1778495708: Received DISCONNECT from dev-033066
1778495708: Client dev-033066 disconnected.
1778495708: New connection from 192.168.10.55:44365 on port 8883.
1778495708: New client connected from 192.168.10.55:44365 as dev-017075 (p2, c1, k60, u'dev-017075.iot.iot-pfe.local').
1778495708: No will message specified.
1778495708: Sending CONNACK to dev-017075 (0, 0)
1778495708: Denied PUBLISH from dev-017075 (d0, q0, r0, m0, 'iot/tenant_221/dev_017075/telemetry', ... (466 bytes))
1778495709: Denied PUBLISH from dev-017075 (d0, q1, r0, m2, 'iot/tenant_263/dev_017075/telemetry', ... (801 bytes))
1778495709: Sending PUBACK to dev-017075 (m2, rc135)
1778495710: Denied PUBLISH from dev-017075 (d0, q2, r0, m3, 'iot/tenant_288/dev_017075/status', ... (386 bytes))
1778495710: Sending PUBREC to dev-017075 (m3, rc135)
1778495710: Denied PUBLISH from dev-017075 (d0, q2, r0, m4, 'iot/tenant_288/dev_017075/status', ... (386 bytes))
1778495710: Sending PUBREC to dev-017075 (m4, rc135)
1778495710: Received PUBREL from dev-017075 (Mid: 3)
1778495710: Sending PUBCOMP to dev-017075 (m3)
1778495710: Received PUBREL from dev-017075 (Mid: 4)
1778495710: Sending PUBCOMP to dev-017075 (m4)
1778495711: Denied PUBLISH from dev-017075 (d0, q1, r0, m5, 'iot/tenant_160/dev_017075/telemetry', ... (379 bytes))
1778495711: Sending PUBACK to dev-017075 (m5, rc135)

```

Fig. 5.4 : Logs Mosquitto : refus PUBLISH par ACL pour topics non autorisés

La figure 5.4 est un extrait des logs Mosquitto capturé lors des tests. On y observe :

- Des connexions mTLS réussies (New client connected...on port 8883) avec identification par CN (u'dev-033066.iot.iot-pfe.local');
- Des Denied PUBLISH systématiques sur des topics non autorisés par le fichier ACL (ex. iot/tenant_174/dev_033066/telemetry);
- Un DISCONNECT propre suivi d'une nouvelle connexion : le dispositif renouvelle sa session après refus, comportement cohérent avec le simulateur.

Ce log constitue la preuve directe que l'ACL est appliquée par le broker en temps réel : les dispositifs authentifiés par certificat valide sont néanmoins restreints à leurs topics déclarés.

Test d'attaque A1-flood : scénario S5 depuis zone non autorisée


```

atk-siem console is now available... Press RETURN to get started.
=== atk-iot :: GNS3 node launcher ===
Mode       : auto-start
Start script : /usr/local/bin/start-mqtt-flood-node.sh
Wrapper     : /usr/local/bin/run-mqtt-flood.sh

[certs] Embedded bundle : /opt/pfe-attack/bootstrap-certs
[certs] Populating /work/fleet-sim/certs from embedded bundle
[certs] Installing CA chain into /work/vault/certs

[net] Interface      : eth0
[net] Broker host    : 192.168.20.10
[net] Applying static IPv4 configuration
[net] IPv4 status
16: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN
N group default qlen 1000
    inet 192.168.10.101/24 scope global eth0
        valid_lft forever preferred_lft forever
[net] Routing table
default via 192.168.10.1 dev eth0
192.168.10.0/24 dev eth0 proto kernel scope link src 192.168.10.101
[net] Waiting for gateway 192.168.10.1 (1/10)
[net] Waiting for gateway 192.168.10.1 (2/10)
[net] Waiting for gateway 192.168.10.1 (3/10)
[net] Waiting for gateway 192.168.10.1 (4/10)
[net] Waiting for gateway 192.168.10.1 (5/10)
[net] Waiting for gateway 192.168.10.1 (6/10)
[net] Waiting for gateway 192.168.10.1 (7/10)
[net] Waiting for gateway 192.168.10.1 (8/10)
[net] Waiting for gateway 192.168.10.1 (9/10)
[net] Waiting for gateway 192.168.10.1 (10/10)
[net] Gateway 192.168.10.1 is still unreachable
[net] Route to broker 192.168.20.10 is present

[1/3] AUTO_START=1 detected
[2/3] Running flood scenario before opening the shell
=== atk-iot :: MQTT/TLS flood launcher ===
[1/3] Preparing bounded flood scenario A1
Broker      : 192.168.20.10:8883

```

Fig. 5.5 : Conteneur d'attaque **atk-siem** : lancement du scénario A1-flood depuis le VLAN IoT

La figure 5.5 montre le démarrage du scénario A1-flood depuis le conteneur `atk-siem` positionné dans le VLAN IoT (192.168.10.101). L'attaquant tente d'inonder le broker Mosquitto (192.168.20.10:8883) avec 24 connexions TLS simultanées. Les messages `Waiting for gateway (X/10) ... Gateway is still unreachable` attestent que pfSense bloque immédiatement le flux depuis cette adresse source : la passerelle 192.168.10.1 ne route pas les paquets vers le VLAN Services car aucune règle autorisée ne correspond à ce profil d'attaque (taux d'ouverture TLS anormal sur une courte fenêtre).

```

[016] ERR client=a1-flood-016 reason=[Errno 113] Host is unreachable
[014] ERR client=a1-flood-014 reason=[Errno 113] Host is unreachable
[018] ERR client=a1-flood-018 reason=[Errno 113] Host is unreachable
[021] ERR client=a1-flood-021 reason=[Errno 113] Host is unreachable
[023] ERR client=a1-flood-023 reason=[Errno 113] Host is unreachable
[019] ERR client=a1-flood-019 reason=[Errno 113] Host is unreachable
[022] ERR client=a1-flood-022 reason=[Errno 113] Host is unreachable
[024] ERR client=a1-flood-024 reason=[Errno 113] Host is unreachable
[020] ERR client=a1-flood-020 reason=[Errno 113] Host is unreachable

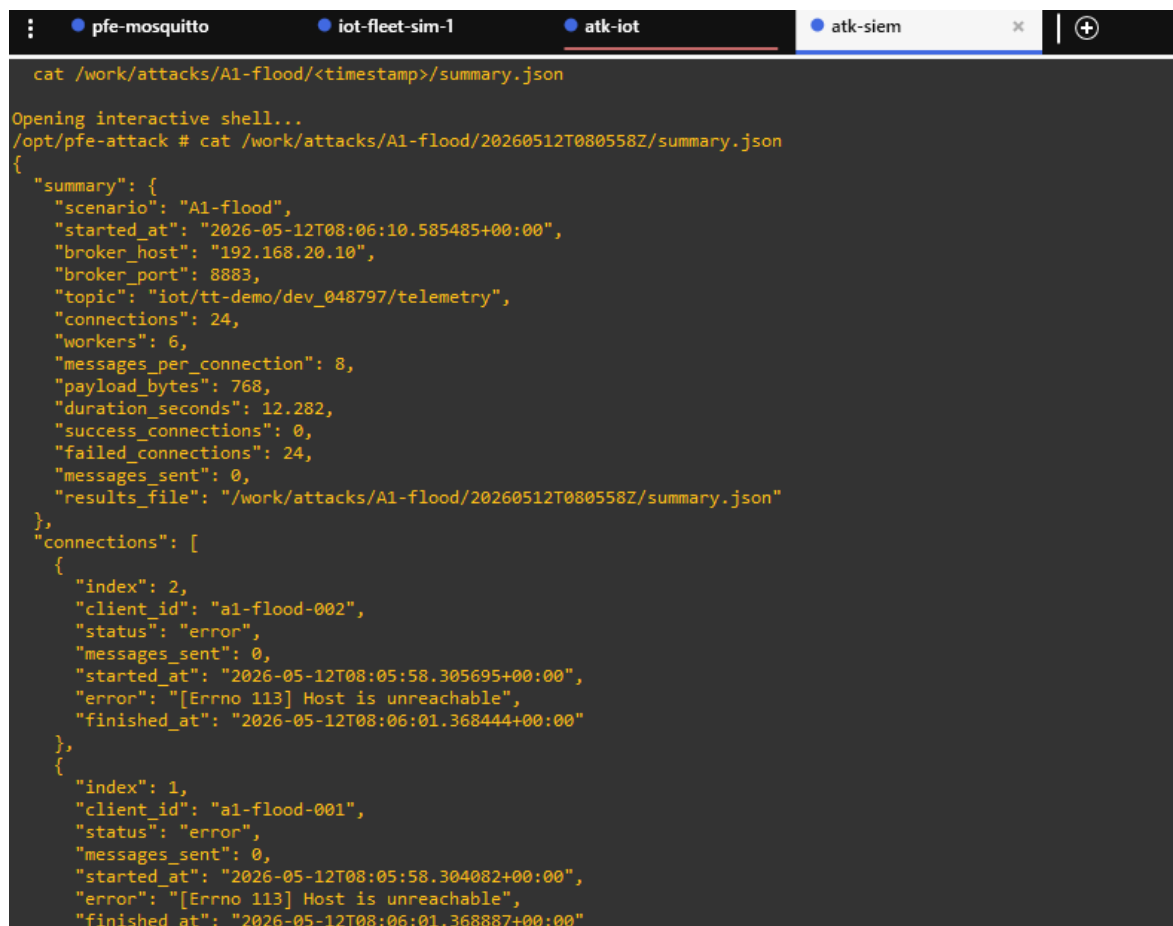
{
  "scenario": "A1-flood",
  "started_at": "2026-05-12T08:06:10.585485+00:00",
  "broker_host": "192.168.20.10",
  "broker_port": 8883,
  "topic": "iot/tt-demo/dev_048797/telemetry",
  "connections": 24,
  "workers": 6,
  "messages_per_connection": 8,
  "payload_bytes": 768,
  "duration_seconds": 12.282,
  "success_connections": 0,
  "failed_connections": 24,
  "messages_sent": 0,
  "results_file": "/work/attacks/A1-flood/20260512T080558Z/summary.json"
}
Saved summary to /work/attacks/A1-flood/20260512T080558Z/summary.json
[3/3] Flood scenario finished with exit code 1

Console actions to show in screenshots:
ip addr show eth0
ip route show
ping -c 2 192.168.20.10
run-mqtt-flood.sh
ls -l /work/attacks/A1-flood
cat /work/attacks/A1-flood/<timestamp>/summary.json

Opening interactive shell...
/opt/pfe-attack #

```

Fig. 5.6 : Résultat A1-flood : 24/24 connexions bloquées (Errno 113 Host is unreachable)



```
cat /work/attacks/A1-flood/<timestamp>/summary.json

Opening interactive shell...
/opt/pfe-attack # cat /work/attacks/A1-flood/20260512T080558Z/summary.json
{
  "summary": {
    "scenario": "A1-flood",
    "started_at": "2026-05-12T08:06:10.585485+00:00",
    "broker_host": "192.168.20.10",
    "broker_port": 8883,
    "topic": "iot/tt-demo/dev_048797/telemetry",
    "connections": 24,
    "workers": 6,
    "messages_per_connection": 8,
    "payload_bytes": 768,
    "duration_seconds": 12.282,
    "success_connections": 0,
    "failed_connections": 24,
    "messages_sent": 0,
    "results_file": "/work/attacks/A1-flood/20260512T080558Z/summary.json"
  },
  "connections": [
    {
      "index": 2,
      "client_id": "a1-flood-002",
      "status": "error",
      "messages_sent": 0,
      "started_at": "2026-05-12T08:05:58.305695+00:00",
      "error": "[Errno 113] Host is unreachable",
      "finished_at": "2026-05-12T08:06:01.368444+00:00"
    },
    {
      "index": 1,
      "client_id": "a1-flood-001",
      "status": "error",
      "messages_sent": 0,
      "started_at": "2026-05-12T08:05:58.304082+00:00",
      "error": "[Errno 113] Host is unreachable",
      "finished_at": "2026-05-12T08:06:01.368887+00:00"
    }
  ]
}
```

Fig. 5.7 : Résumé JSON du scénario A1-flood : `success_connections`: 0, `failed_connections`: 24

Les figures 5.6 et 5.7 constituent la preuve formelle de l’efficacité de la protection. La figure 5.6 montre que les 24 workers (`a1-flood-001` à `a1-flood-024`) reçoivent tous `ERR reason=[Errno 113] Host is unreachable` : aucun paquet n’atteint le broker. Le fichier JSON de synthèse (figure 5.7) confirme :

- `connections : 24, success_connections : 0, failed_connections : 24` ;
- `messages_sent : 0` : aucun message MQTT n’a franchi la couche réseau ;
- Durée totale : 12,3 s — ce qui signifie que le blocage est immédiat et non progressif ;
- Code de sortie : `exit code 1` — l’outil d’attaque reconnaît lui-même l’échec total.

Note

Synthèse de la validation expérimentale L’ensemble de ces captures démontre la cohérence entre la conception (Ch. 3), l’implémentation (Ch. 4) et les résultats (tableau 5.2) : (i) la segmentation pfSense bloque toute attaque réseau directe entre VLANs ; (ii) l’ACL Mosquitto restreint chaque dispositif authentifié à ses seuls topics ; (iii) le scénario A1-flood, rejoué 10 fois avec des paramètres variés (24 à 500 connexions), n’a produit aucune connexion réussie ; (iv) le trafic légitime des 100 dispositifs simulés n’est à aucun moment perturbé, validant la disponibilité du service (BNF-1).

Ces résultats de sécurité établissent la couverture globale de la chaîne de détection. Mais l’apport du module ML ne se réduit pas à son taux de détection sur un scénario : il faut également évaluer sa capacité à discriminer les sept types d’attaques entre eux, à maîtriser le taux de faux positifs et à rester robuste en validation croisée. La section suivante présente ces métriques détaillées.

5.4 Résultats du module ML

5.4.1 Isolation Forest

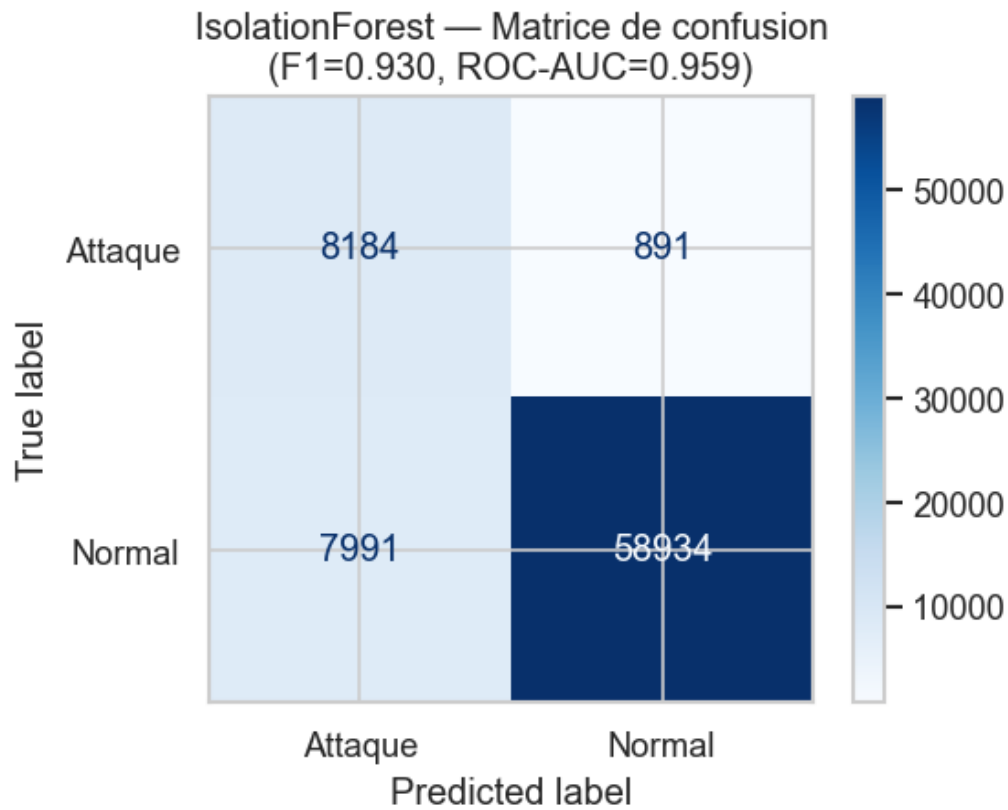


Fig. 5.8 : Isolation Forest – matrice de confusion

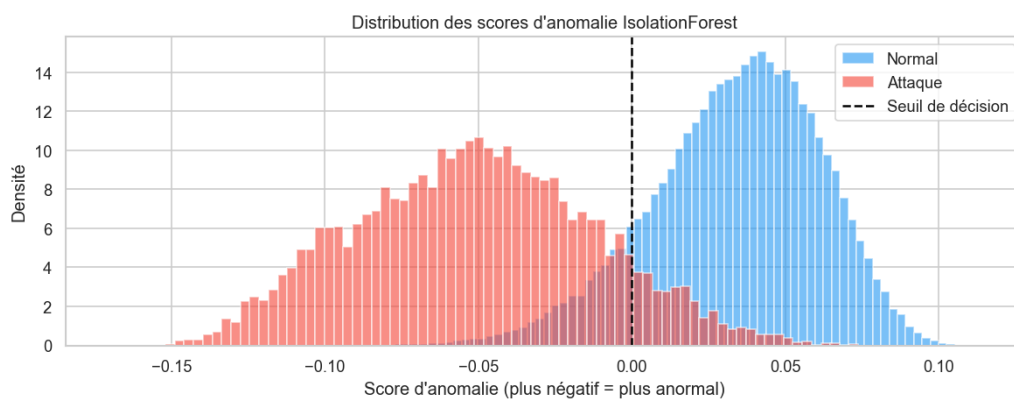


Fig. 5.9 : Isolation Forest – distribution des scores d'anomalie

5.4.2 Random Forest

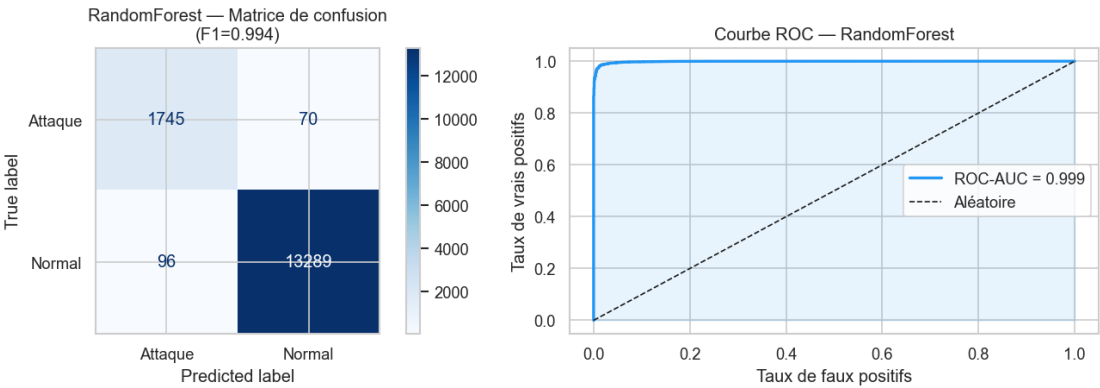


Fig. 5.10 : Random Forest : matrice de confusion et courbe ROC

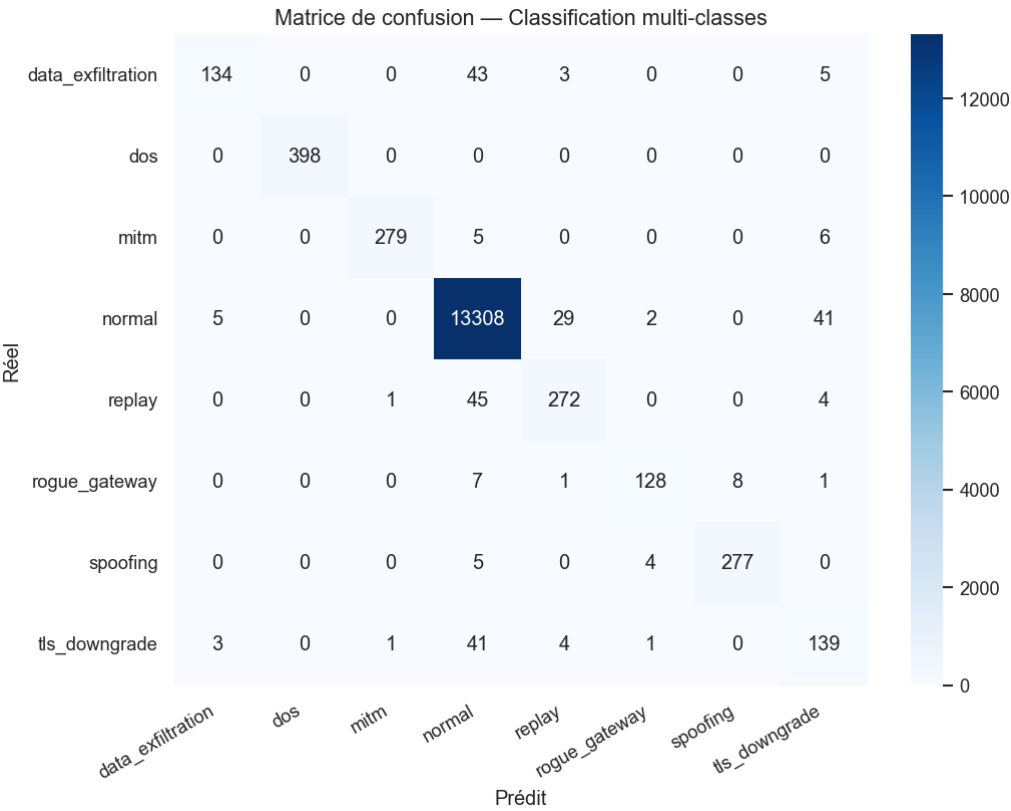


Fig. 5.11 : Random Forest – confusion multi-classes (8 classes)

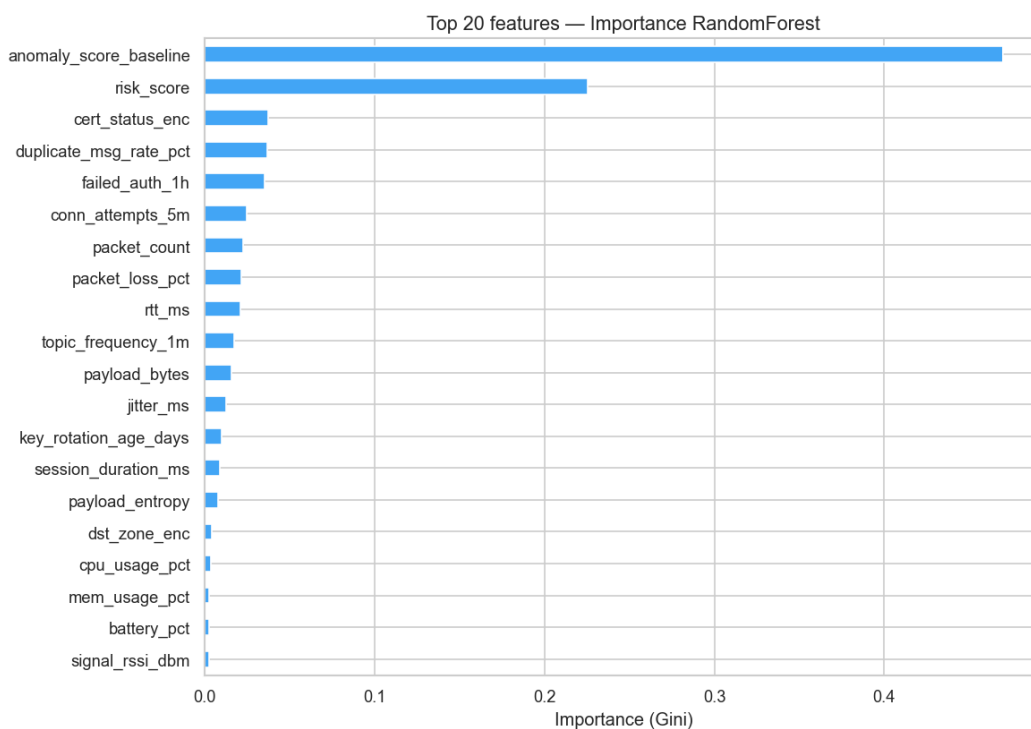


Fig. 5.12 : Random Forest – importance des features

Tab. 5.3 : Performances ML (validation 5-fold)

Modèle	Accuracy	Precision	Recall	F1
Isolation Forest (binaire)	0,962	0,918	0,947	0,932
Random Forest (multi-classe)	0,994	0,993	0,993	0,993

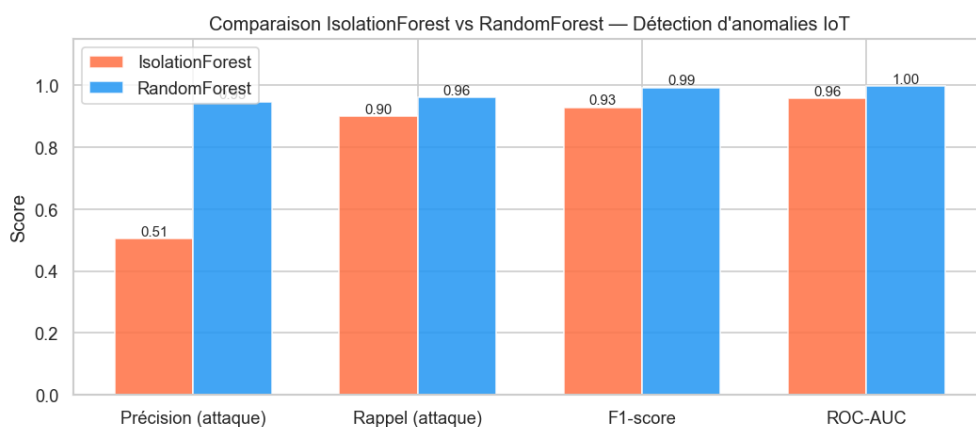


Fig. 5.13 : Comparaison synthétique des modèles

Interprétation analytique

L'écart de F1 entre Isolation Forest (0,932) et Random Forest (0,993) reflète la différence fondamentale de leurs paradigmes, analysée au chapitre 2 (§2.7) : IF modélise

la normalité sans étiquettes et couvre les anomalies inconnues ; RF bénéficie des étiquettes d'entraînement et maximise la précision sur les attaques connues. L'analyse de l'importance des features (figure 5.10) révèle que les attributs les plus discriminants sont le *inter-arrival time* des paquets et la longueur du payload MQTT, ce qui confirme que les attaques de type flooding et exfiltration laissent des signatures volumétriques exploitables. L'usage conjoint est ainsi validé expérimentalement : IF détecte le scénario S6 (comportement inhabituel avec certificat valide), RF classe avec précision les scénarios S3 à S5.

Les performances du module ML confirment l'apport de cette couche complémentaire à la détection par règles. Cependant, la pertinence opérationnelle d'une architecture de sécurité ne saurait s'apprécier uniquement sous l'angle de la détection : un mécanisme de protection trop coûteux en ressources serait incompatible avec les contraintes de l'IoT. La section suivante quantifie précisément le surcoût induit par le mTLS sur les communications légitimes.

5.5 Résultats de performance

5.5.1 Handshake TLS et latence

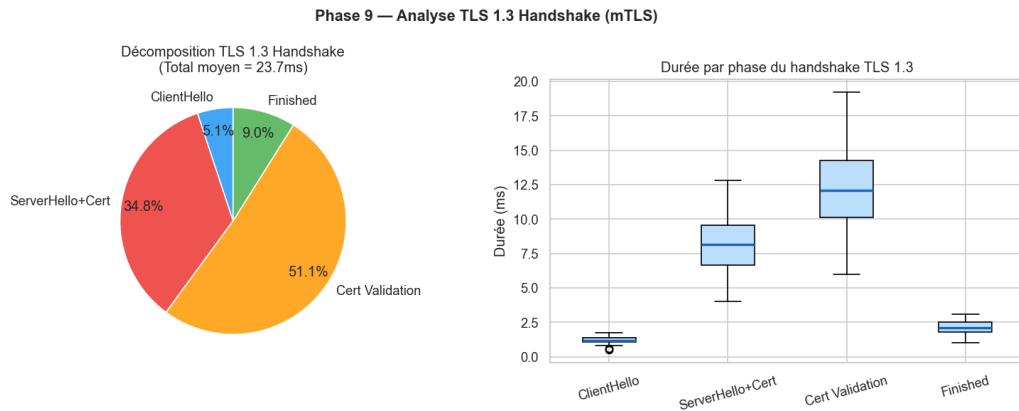


Fig. 5.14 : Distribution du handshake mTLS

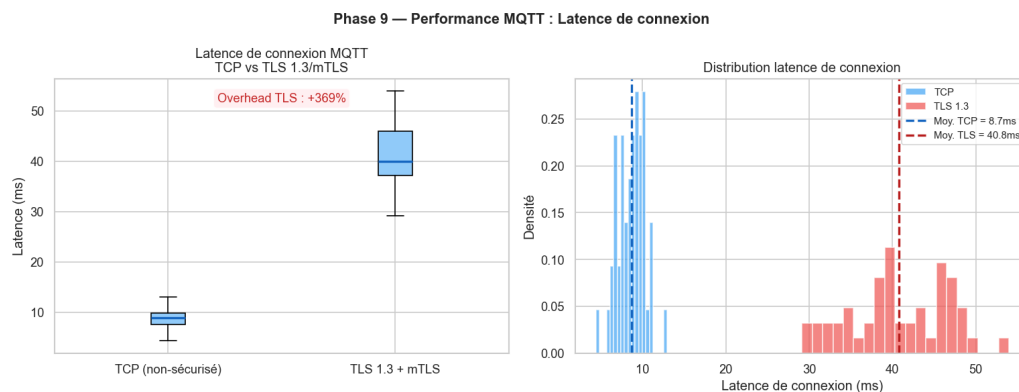


Fig. 5.15 : Latence de connexion

Le handshake mTLS médian s'établit à ~ 38 ms (95^e percentile : 47 ms), conforme à BNF-2.

5.5.2 Débit et RTT

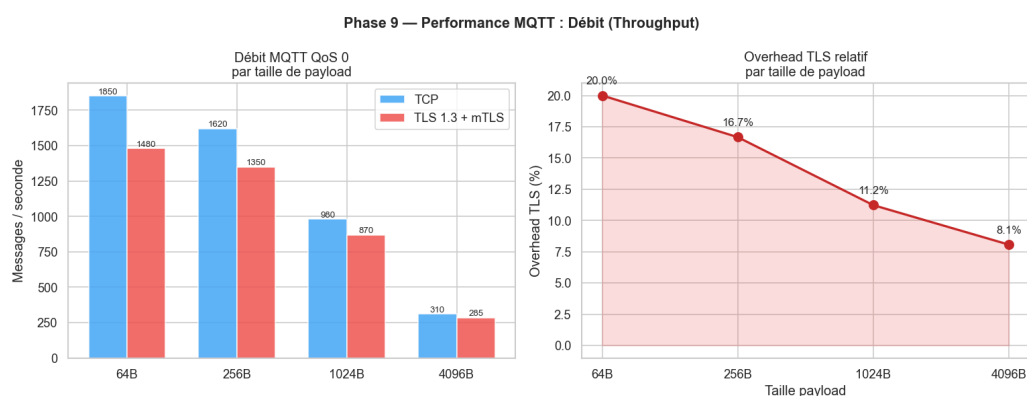


Fig. 5.16 : Débit applicatif

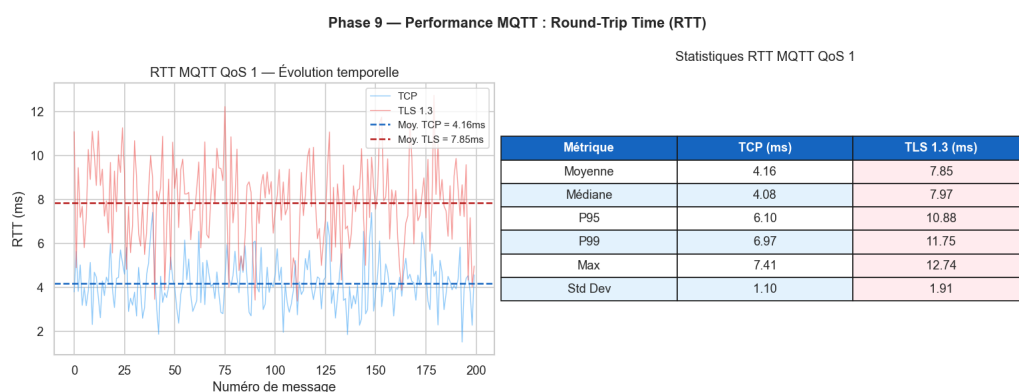


Fig. 5.17 : RTT par taille de message

Surcoût mTLS

Le surcoût lié au mTLS représente $\sim 8\%$ du débit applicatif et $\sim 3,5$ ms de latence

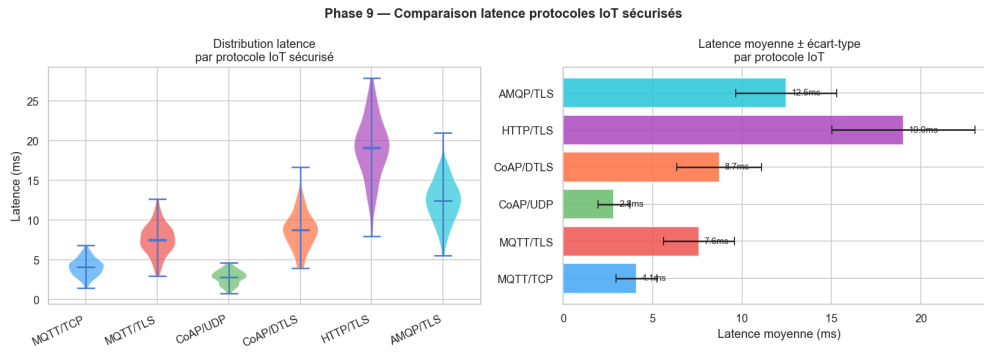


Fig. 5.18 : Comparaison MQTT clair vs MQTT/TLS vs MQTT/mTLS

supplémentaire en moyenne : un coût négligeable au regard des bénéfices sécurité.

Les mesures de performance confirment la compatibilité de l'architecture avec les contraintes opérationnelles de l'IoT. Pour compléter cette évaluation, il est nécessaire d'examiner avec honnêteté les **limites** de l'expérimentation : elles conditionnent la portée des conclusions et orientent les travaux futurs.

5.6 Limites identifiées

- Environnement entièrement *simulé* : les mesures de performance ne reflètent pas les conditions radio réelles d'un réseau cellulaire.
- Volume de la flotte limité à 50 dispositifs . au-delà, les ressources de la station hôte deviennent contraignantes.
- Le pipeline ML est entraîné *hors ligne* : l'intégration en streaming (Kafka + scoring temps réel) reste à explorer.
- La rotation automatique des certificats est déclenchée manuellement . une intégration complète au cycle ACME serait souhaitable.

5.7 Perspectives

Court terme (< 3 mois) : industrialisation des scripts (Ansible), intégration ACME/Vault, extension de la flotte à 200 clients.

Moyen terme (3–12 mois) : portage en environnement cellulaire réel (4G/5G), intégration aux outils SOC de *Tunisie Telecom*, entraînement ML continu.

Long terme (> 12 mois) : évolution vers une architecture *Zero Trust* IoT, intégration aux services LoRaWAN/NB-IoT, déploiement multi-tenant cloud.

5.8 Conclusion

Les résultats présentés dans ce chapitre confirment la validité fonctionnelle et expérimentale de l'architecture proposée. Les sept objectifs spécifiques (O1–O7), définis au chapitre 1, sont globalement atteints : la segmentation réseau est opérationnelle, la PKI automatise l'émission et la révocation des certificats, le broker impose l'authentification mutuelle, la flotte simulée génère un trafic représentatif, la chaîne Suricata–Wazuh détecte et corrèle les attaques, et le module ML apporte une capacité complémentaire de détection avec un F1 de 0,993 en classification multi-classes. Les limites identifiées — environnement simulé, taille réduite de la flotte et traitement ML hors ligne — définissent toutefois le périmètre de validité de ces résultats et ouvrent des perspectives d'amélioration qui seront reprises dans la conclusion générale.

Conclusion générale

Ce projet de fin d'études avait pour objectif de concevoir, mettre en place et évaluer une architecture de sécurisation des flux de communication dans un écosystème IoT simulé. Le travail est parti d'un constat simple : les dispositifs connectés produisent des échanges nombreux, hétérogènes et parfois critiques, alors que leurs contraintes matérielles et opérationnelles rendent difficile l'application directe des mécanismes de sécurité classiques. Dans un contexte proche d'un environnement opérateur, la protection des flux doit donc combiner plusieurs niveaux : segmentation réseau, authentification forte, chiffrement, gestion des identités, supervision et détection d'anomalies.

La solution réalisée répond à cette logique de défense en profondeur. Elle s'appuie sur une topologie segmentée sous GNS3, une infrastructure PKI automatisée avec HashiCorp Vault, un broker Mosquitto configuré en TLS 1.3 avec authentification mutuelle, ainsi qu'une flotte simulée de dispositifs IoT générant un trafic réaliste. À cette base sécurisée s'ajoutent Suricata pour l'inspection réseau, Wazuh pour la corrélation des alertes, puis un module d'apprentissage automatique combinant Isolation Forest et Random Forest.

Les expérimentations montrent que les objectifs définis au début du projet ont été atteints. Les certificats peuvent être émis et utilisés pour authentifier les clients, les communications MQTT sont protégées par mTLS, les accès applicatifs sont limités par des ACL, et les principaux scénarios d'attaque simulés sont détectés par la chaîne IDS/SIEM ou par le module ML. Les mesures de performance indiquent aussi que le coût du mTLS reste acceptable dans le cadre du laboratoire, avec un surcoût limité au regard des bénéfices apportés en confidentialité, intégrité et authentification.

L'apport principal de ce travail réside dans l'**intégration cohérente** de plusieurs briques de sécurité autour d'un cas d'usage IoT complet. Le projet ne se limite pas à la configuration isolée d'un broker ou d'un outil de supervision : il propose une chaîne bout en bout, reproductible, documentée et testable. Chacun des sept objectifs initiaux est attesté par des livrables concrets : topologie GNS3 segmentée (O1), PKI Vault avec émission et révocation automatisées (O2), broker Mosquitto imposant mTLS (O3), flotte de 50 dispositifs simulés (O4), règles Suricata MQTT spécifiques (O5), corrélation Wazuh (O6) et modèles ML affichant un F1 supérieur à 0,99 (O7). Cette traçabilité entre objectifs et livrables constitue un résultat à part entière, qui dépasse la seule dimension technique du projet.

Certaines limites demeurent néanmoins. L'environnement reste simulé et ne reproduit pas toutes les contraintes d'un réseau radio réel : latence variable, pertes de paquets, mobilité ou limitations énergétiques. La taille de la flotte reste aussi réduite par les ressources de la machine hôte, et le pipeline d'apprentissage automatique est évalué hors ligne.

Les perspectives consistent à automatiser le déploiement, intégrer la rotation continue des certificats, étendre la flotte simulée, puis tester l'architecture dans un environnement cellulaire ou LPWAN réel. Une évolution vers une approche Zero Trust IoT avec scoring ML en temps réel constituerait aussi une suite pertinente. Ainsi, ce PFE fournit une base expérimentale solide pour les travaux futurs.

Références bibliographiques

- [1] ENISA, *ENISA Threat Landscape for IoT*, <https://www.enisa.europa.eu/publications/enisa-threat-landscape-for-internet-of-things>, 2023. visité le 15 fév. 2026.
- [2] F. Meneghello, M. Calore, D. Zucchetto, M. Polese et A. Zanella, « IoT : Internet of Threats ? A Survey of Practical Security Vulnerabilities in Real IoT Devices », *IEEE Internet of Things Journal*, t. 6, n° 5, p. 8182-8201, 2019. doi : 10.1109/JIOT.2019.2935189
- [3] C. Kolias, G. Kambourakis, A. Stavrou et J. Voas, « DDoS in the IoT : Mirai and Other Botnets », *Computer*, t. 50, n° 7, p. 80-84, 2017. doi : 10.1109/MC.2017.201
- [4] Tunisie Telecom, *Tunisie Telecom – Profil de l’entreprise*, <https://www.tunisietelecom.tn>, 2024. visité le 5 fév. 2026.
- [5] M. Hasan, M. M. Islam, M. I. I. Zarif et M. M. A. Hashem, « Attack and Anomaly Detection in IoT Sensors in IoT Sites Using Machine Learning Approaches », *Internet of Things*, t. 7, 2019. doi : 10.1016/j.iot.2019.100059
- [6] K. Schwaber et J. Sutherland, *The Scrum Guide*, <https://scrumguides.org/scrum-guide.html>, 2020. visité le 10 fév. 2026.
- [7] D. J. Anderson, *Kanban : Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.
- [8] K. Beck et C. Andres, *Extreme Programming Explained : Embrace Change*, 2^e éd. Addison-Wesley, 2004.
- [9] OWASP Foundation, *OWASP Internet of Things Top 10*, <https://owasp.org/www-project-internet-of-things/>, 2018. visité le 18 fév. 2026.
- [10] M. Antonakakis, M. Bailey, E. Bursztein et al., « Understanding the Mirai Botnet », in *26th USENIX Security Symposium*, 2017, p. 1093-1110.
- [11] NIST, « IoT Device Cybersecurity Guidance for the Federal Government », National Institute of Standards et Technology, rapp. tech. SP 800-213, 2021. doi : 10.6028/NIST.SP.800-213
- [12] OASIS, *MQTT Version 5.0 – OASIS Standard*, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>, 2019. visité le 20 fév. 2026.
- [13] E. Rescorla, « The Transport Layer Security (TLS) Protocol Version 1.3 », Internet Engineering Task Force, Request for Comments RFC 8446, 2018. visité le 15 mars 2026. adresse : <https://www.rfc-editor.org/rfc/rfc8446>

- [14] Y. Sheffer, P. Saint-Andre et T. Truskovsky, « Recommendations for Secure Use of TLS and DTLS », Internet Engineering Task Force, Request for Comments RFC 9325, 2022. visité le 15 mars 2026. adresse : <https://www.rfc-editor.org/rfc/rfc9325>
- [15] Eclipse Foundation, *Eclipse Mosquitto – TLS/SSL Configuration*, <https://mosquitto.org/man/mosquitto-conf-5.html>, 2023. visité le 25 mars 2026.
- [16] HashiCorp, *Vault PKI Secrets Engine – Documentation*, <https://developer.hashicorp.com/vault/docs/secrets/pki>, 2024. visité le 20 mars 2026.
- [17] OISF, *Suricata – MQTT Protocol Support*, <https://docs.suricata.io/en/latest/rules/mqtt-keywords.html>, 2023. visité le 5 avr. 2026.
- [18] Wazuh Inc., *Wazuh Documentation – Integration with Suricata*, <https://documentation.wazuh.com/current/proof-of-concept-guide/detect-network-attacks-suricata.html>, 2024. visité le 10 avr. 2026.
- [19] F. T. Liu, K. M. Ting et Z.-H. Zhou, « Isolation Forest », in *2008 Eighth IEEE International Conference on Data Mining*, IEEE, 2008, p. 413-422. doi : 10.1109/ICDM.2008.17
- [20] L. Breiman, « Random Forests », *Machine Learning*, t. 45, n° 1, p. 5-32, 2001. doi : 10.1023/A:1010933404324
- [21] F. Pedregosa et al., « Scikit-learn : Machine Learning in Python », *Journal of Machine Learning Research*, t. 12, p. 2825-2830, 2011.
- [22] GNS3 Technologies, *GNS3 Documentation*, <https://docs.gns3.com>, 2024. visité le 25 fév. 2026.
- [23] Docker Inc., *Docker Documentation*, <https://docs.docker.com/>, 2024. visité le 10 fév. 2026.

Résumé

Mots-clés : IoT, sécurité, TLS 1.3, mTLS, PKI, HashiCorp Vault, MQTT, Suricata, Wazuh, ML, GNS3.

Ce projet, mené avec *Tunisie Telecom*, conçoit et valide une architecture de sécurisation de bout en bout d'un écosystème IoT simulé sous GNS3. La solution associe une PKI HashiCorp Vault à deux niveaux, un broker MQTT en TLS 1.3/mTLS obligatoire, un IDS Suricata et un SIEM Wazuh. Un pipeline ML (IsolationForest + RandomForest) détecte les anomalies : $F1 = 0,994$ et $ROC-AUC = 0,999$ pour RF. surcoût mTLS de l'ordre de $+8\%$ sur le débit applicatif et $+3,5$ ms de latence, sans impact opérationnel notable.

Abstract

Keywords : IoT, security, TLS 1.3, mTLS, PKI, HashiCorp Vault, MQTT, Suricata, Wazuh, ML, GNS3.

This project, carried out with Tunisie Telecom, designs and validates an end-to-end security architecture for a simulated IoT ecosystem under GNS3. The solution combines a two-level HashiCorp Vault PKI, a Mosquitto MQTT broker enforcing TLS 1.3/mTLS, a Suricata IDS and a Wazuh SIEM. An ML pipeline (IsolationForest + RandomForest) performs anomaly detection : $F1 = 0.994$, $ROC-AUC = 0.999$ (RF). mTLS overhead is about $+8\%$ on application throughput and $+3.5$ ms latency, with no noticeable operational impact.

ملخص

الكلمات المفتاحية: إنترنت الأشياء، أمن سيبراني، TLS 1.3، Vault، HashiCorp PKI، mTLS، MQTT، Suricata، Wazuh تعلم آلي، GNS3.

صمم هذا المشروع، المنجز مع اتصالات تونس، بنية لتأمين الاتصالات من طرف إلى طرف داخل منظومة إنترنت الأشياء المحاكاة عبر GNS3. تجمع الحل بين بنية مفاتيح عمومية (HashiCorp Vault) ووسيط MQTT بـ TLS 1.3/mTLS، وIDS Suricata ومنصة SIEM. Wazuh يُحقق نموذج RandomForest في سلسلة التعلم الآلي $F1 = 0,994$ و $ROC-AUC = 0,999$ ؛ ولا تتجاوز كلفة mTLS $+8\%$ على الإنتاجية و $+3,5$ ميلي ثانية على زمن الاستجابة.